

Neural Networks and Deep Learning

Konstantinos Makantasis
konstantinos.makantasis@um.edu.mt



L-Università ta' Malta
Faculty of Information &
Communication Technology

Department
of Artificial
Intelligence

Contents

PART 1

- The Learning Framework
- Neural Networks History
- Feedforward Neural Networks
- Activation Functions
- Regularization Techniques
- Training Deep Neural Networks

PART 2

- Loss functions
- Performance metrics
- Popular Architectures
- Practical methodology

Contents

PART 1

- The Learning Framework
- Neural Networks History
- Feedforward Neural Networks
- Activation Functions
- Regularization Techniques
- Training Deep Neural Networks

PART 2

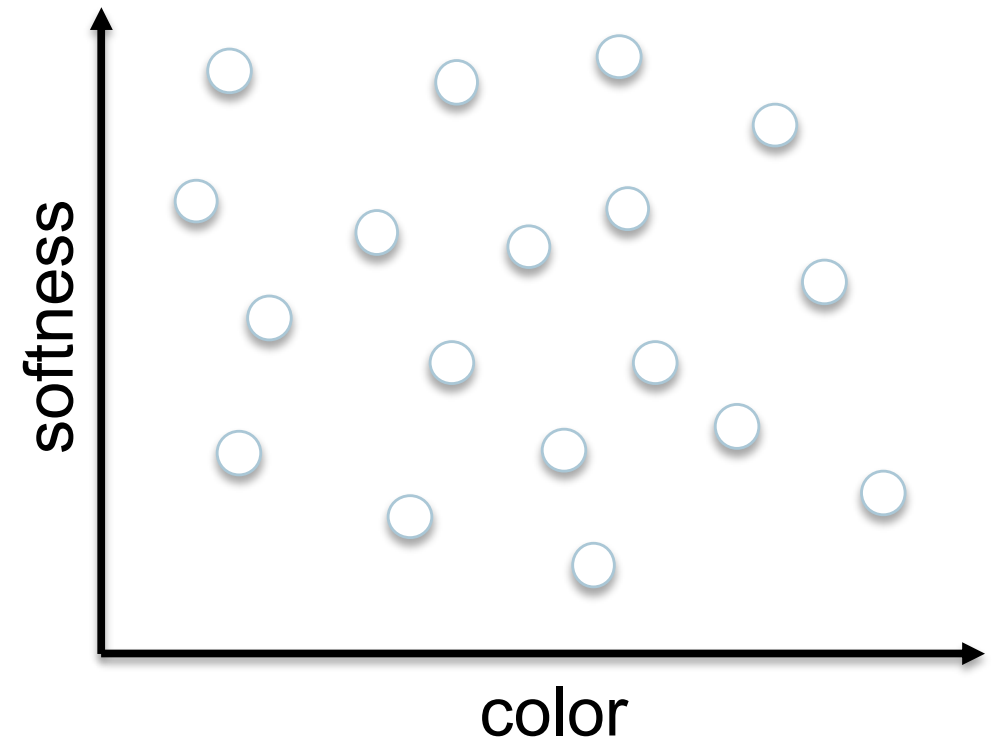
- Loss functions
- Performance metrics
- Popular Architectures
- Practical methodology

The Learning Framework

Learner's input:

- Domain set $\mathcal{X} \rightarrow$ features space
- Label set $\mathcal{Y}$ e.g. $\mathcal{Y} = \{0,1\}$
- Training data $S = \{(x_i, y_i)\}_{i=1}^m$

Learn the taste of a fruit based
on softness and color



The Learning Framework

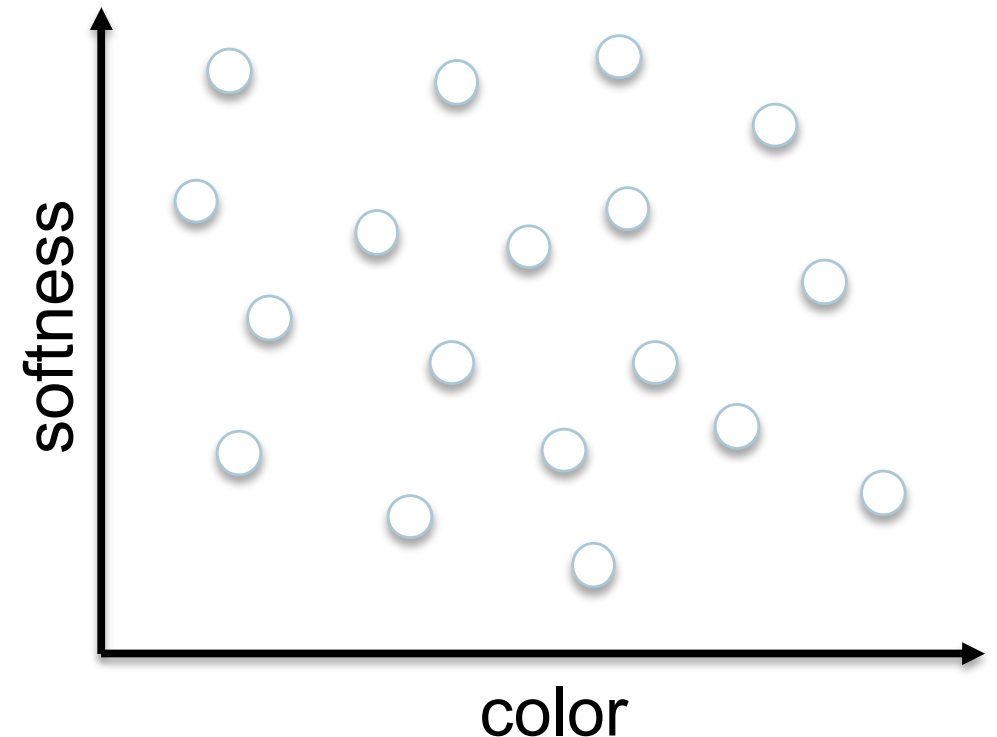
Learner's input:

- Domain set $\mathcal{X} \rightarrow$ features space
- Label set $\mathcal{Y}$ e.g. $\mathcal{Y} = \{0,1\}$
- Training data $S = \{(x_i, y_i)\}_{i=1}^m$

Learner's output:

- Prediction rule $h \in \mathcal{H}: \mathcal{X} \rightarrow \mathcal{Y}$

Learn the taste of a fruit based
on softness and color



The Learning Framework

Learner's input:

- Domain set $\mathcal{X} \rightarrow$ features space
- Label set $\mathcal{Y}$ e.g. $\mathcal{Y} = \{0,1\}$
- Training data $S = \{(x_i, y_i)\}_{i=1}^m$

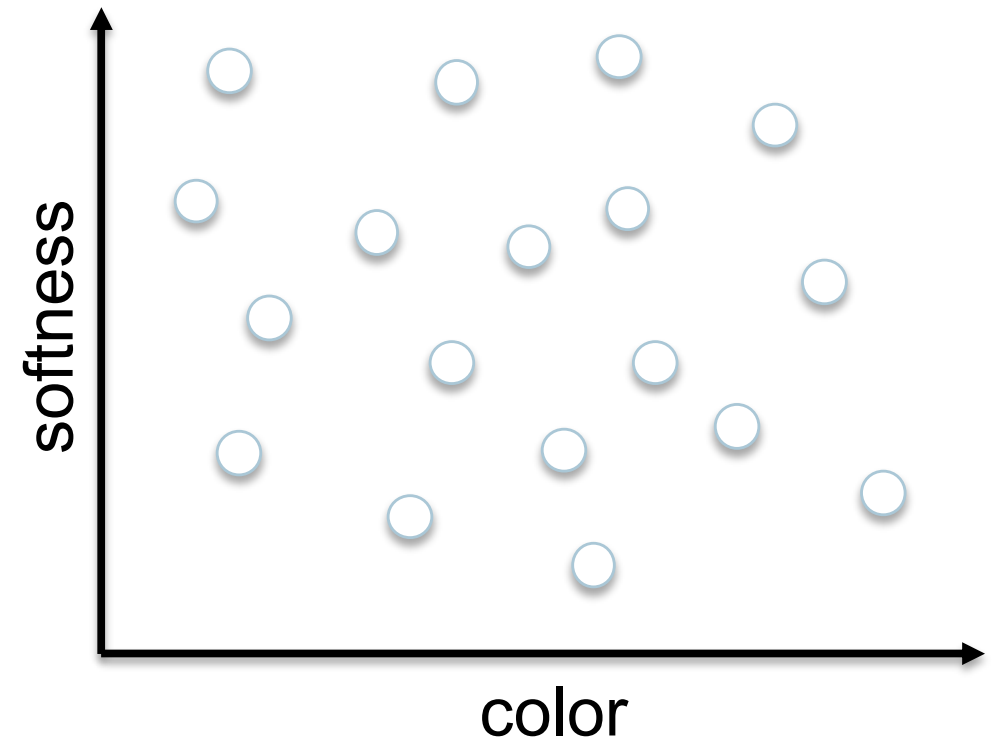
Learner's output:

- Prediction rule $h \in \mathcal{H}: \mathcal{X} \rightarrow \mathcal{Y}$

Measures of Success:

- Classification accuracy, MSE, etc.

Learn the taste of a fruit based
on softness and color

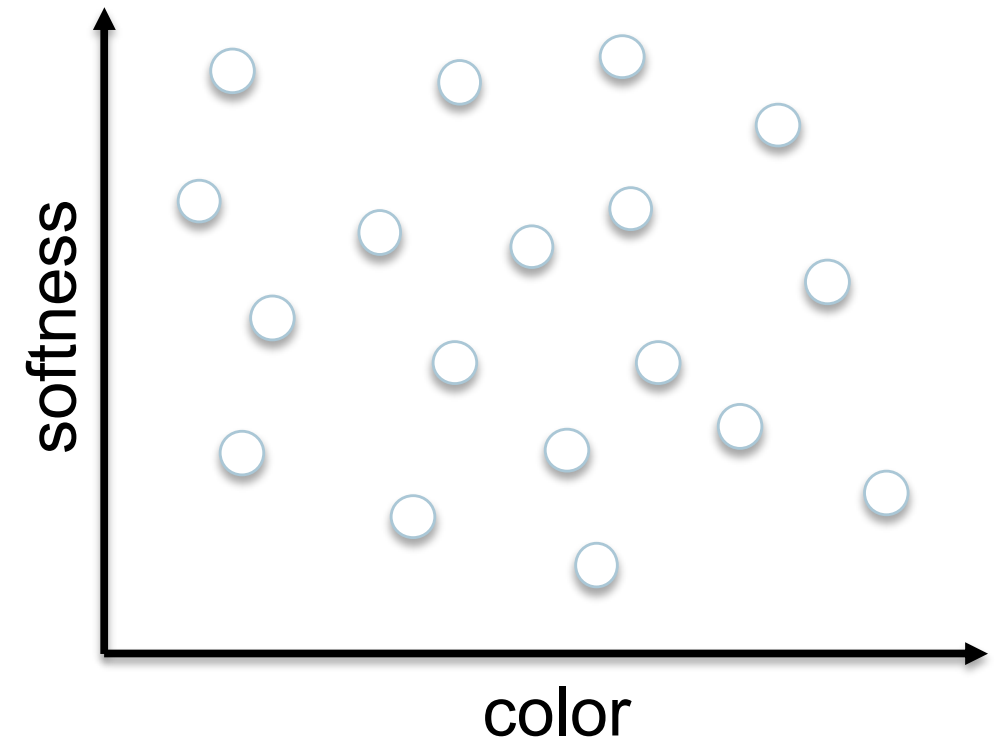


The Learning Framework

How the prediction rule can be derived?

- Minimize training error

Learn the taste of a fruit based
on softness and color



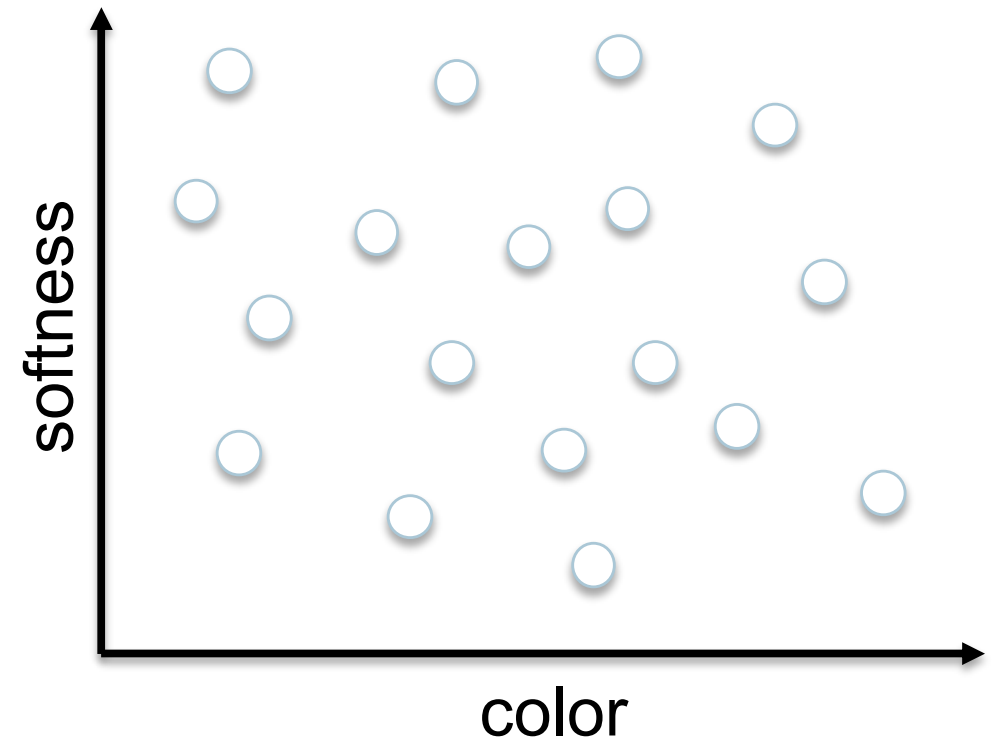
The Learning Framework

How the prediction rule can be derived?

- Minimize training error

$$h_S(x) = \begin{cases} y_i & \text{if } \exists i \in [m] \text{ s.t. } x_i = x \\ 0 & \text{otherwise.} \end{cases}$$

Learn the taste of a fruit based
on softness and color



The Learning Framework

How the prediction rule can be derived?

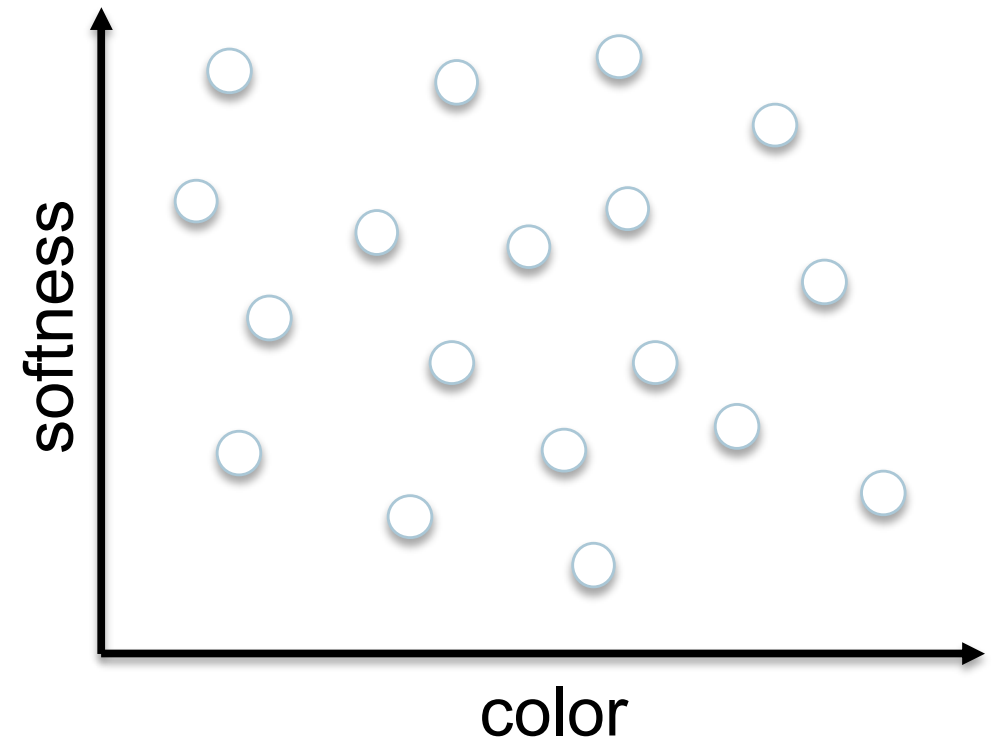
- Minimize training error

$$h_S(x) = \begin{cases} y_i & \text{if } \exists i \in [m] \text{ s.t. } x_i = x \\ 0 & \text{otherwise.} \end{cases}$$

Overfitting

Not every family of functions $\mathcal{H}$ can be used for learning problems

Learn the taste of a fruit based on softness and color

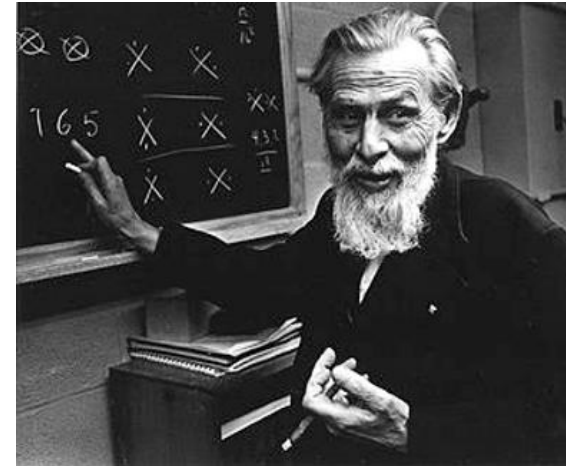


Neural Networks History

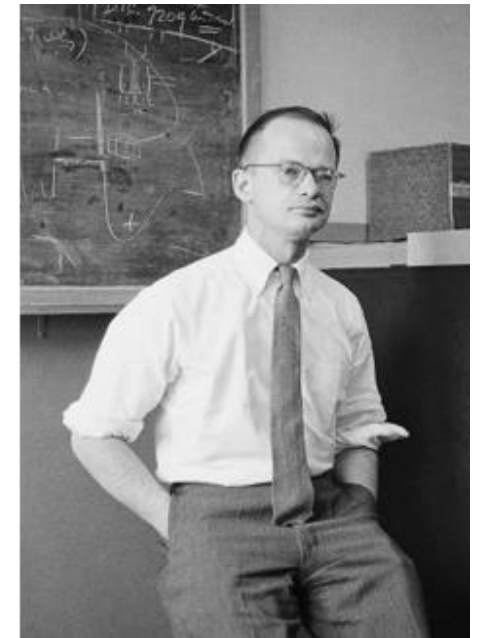
Historical Notes

McCulloch and Pitts (1948):

- General concept of neural networks
- Hierarchy of single processing unit (neurons) which work together and exchange information to complete complex tasks.



Warren McCulloch



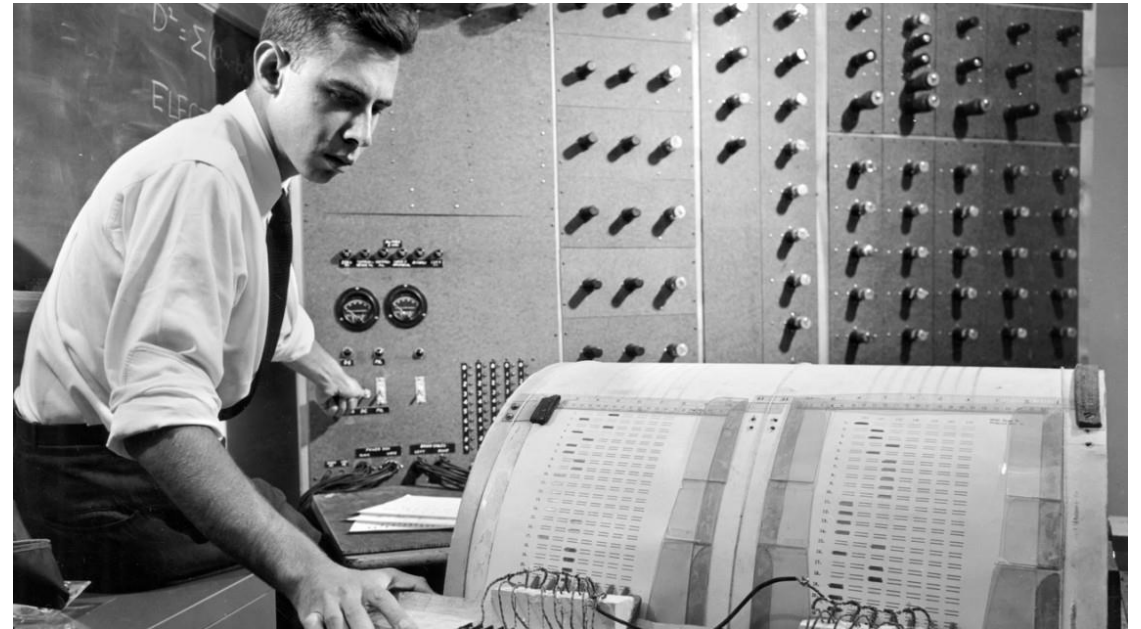
Walter Pitts

Neural Networks History

Historical Notes

Rosenblatt (1958)- Perceptron:

- The first learning model that was exclusively data-driven
- No assumption about the distribution of the data
- Excitement - Strong AI



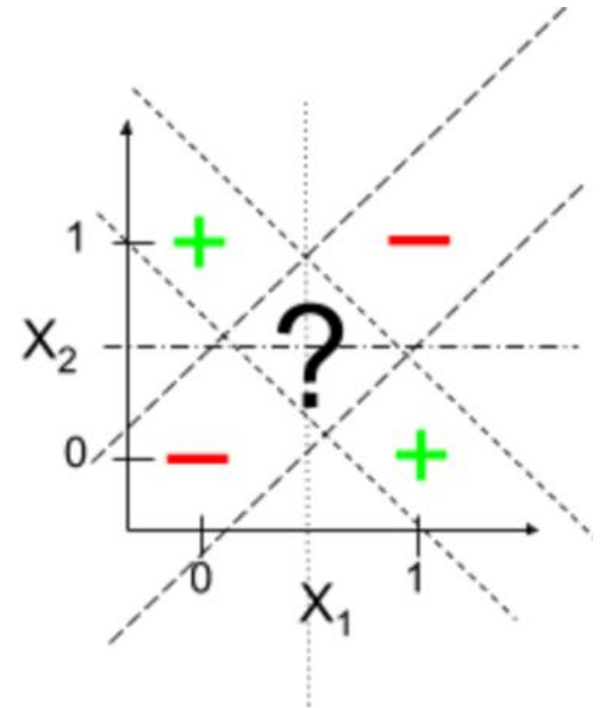
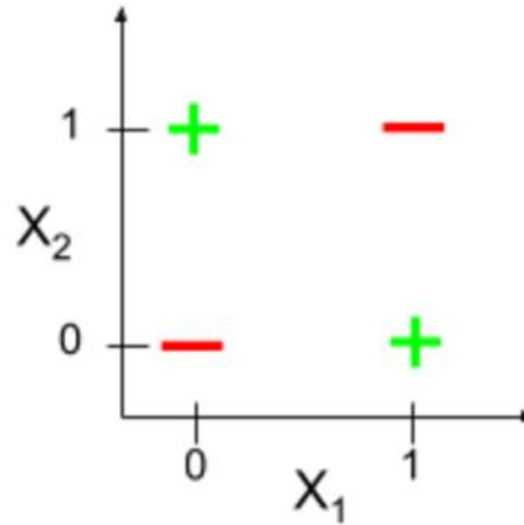
<https://news.cornell.edu/stories/2019/09/professors-perceptron-paved-way-ai-60-years-too-soon>

Neural Networks History

Historical Notes

Minsky & Papert paper (1969):

- Perceptron limitations
- Learning XOR
- Backlash against the entire neural network approach



Neural Networks History

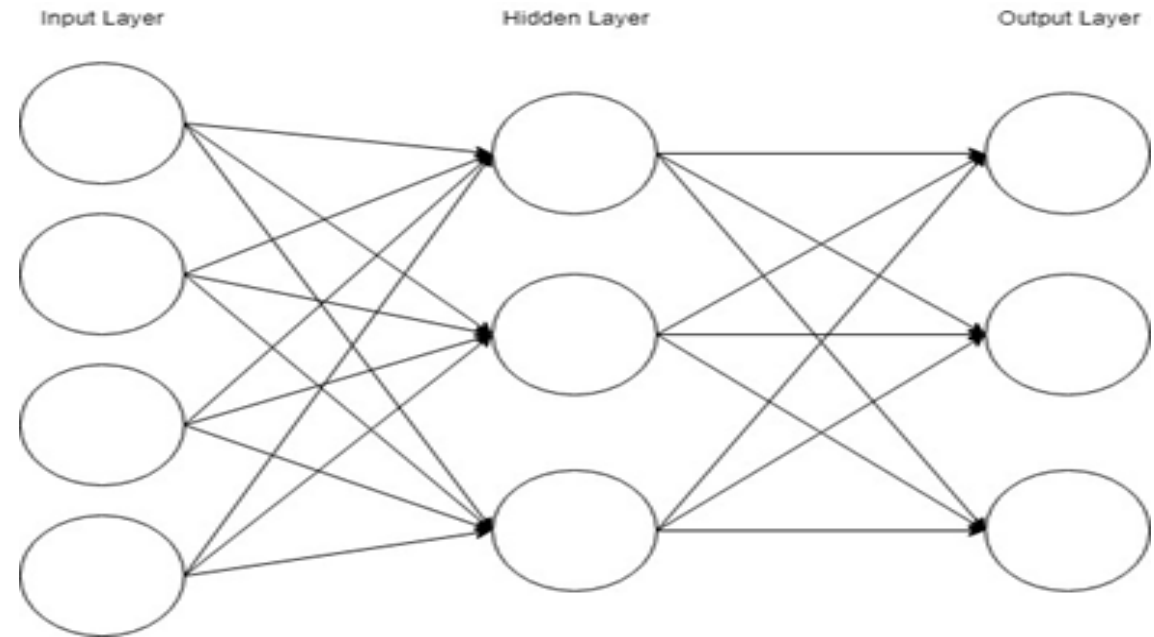
Historical Notes

Multilayer perceptron:

- Nonlinear (XOR solved)
- Universal approximation properties
- Backpropagation algorithm (1986)

Applicability:

- Computational cost
- Algorithmic Problems



<https://deepai.org/machine-learning-glossary-and-terms/multilayer-perceptron>

Neural Networks History

Historical Notes

1990 – 2000 – Simple low-complexity problem solvers

- Recurrent neural networks (Hochreiter and Schmidhuber, 1996)
- Advanced gradient descent (Schaback and Werner, 1992)
- Convolutional neural networks (LeCunn, 1998)

2000 – Era of deep learning

- Auto-encoders (Bengio, 2009)
- GPU acceleration (Chellapilla et al. 2006)
- Image Net (Krizhevsky, 2012)
- Foundation models and LLMs

Neural Networks History

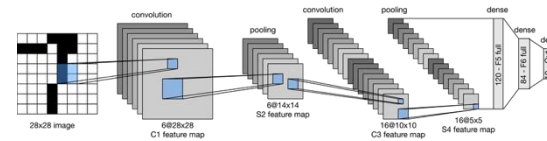
Historical Notes



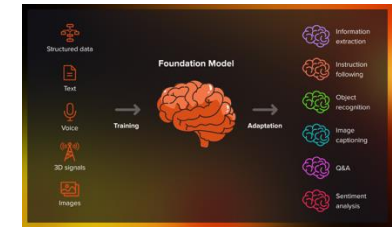
1950: Computing Machinery and Intelligence



1969: Perceptrons: an introduction to computational geometry



1998: Convolutional Neural Networks



2020: Foundation Models

Early Ideas – Theoretical Foundations

1957: Perceptron



AI Winter

1986: Back-Propagation algorithm



Revival and Expansion

2012: AlexNet Deep Learning

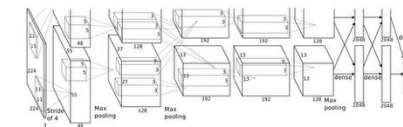
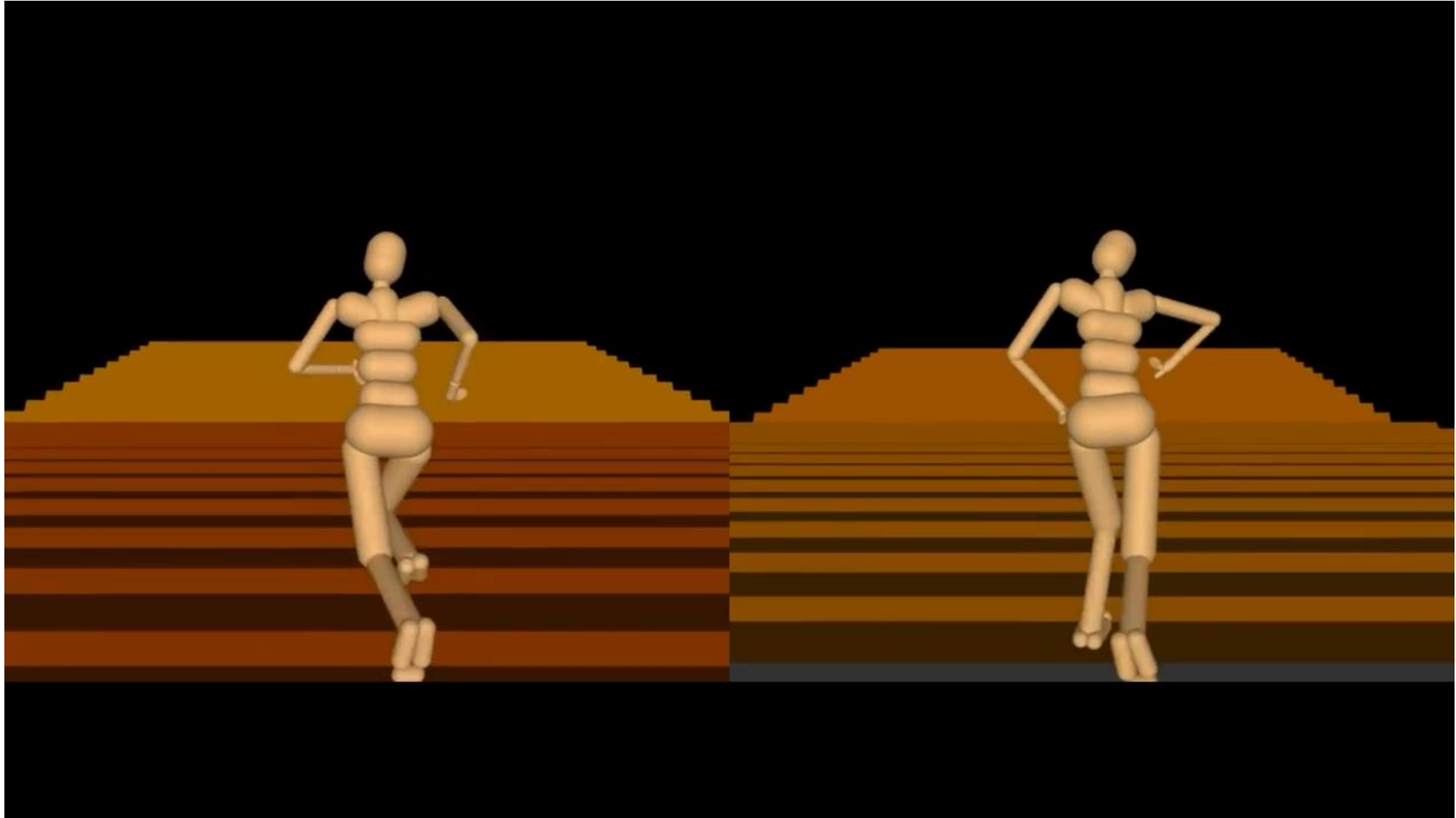
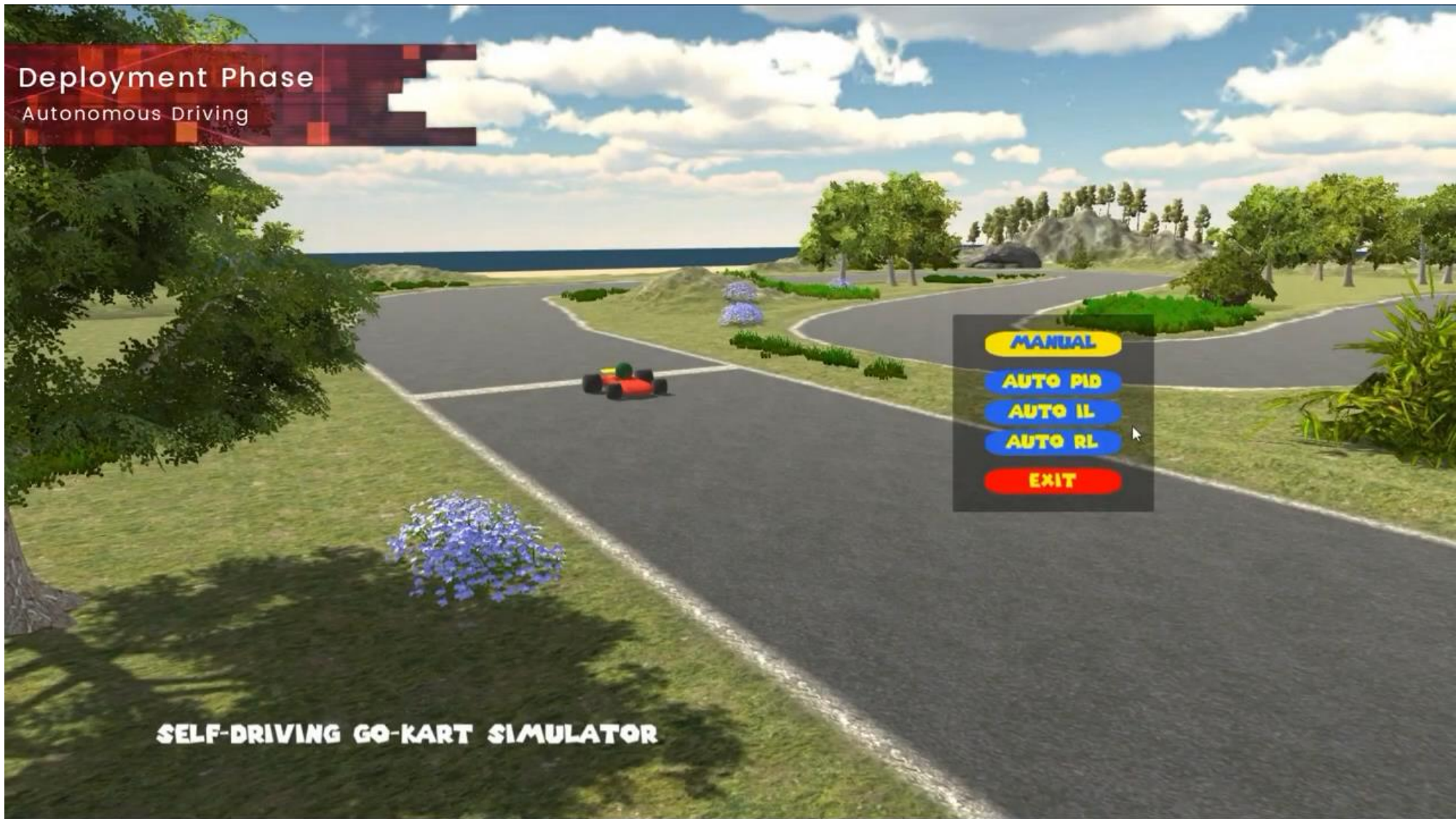


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 255,440-186,624-64,896-64,896-43,264-4096-4096-1000.

Exploitation: Era of Big Data and Deep Learning



Merel, Josh, et al. "Learning human behaviors from motion capture by adversarial imitation." *arXiv preprint arXiv:1707.02201* (2017).



<https://www.youtube.com/channel/UCOKTr4-UCZAd7Y56bKQWbvQ>

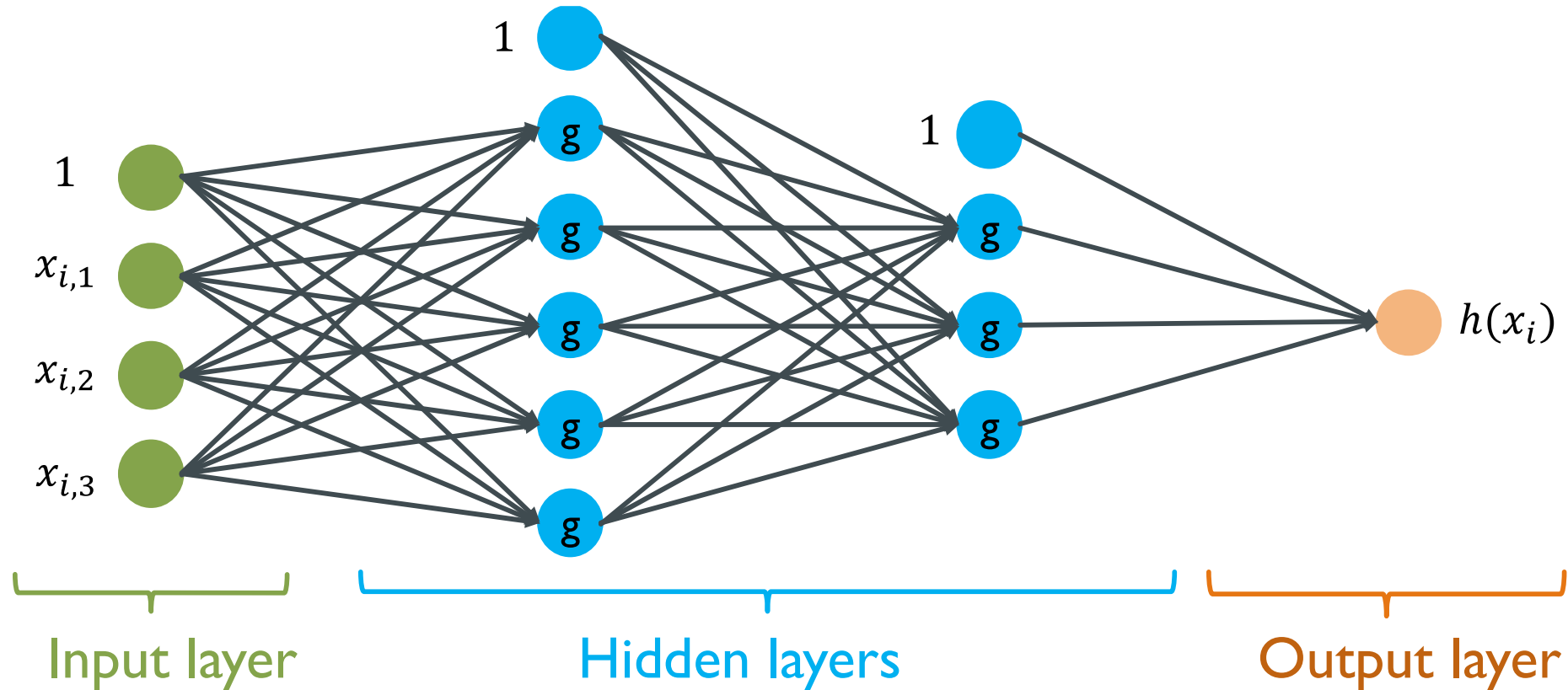






<https://github.com/michalnand/>

Feedforward Neural Networks



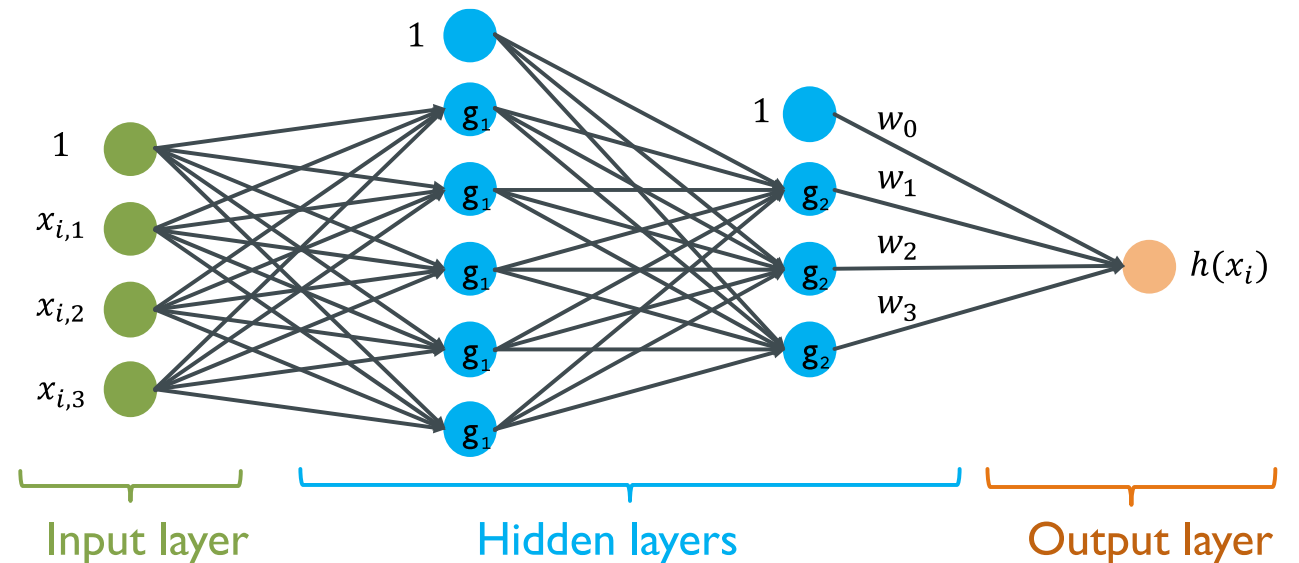
Feedforward Neural Networks

Output of a network

$$h(x_i) = f([w_1, w_2, w_3]^T g_2(z_1) + w_0)$$

The function f can be:

- the **Identity**,
- the **Logistic**,
- the **Softmax**.



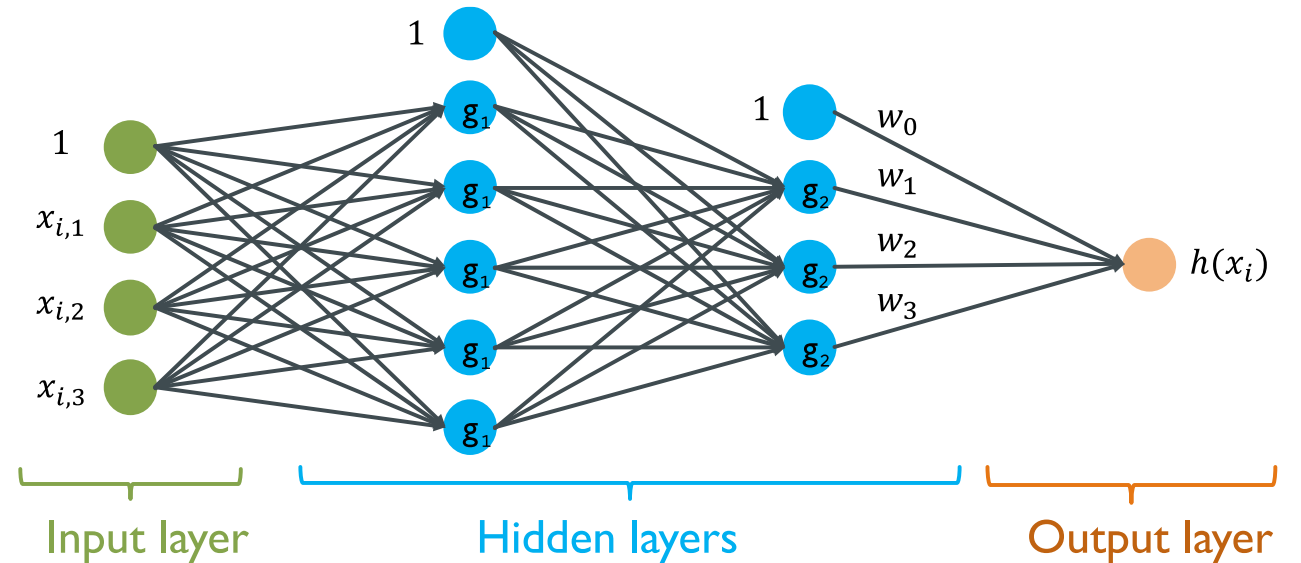
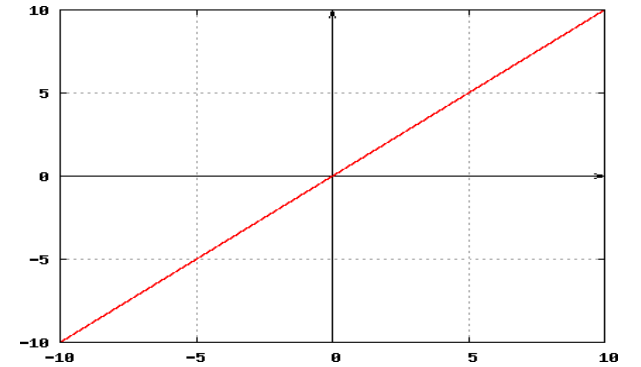
Feedforward Neural Networks

Output: Identity

$$h(x_i) = [w_1, w_2, w_3]^T g_2(z_1) + w_0$$

The output is a **real** number α_i .

- Regression problems
- $a_i = \mathbb{E}[y_i | x_i]$
- Mean Squared Error



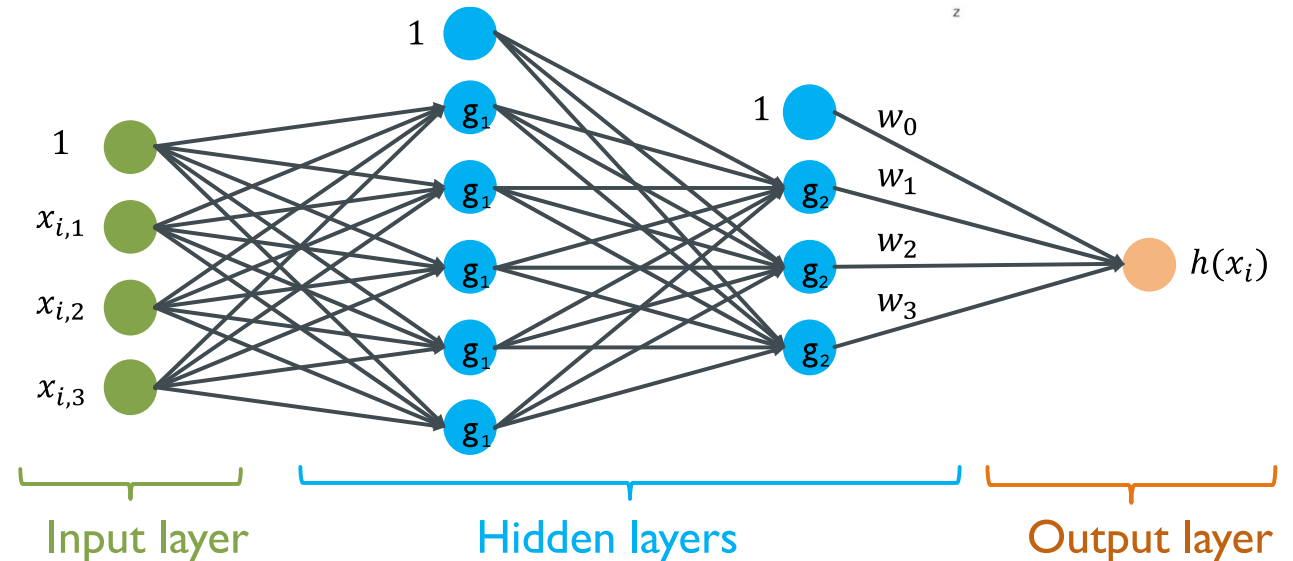
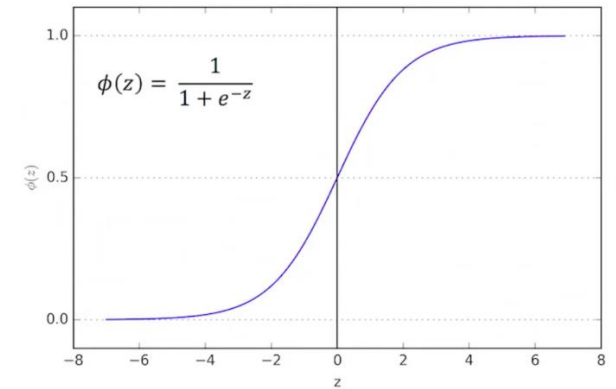
Feedforward Neural Networks

Output: Logistic

$$h(x_i) = \sigma([w_1, w_2, w_3]^T g_2(z_1) + w_0)$$

The output is a number $\alpha_i \in (0,1)$.

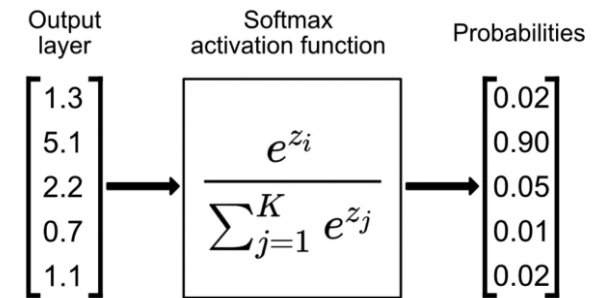
- Binary classification problems
- $a_i = \mathbb{P}[y_i = 1|x_i]$
- Negative log-likelihood



Feedforward Neural Networks

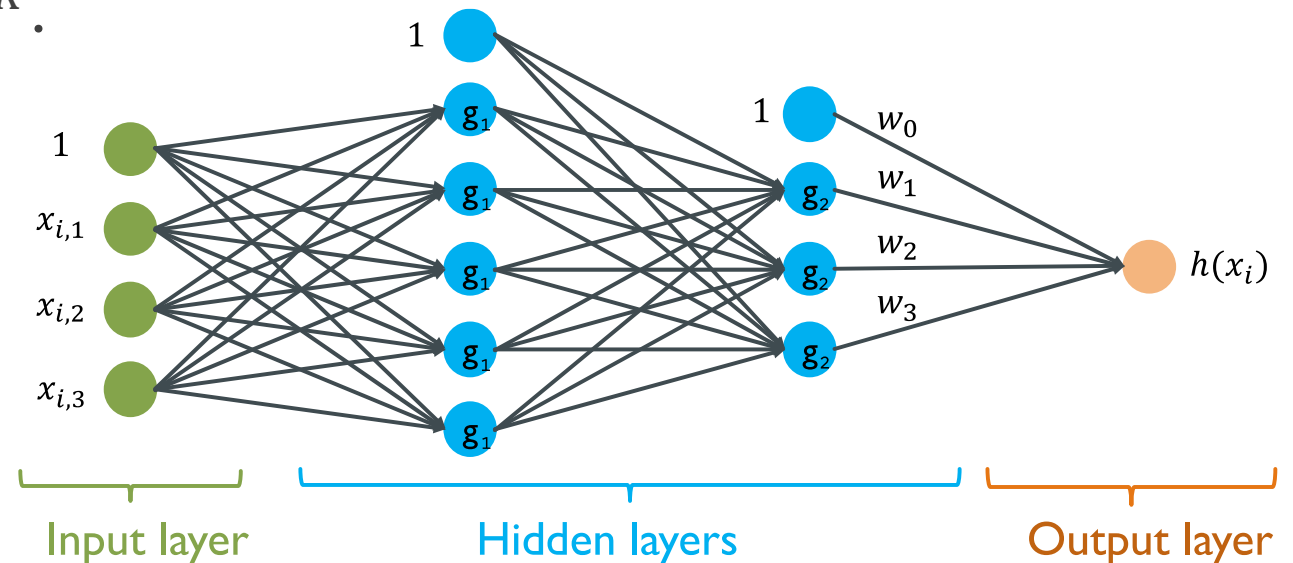
Output: Softmax

$$h(x_i) = S([w_1, w_2, w_3]^T g_2(x_i) + w_0)$$



The output is a vector $\alpha_i \in \{(0,1)\}^K$.

- Classification problems
- $a_i^{(k)} = \mathbb{P}[y_i^{(k)} = 1 | x_i]$
- Negative log-likelihood



Feedforward Neural Networks

Hidden Layers and Units

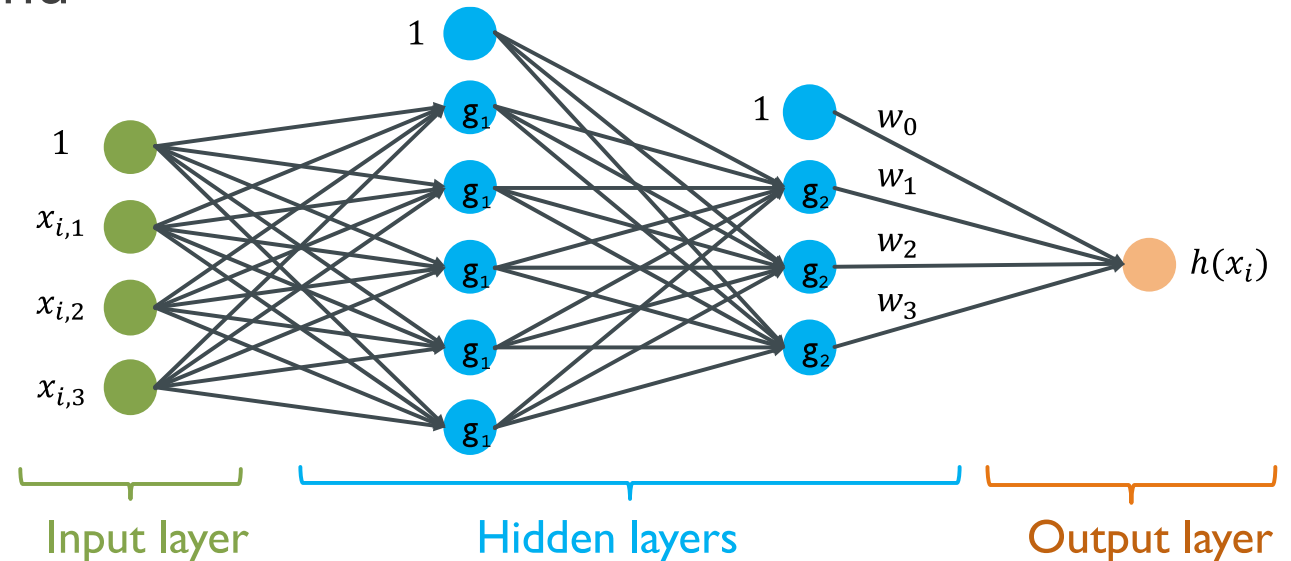
- Propagate input to output
- Non-linear input transformations

Transformations are **data driven** and are not “seen”

- Hence the term “hidden”

Non-linear transformations:

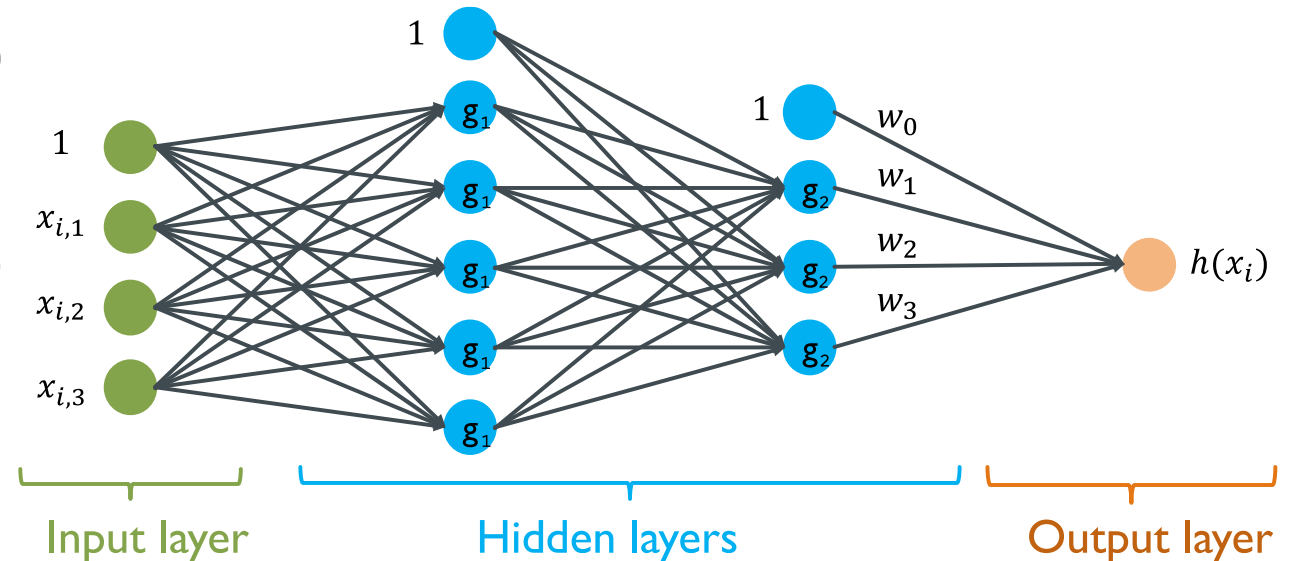
- Logistic and tanh (before deep learning era)
- ReLU variations (during the deep learning era)



Feedforward Neural Networks

Architecture design

- Chain of layers
 - Number of layers (**depth**)
- Each layer is a set of neurons
 - Number of neurons (**width**)
- Activation functions
- Optimization (algorithm & rate)
- Loss function



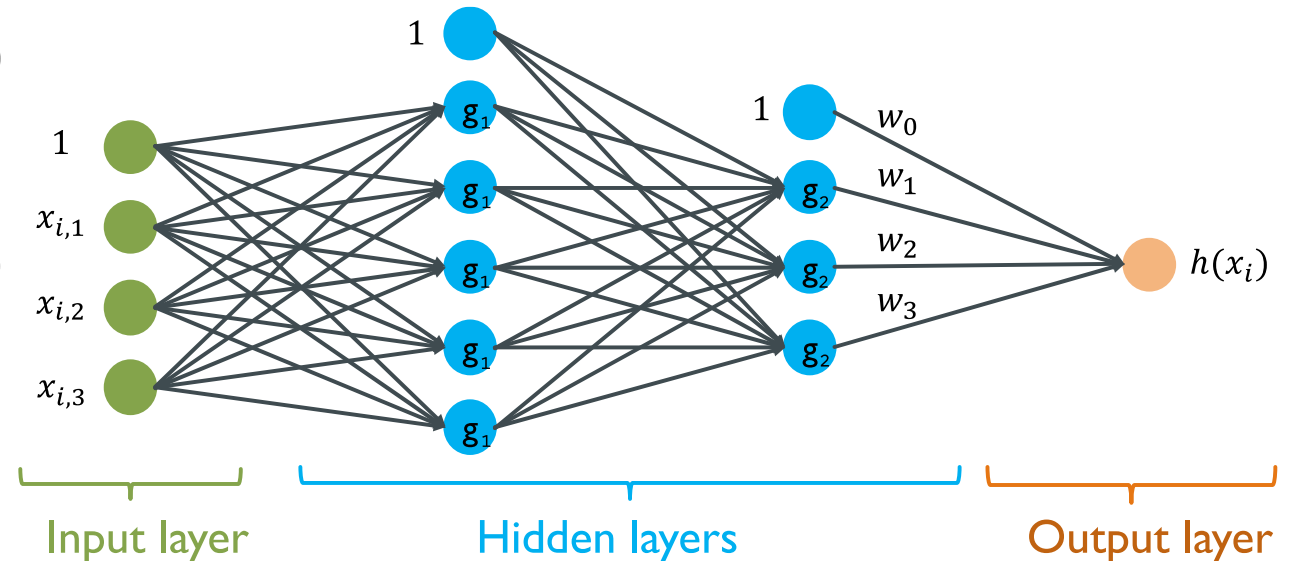
Feedforward Neural Networks

Architecture design

- Chain of layers
 - Number of layers (**depth**)
- Each layer is a set of neurons
 - Number of neurons (**width**)
- Activation functions
- Optimization (algorithm & rate)
- Loss function

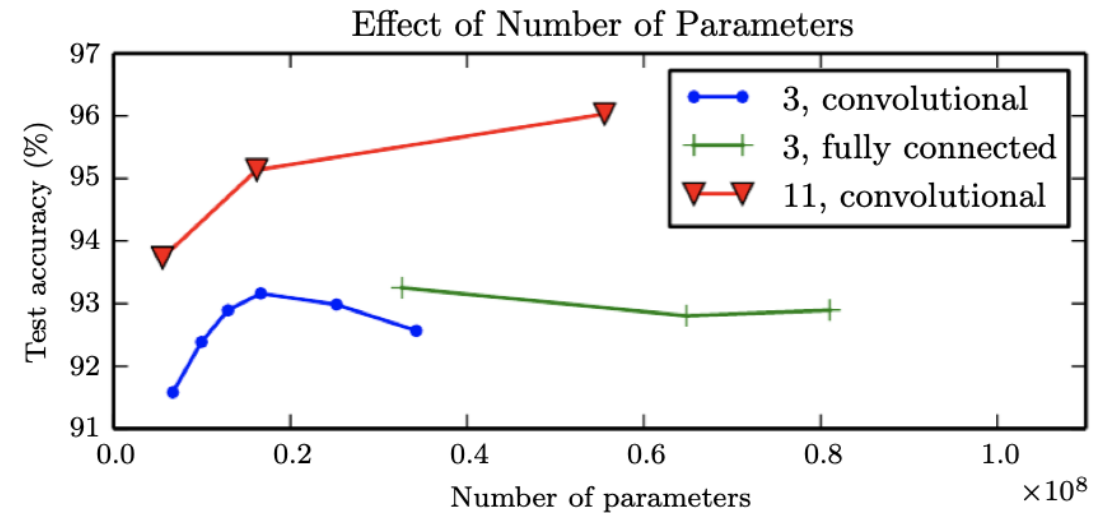
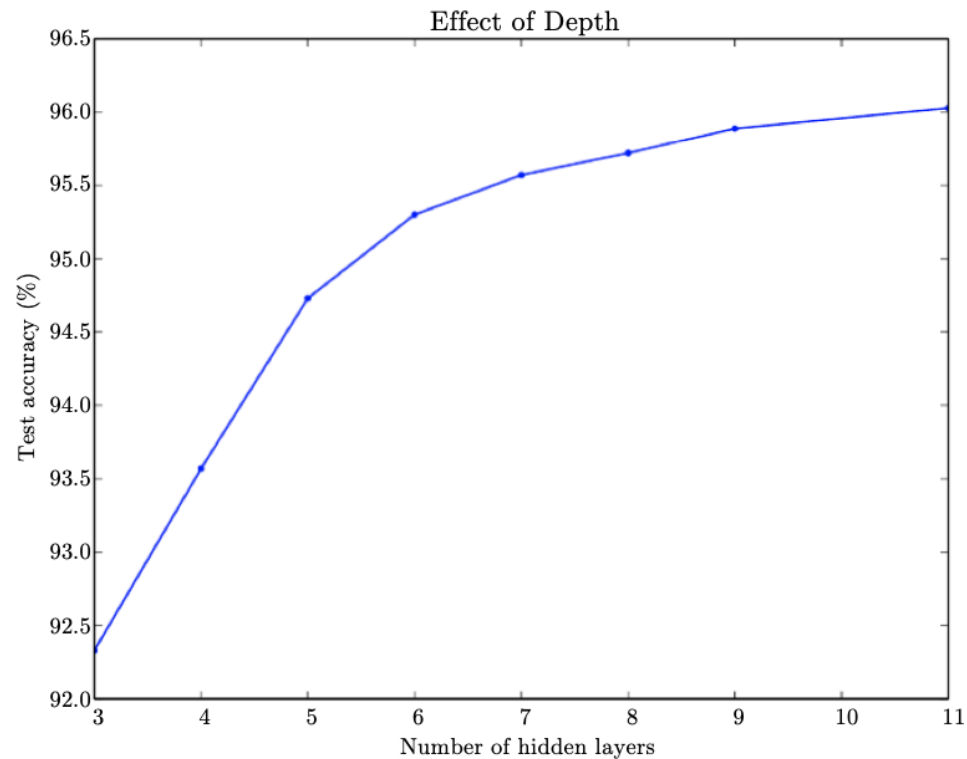
Design choices:

- Empirically
 - Trial-and-error - Application at hand
- Rules of thumb
 - Overfitting/underfitting



Feedforward Neural Networks

Architecture design: depth vs width

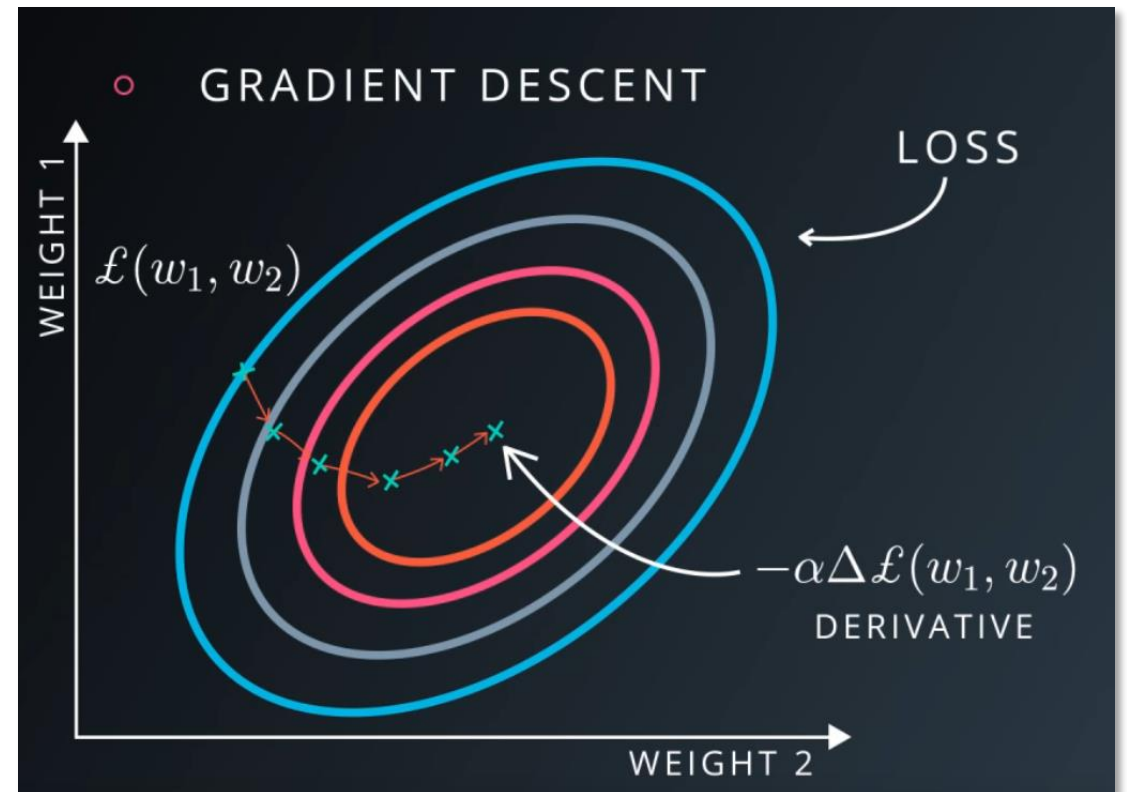


Both images are taken from: Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.

Feedforward Neural Networks

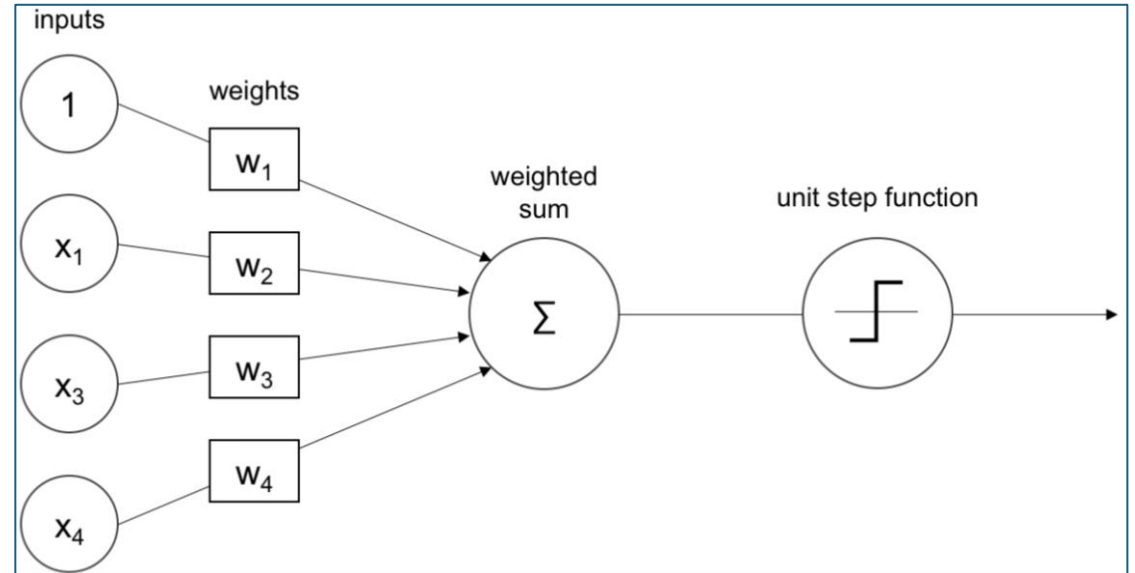
Gradient based optimization

- Randomly initialize weights
- Decide on the loss function $J(w)$
- Back-propagation to compute the gradients of $J(w)$ w.r.t. w
- Move towards the negative direction of the gradient
- Repeat until satisfied
- **Local minima**



Activation Functions

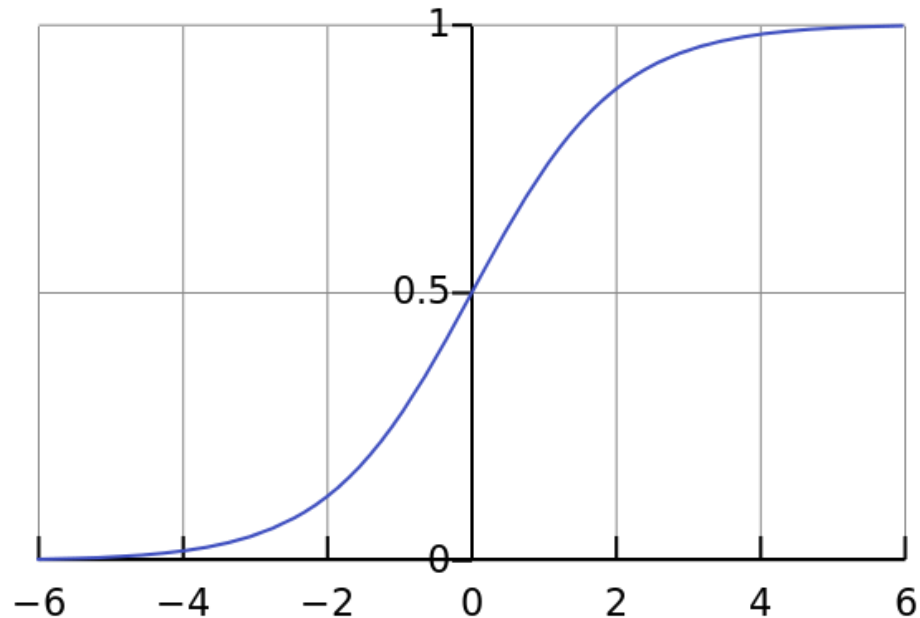
- **Determine the output** of a neuron
 - Indicates when a neuron “fires”
 - Nonlinear (why?)
 - Affect gradient based optimization algorithm
 - Differential and continuous
 - Derivative at every point



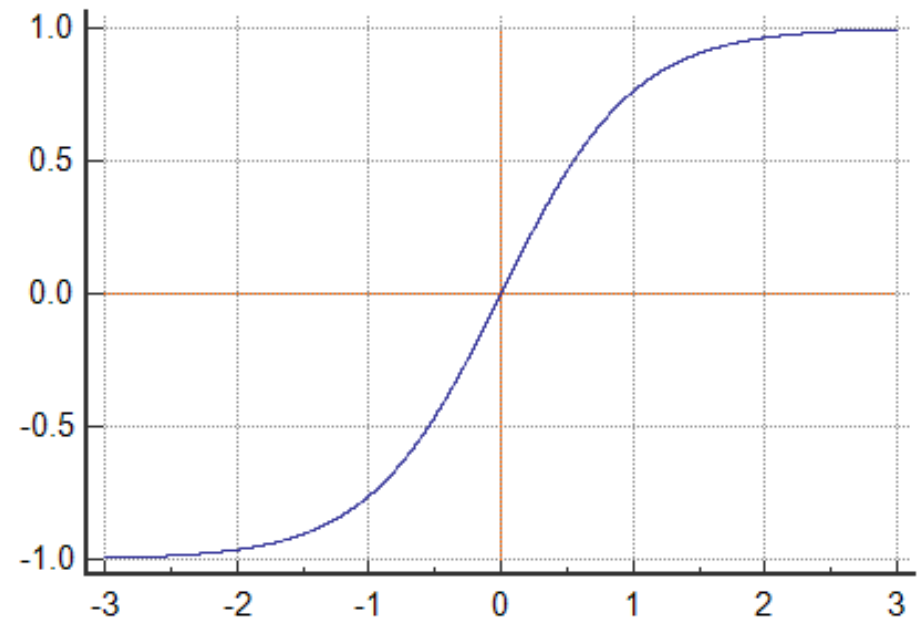
Activation Functions

Sigmoid activations: logistic & tanh

$$\text{Logistic: } A = \frac{1}{1+e^{-z}}$$



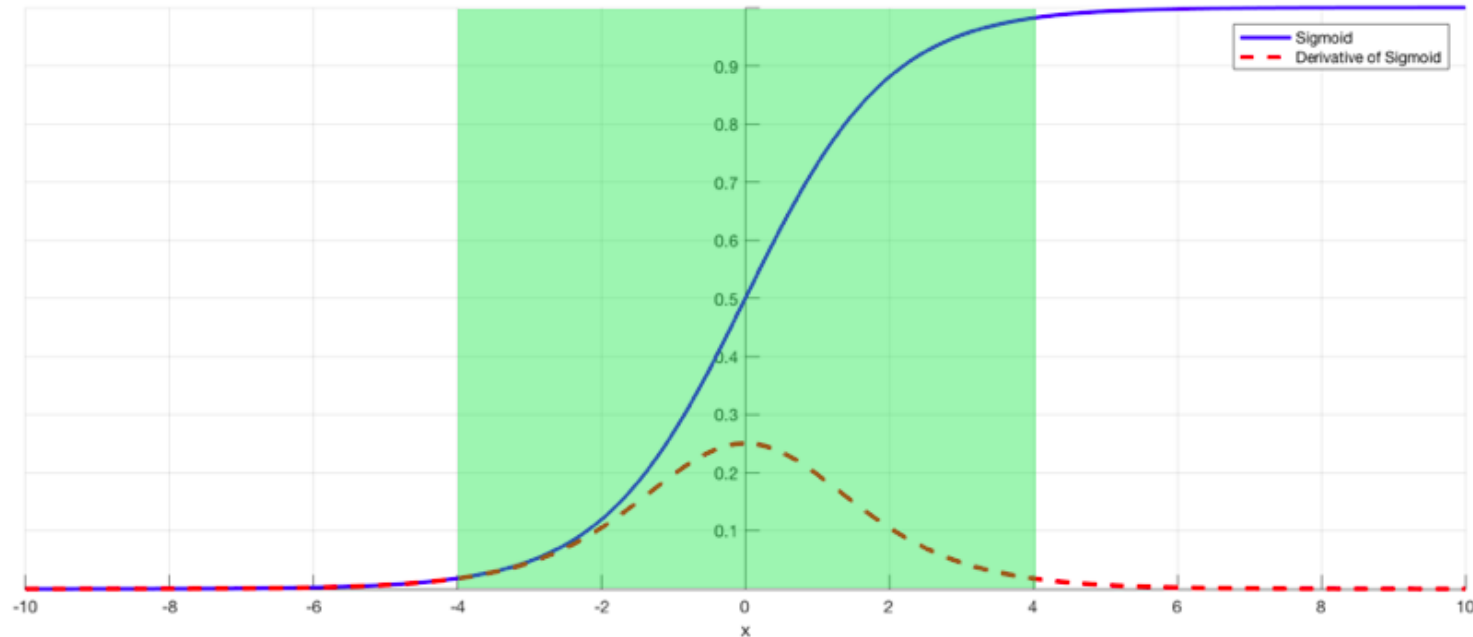
$$\text{Tanh: } A = \frac{2}{1+e^{-2z}} - 1$$



Activation Functions

Sigmoid activations: logistic & tanh → vanishing gradient

As more layers are added to neural networks, the gradients of the loss function **approaches zero**, making the network hard to train.

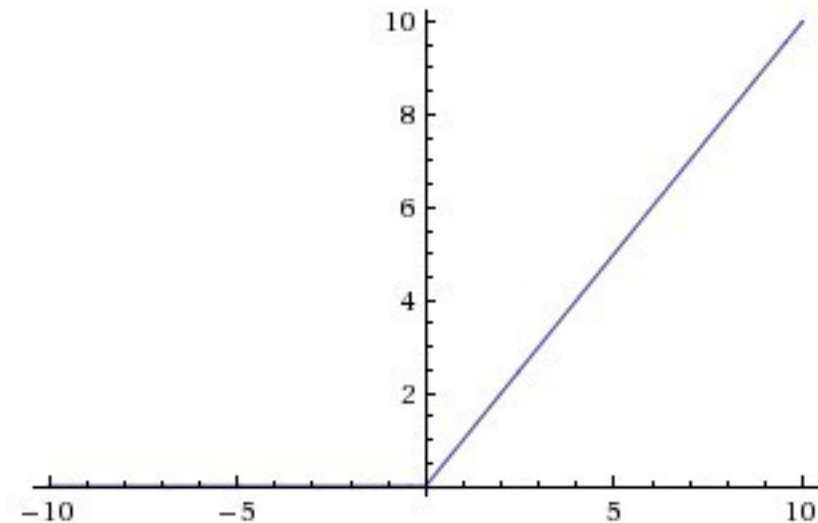


Activation Functions

Rectified Linear Units (ReLU)

- Sigmoids fire all the time, making an ANN heavy.
- ReLU maps input x to $\max(0, x)$
- ReLU lets big numbers pass.
- Faster training: doesn't fire all the time and less computationally expensive.

$$f(z) = \max(0, z)$$



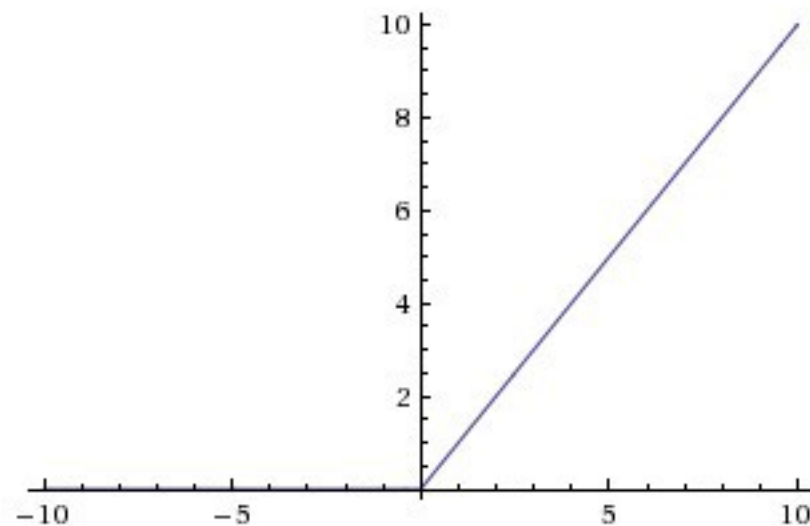
Activation Functions

Rectified Linear Units (ReLU)

- Sigmoids fire all the time, making an ANN heavy.
- ReLU maps input x to $\max(0, x)$
- ReLU lets big numbers pass.
- Faster training: doesn't fire all the time and less computationally expensive.

Cannot learn via gradient-based methods on examples for which their activation is zero.

$$f(z) = \max(0, z)$$



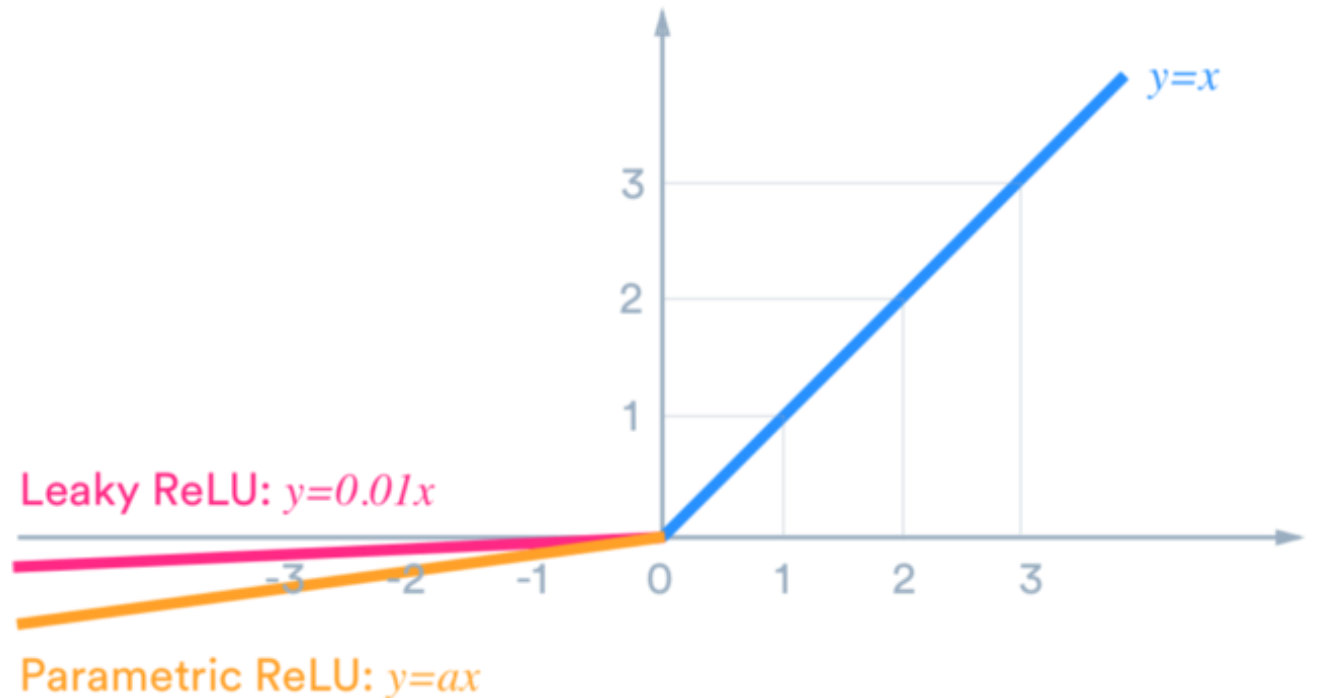
Activation Functions

Leaky RELU

- Allows for a parametric configuration for values of $z < 0$.
- Predefined a value

Parametric ReLU (PReLU)

- a is a learnable parameter



Activation Functions

RELU-6

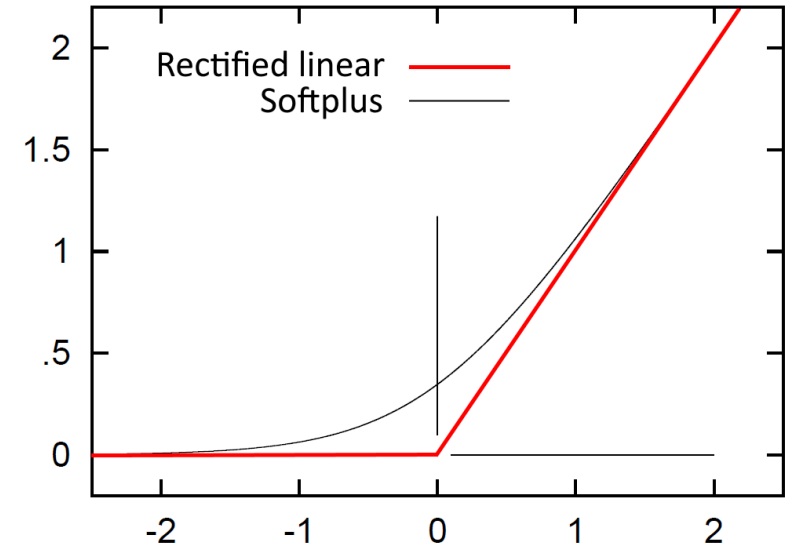
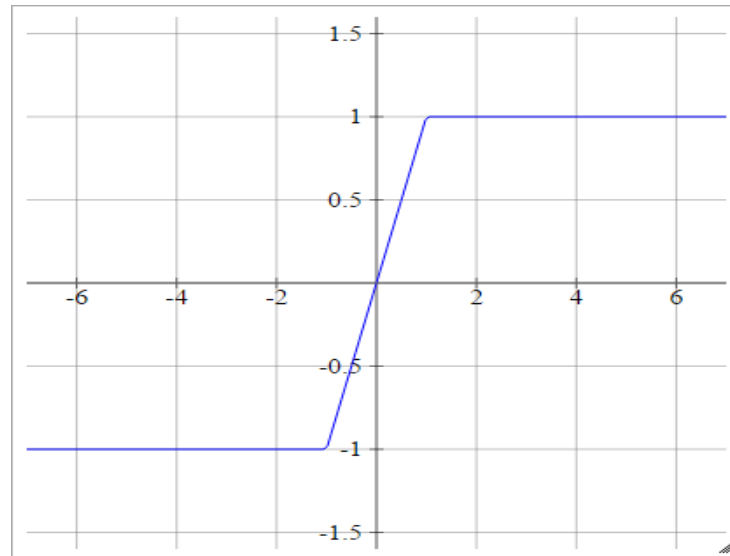
- ReLU has limits $[0, \infty]$ that can also keep pushing activations and this can also be less favourable for the network.
- Solution: Clip z at a value n .
- 6 is an arbitrary value generally used for n .
- Compression



Activation Functions

Other activation functions

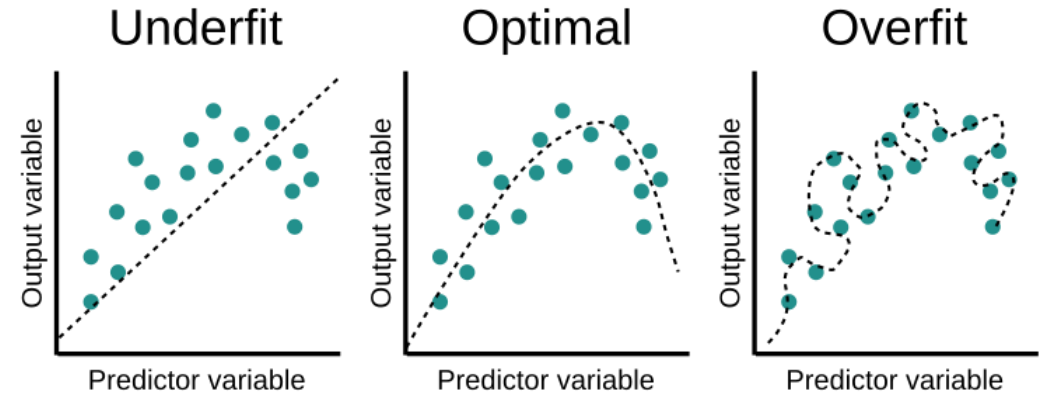
- **Softplus**
 - Smooth version of ReLU
 - Empirically underperforms compared to ReLU
- **Hard Tanh**



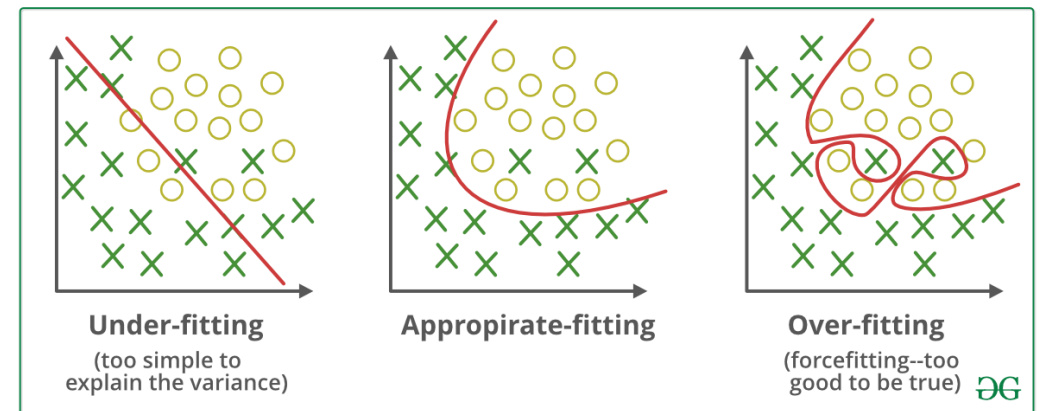
Reguralisation

Why do we need regularization?

- Avoid overfitting
- Increase generalisation
- At the expense of increased training error
- One of the major research efforts in the field



<https://www.educative.io/answers/overfitting-and-underfitting>



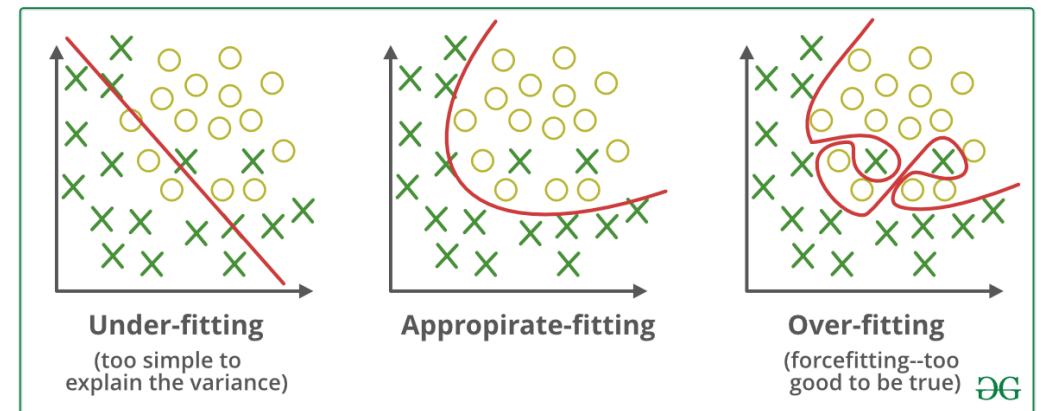
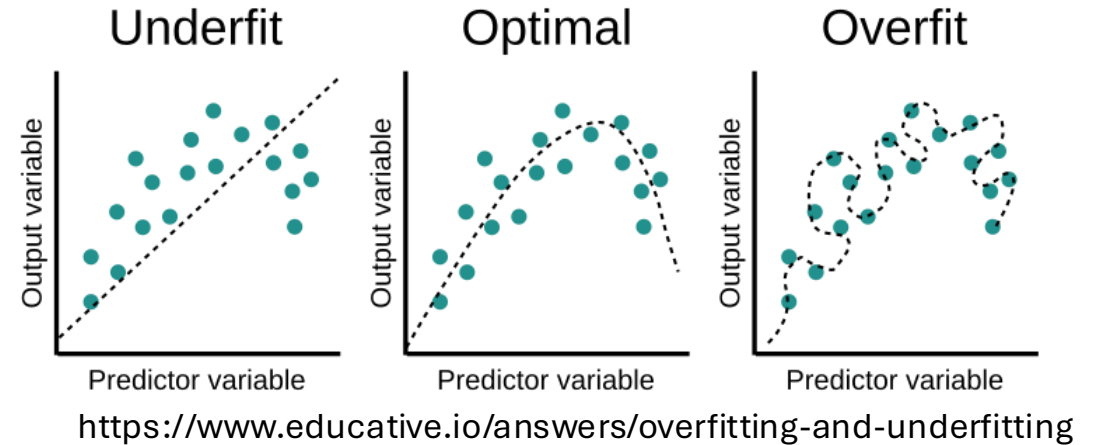
<https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/>

Reguralisation

Parameter Norm Penalties

$$\bar{J}(w; X, y) = J(w; X, y) + \alpha \Omega(w)$$

- L^2 parameter regularization
- $\Omega = \frac{w^T w}{2}$
- Differentiable
- Favours small weights values
- **Very small but not zero**

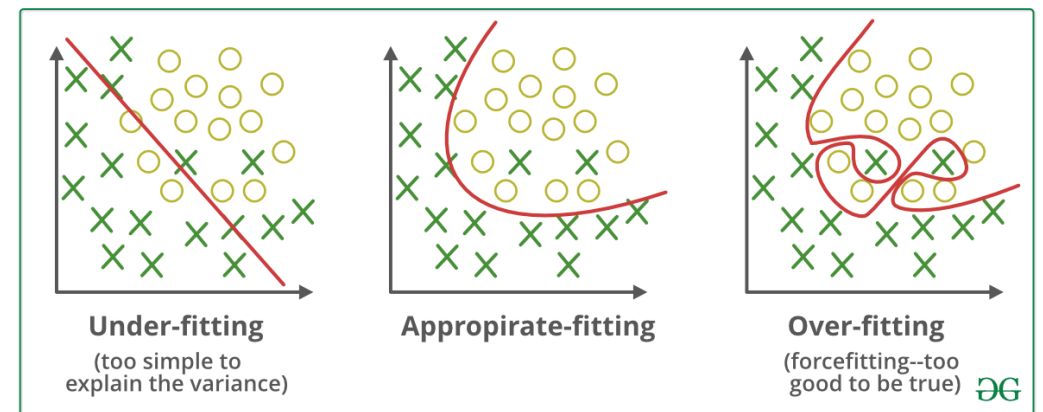
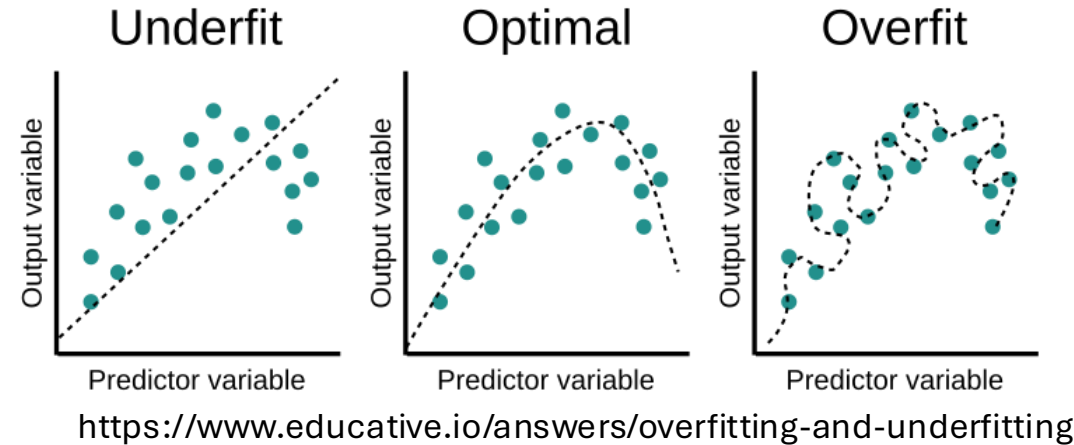


Reguralisation

Parameter Norm Penalties

$$\bar{J}(w; X, y) = J(w; X, y) + \alpha\Omega(w)$$

- L^1 parameter regularization
- $\Omega = \sum_i |w_i|$
- Not everywhere differentiable
- Favours small weights values
- Sets some weights to zero

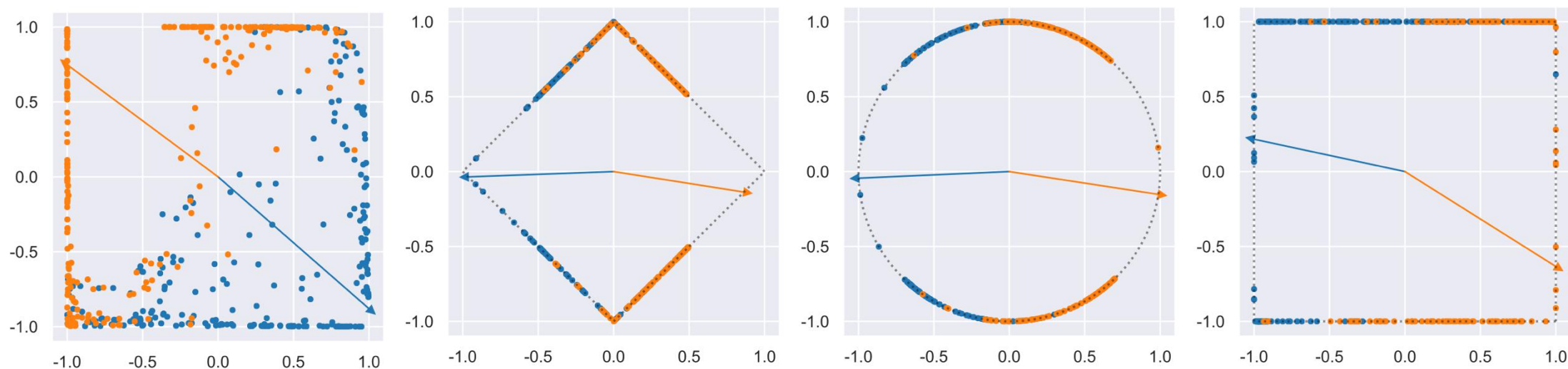


<https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/>

Reguralisation

Norm constrained softmax (classification problems)

- Restrict the output of the penultimate layer to lie on a manifold and its magnitude to be equal to a



No normalization $\|\cdot\|_1$ normalization $\|\cdot\|_2$ normalization $\|\cdot\|_\infty$ normalization

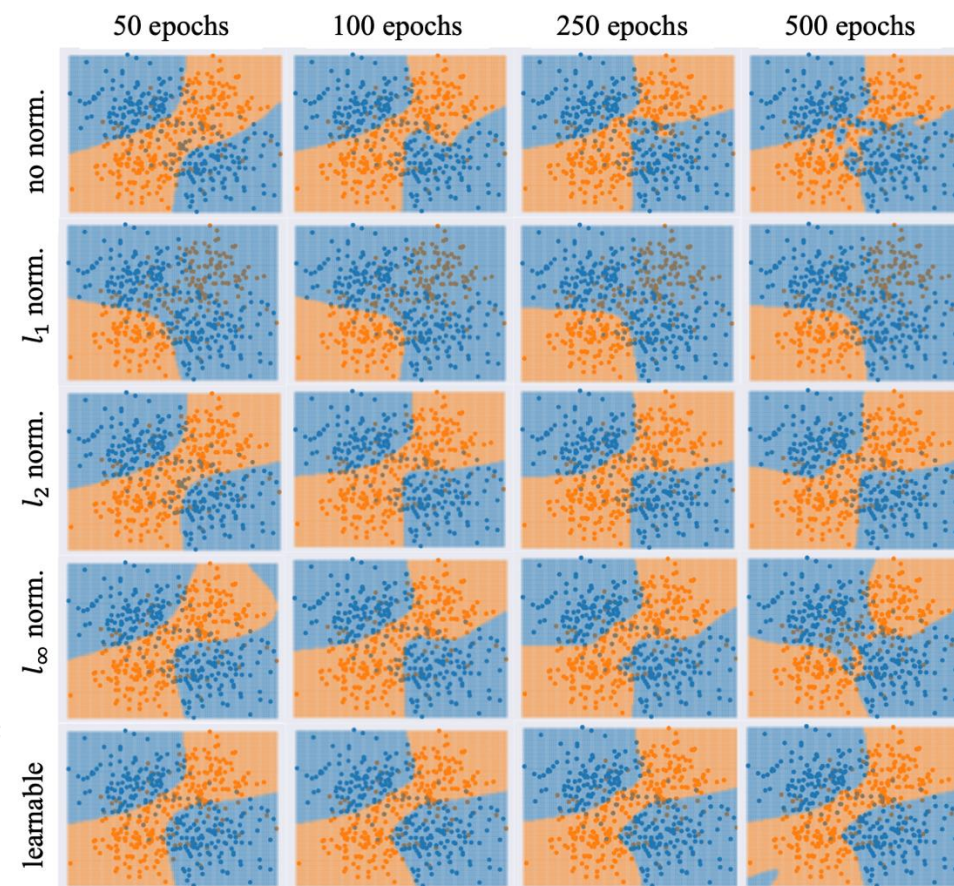
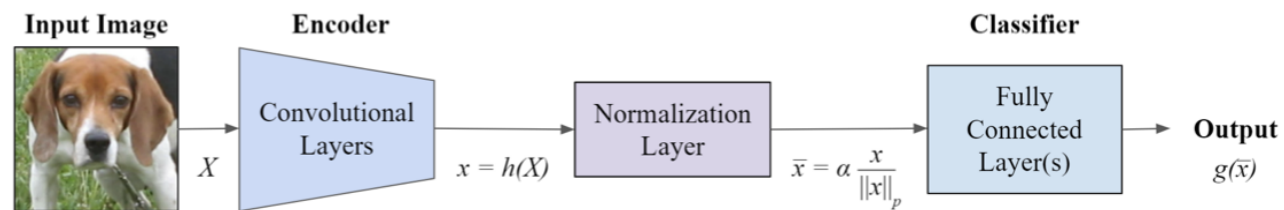
Trivedi, Chintan, Konstantinos Makantasis, Antonios Liapis, and Georgios N. Yannakakis. "Revisiting lp-constrained Softmax Loss: A Comprehensive Study." arXiv preprint arXiv:2206.09616 (2022).

Reguralisation

Norm constrained softmax

$$\bar{h}(X) = \alpha \frac{h(X)}{\|h(X)\|}$$

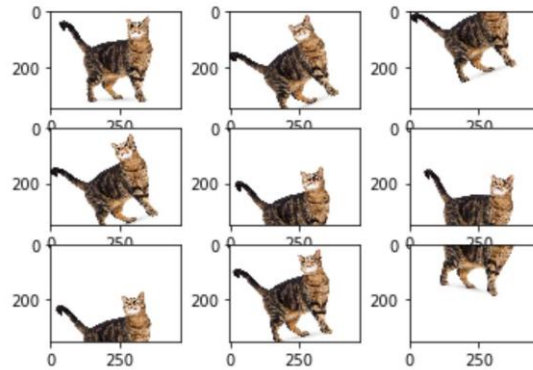
- $h(X)$ is the output of the penultimate layer of a network.



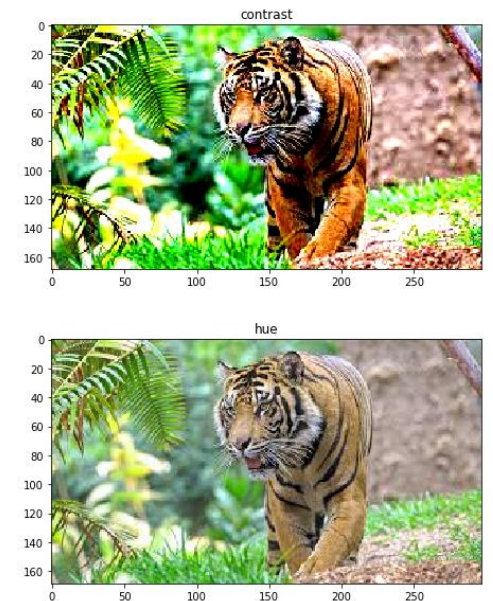
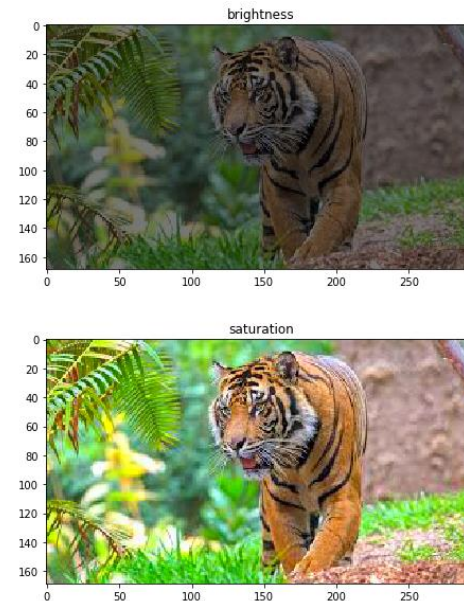
Reguralisation

Dataset augmentation

- Generate new data via deterministic transformations
- Inject noise



<https://analyticsindiamag.com/image-data-augmentation-impacts-performance-of-image-classification-with-codes/>



<https://www.v7labs.com/blog/data-augmentation-guide>

Reguralisation

Early stopping

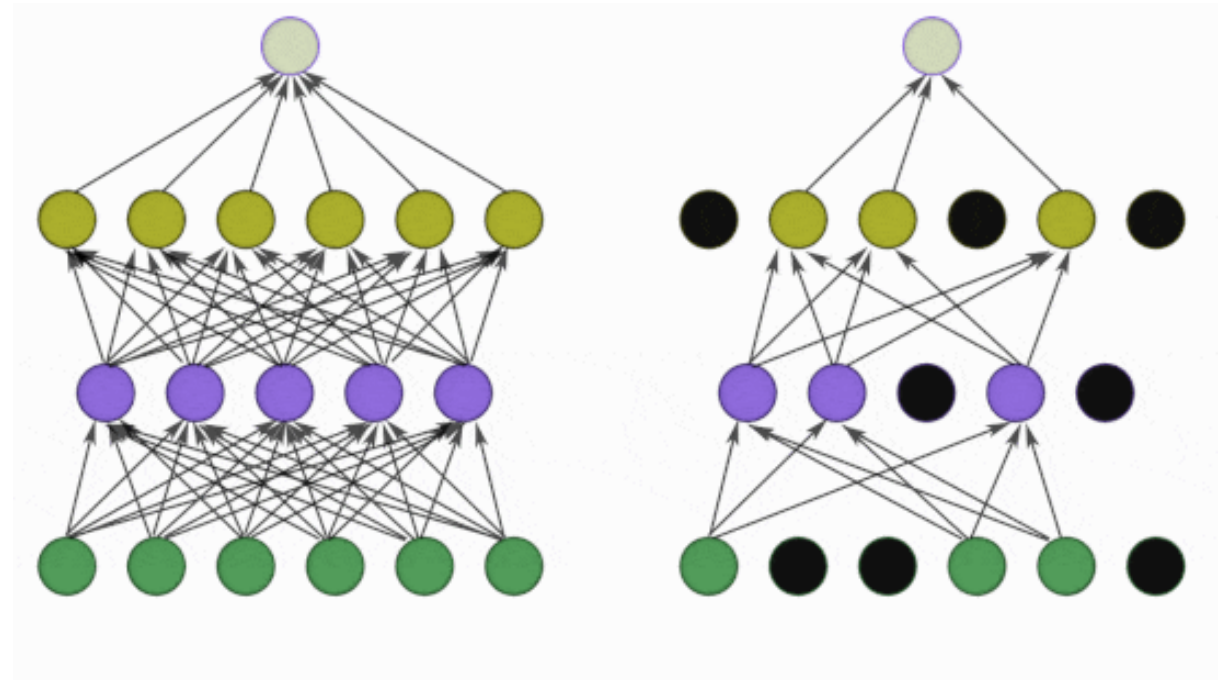
- Stop training when validation loss does not improve
- Sacrifice a subset of the data for validation purposes



Reguralisation

Dropout

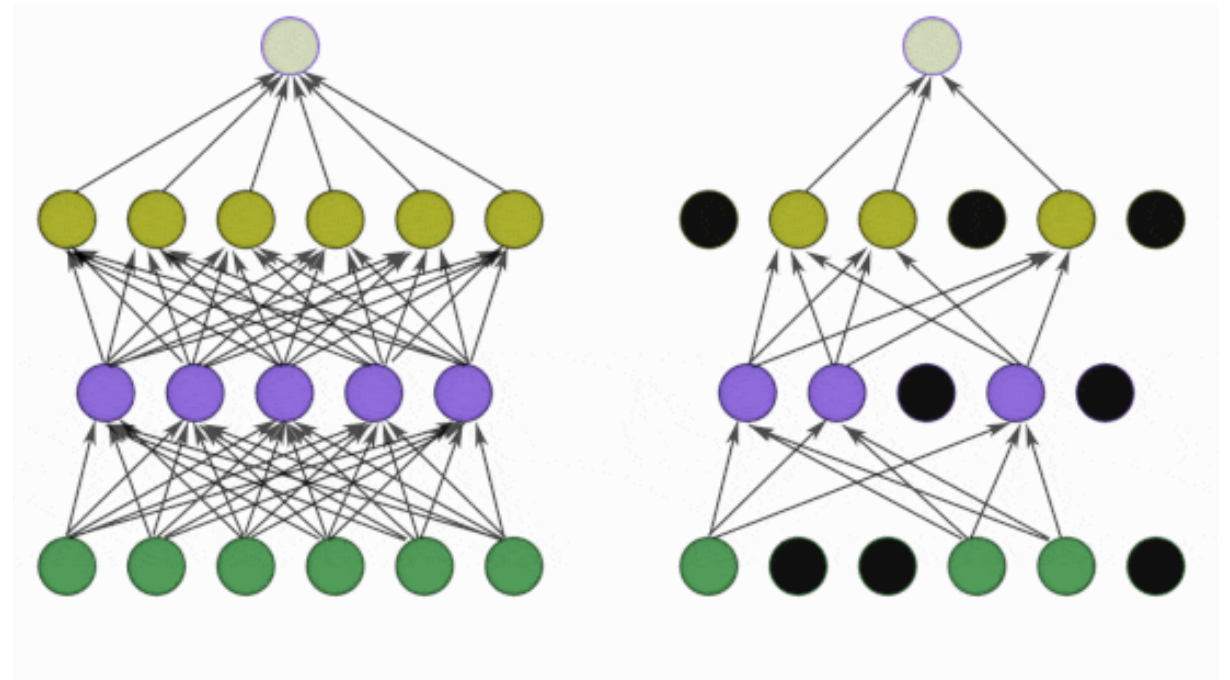
- Inspired by the the brain's plasticity.
- Prevents complex co-adaptation on training data.
- Learn robust features
- Useful with many different random subsets of the other neurons.



Reguralisation

How does dropout work?

- For each training case, a percentage of hidden neurons are randomly selected and set to zero and this leaves different architectures for each case.
- Dropout roughly doubles the number of iterations required to converge. However, training time for each epoch is less.



Training Deep Neural Networks

Deep neural networks are **data hungry**...

- Millions of trainable parameters
- Huge datasets

Training Deep Neural Networks

Deep neural networks are **data hungry**...

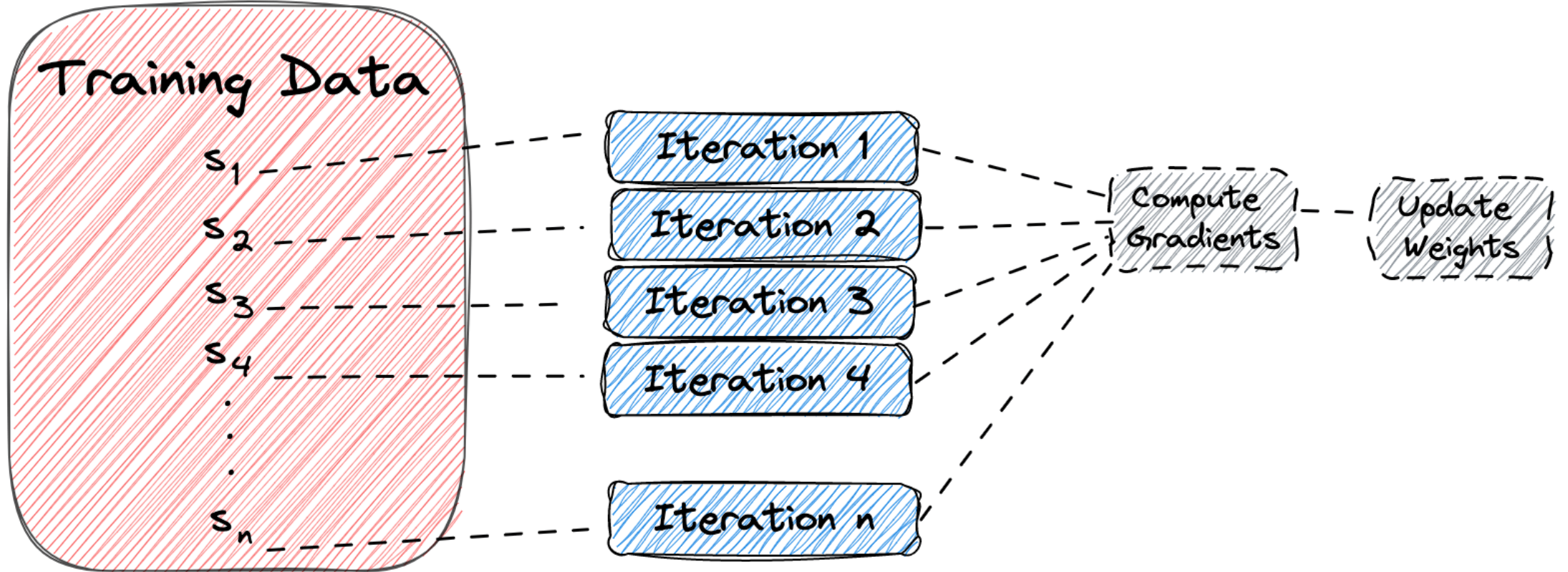
- Millions of trainable parameters
- Huge datasets

How can we handle datasets with millions of examples?

- We cannot load the dataset into the memory of a computer

Training Deep Neural Networks

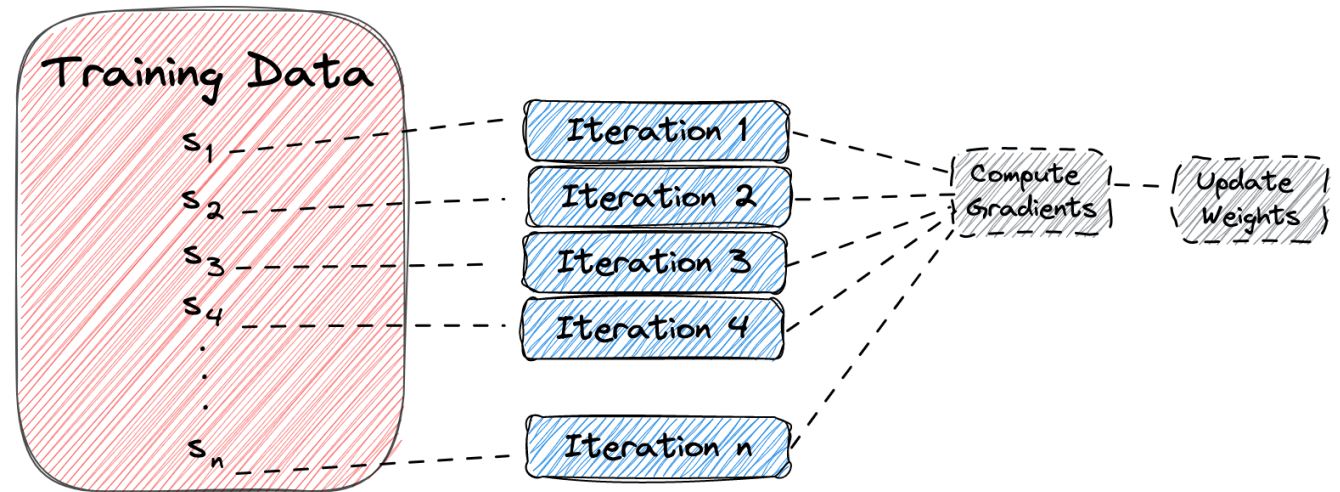
Minibatch training



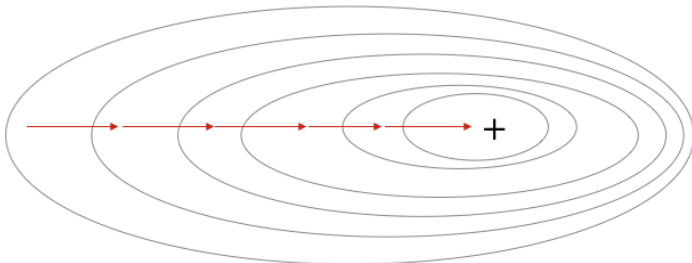
Training Deep Neural Networks

Minibatch training

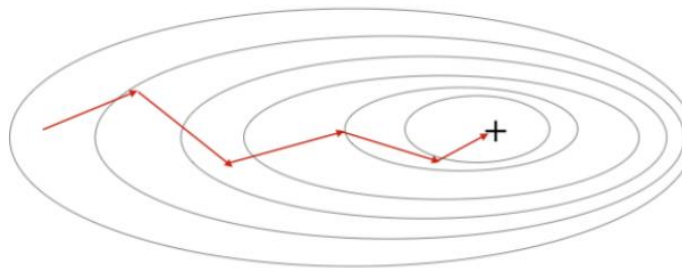
- “Avoid” local minima
- Memory efficient
- Parallelization



Gradient Descent



Mini-Batch Gradient Descent



<https://medium.datadriveninvestor.com/batch-vs-mini-batch-vs-stochastic-gradient-descent-with-code-examples-cd8232174e14>

Training Deep Neural Networks

Challenges in training deep neural networks

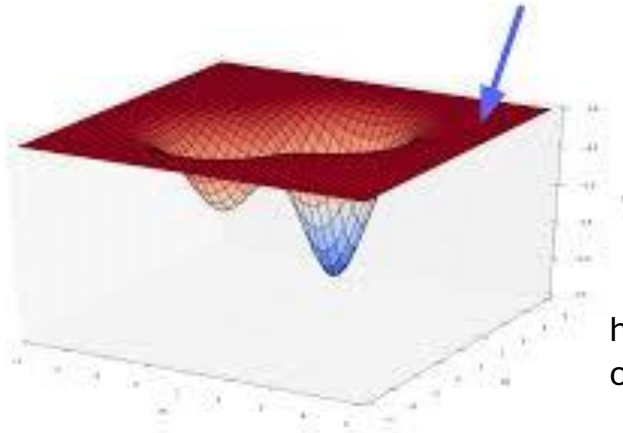
- **Local Minima**
 - Non-convex, highly nonlinear functions
 - Affect the quality of the solution

Practically it is not a problem if the cost function that corresponds to it is very low.

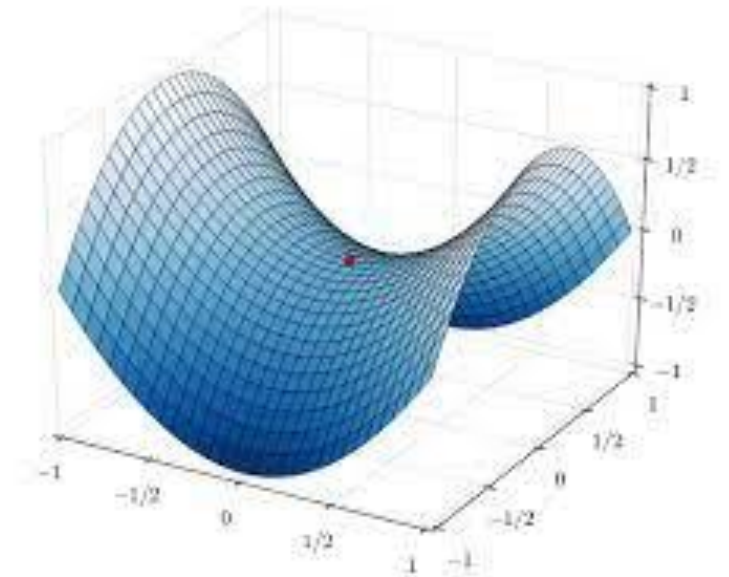
Training Deep Neural Networks

Challenges in training deep neural networks

- **Flat regions and saddle points**
 - Converge to no local minima points
 - Affect the quality of the solution
 - Optimization algorithm parameterization
 - E.g., momentum, learning rate



https://www.cs.toronto.edu/~rgrosse/courses/csc321_2017/slides/lec7.pdf



https://en.wikipedia.org/wiki/Saddle_point

Training Deep Neural Networks

Challenges in training deep neural networks

- **Parameter initialization**
 - Defined by the user
 - Determines whether the algorithms converges at all
 - Affect the quality of the final solution (high vs low cost)
 - Strategies are simple and heuristic

Training Deep Neural Networks

Challenges in training deep neural networks

- **Parameter initialization guidelines**
 - Initial parameters need to “break symmetry”
 - Neighbouring neurons must have different initial values
 - Deterministic methods (cost, optimization) → same initialization - same updates
 - Reduces networks capacity to learn interesting patterns

Training Deep Neural Networks

Challenges in training deep neural networks

- **Parameter initialization guidelines**
 - Initial parameters with values drawn randomly from a **Gaussian** distribution
 - Larger values yield a stronger symmetry breaking effect
 - Too large values result in exploding values during forward or back propagation

Training Deep Neural Networks

Challenges in training deep neural networks

- **Parameter initialization guidelines**
 - **Glorot** initialization
 - Consider a layer with m inputs and n outputs

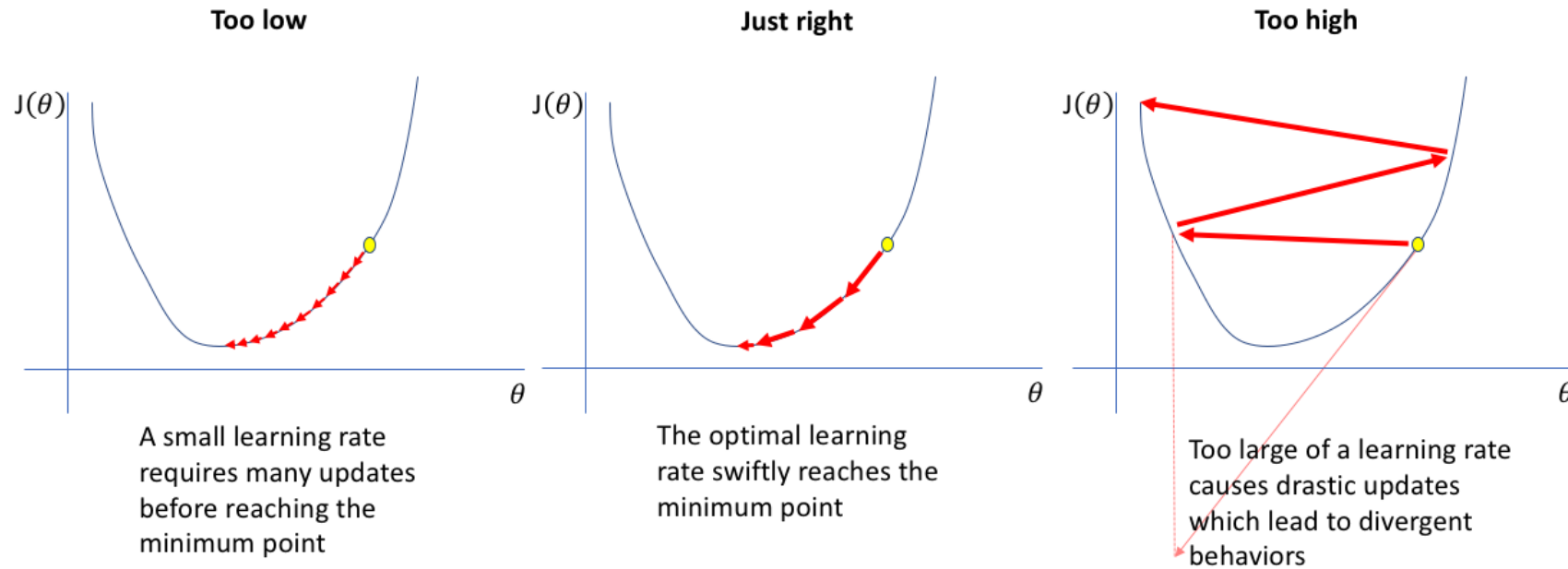
$$W \sim U\left(-\frac{\sqrt{6}}{\sqrt{m+n}}, \frac{\sqrt{6}}{\sqrt{m+n}}\right)$$

- All layers have the same activation variance

Training Deep Neural Networks

Challenges in training deep neural networks

- Learning rate of the gradient-based optimization algorithm



Training Deep Neural Networks

Challenges in training deep neural networks

- **Learning rate of the gradient-based optimization algorithm**
 - Adaptive learning rate
 - Reduce learning rate as the training progresses
 - Algorithms with adaptive learning rates
 - AdaGrad
 - RMSProp
 - Etc.

Training Deep Neural Networks

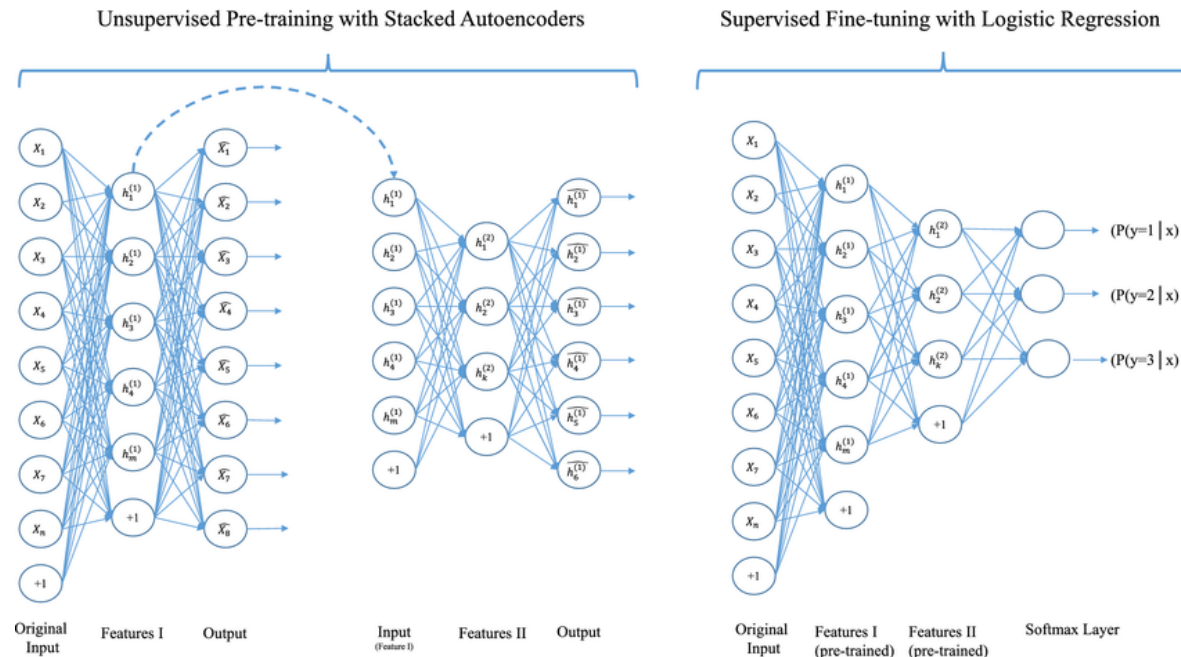
Challenges in training deep neural networks

- **Meta-learning - Batch normalization**
 - Increase the stability and performance of neural network training
 - This normalizes the output from a layer with zero mean and a standard deviation of 1.
 - This reduces overfitting and makes the network train faster. It is very useful in training complex neural networks.

Training Deep Neural Networks

Challenges in training deep neural networks

- **Meta-learning - Unsupervised pretraining**
 - Restricts the search space to “something” meaningful



https://www.researchgate.net/figure/Stacked-autoencoders-based-deep-learning-Unsupervised-pre-training-on-the-left_fig2_326501953

Contents

PART 1

- The Learning Framework
- Neural Networks History
- Feedforward Neural Networks
- Activation Functions
- Regularization Techniques
- Training Deep Neural Networks

PART 2

- Loss functions
- Performance metrics
- Popular Architectures
- Practical methodology

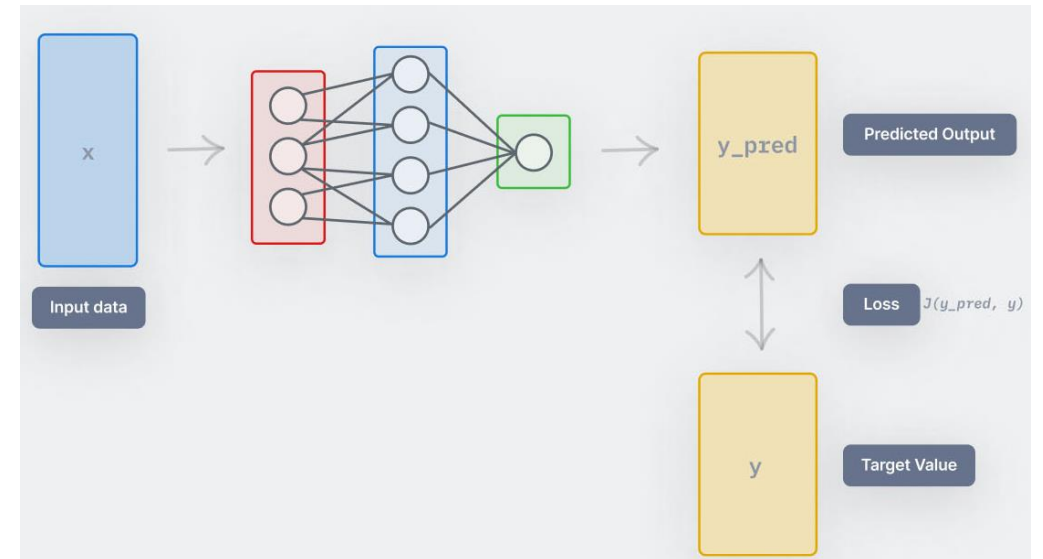
Loss Functions

Loss functions measure **how close** a predicted value is to the actual value.

- Small deviations -> small loss
- Large deviations -> large loss

Mathematical objects that **guide the model training** process towards correct predictions.

- Model parameters that **minimize** the loss of the model on a given training dataset.

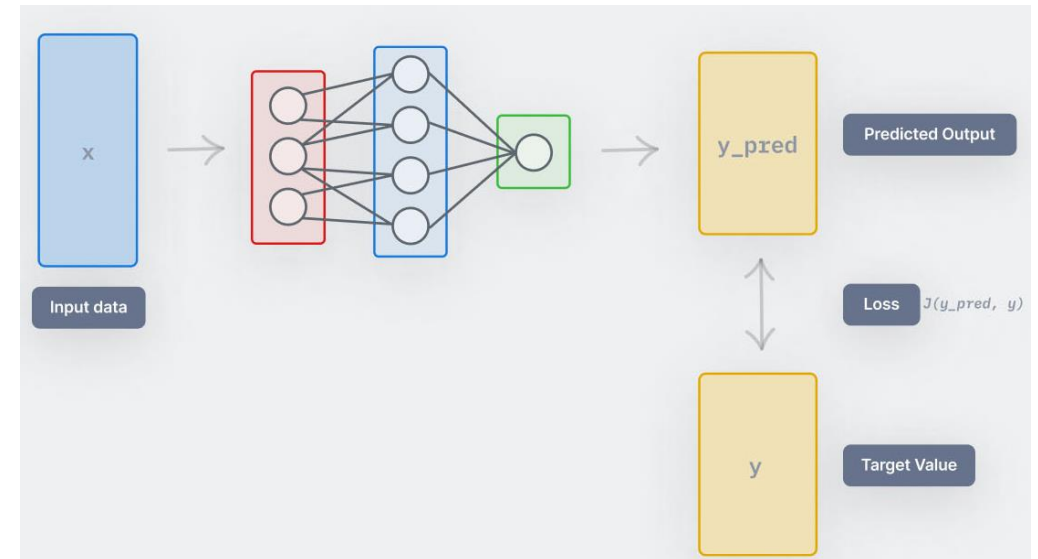


<https://www.v7labs.com/blog/pytorch-loss-functions>

Loss Functions

The form of the loss function depends on the problem at hand :

- **Regression**
 - L1 Loss (Mean Absolute Error)
 - Smooth L1 Loss
 - Mean Squared Error
- **Classification**
 - Cross Entropy Loss
 - KL Divergence Loss
- **Ranking**
 - Margin Ranking Loss
- **Representation learning**
 - Cosine Embedding Loss
 - Triplet Margin Loss



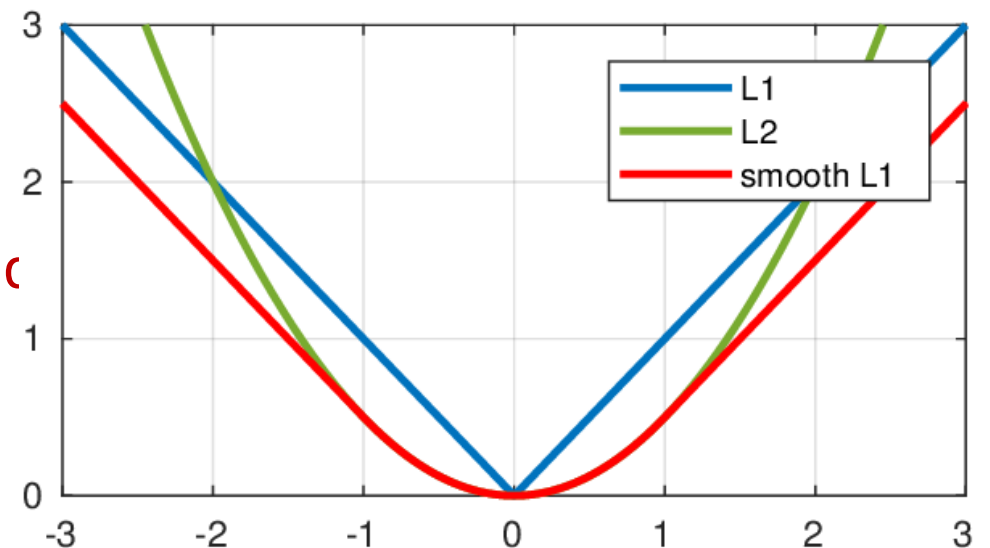
<https://www.v7labs.com/blog/pytorch-loss-functions>

Loss Functions - Regression

L2 Loss (Mean Squared Error) gives the mean squared error between predictions $\hat{y}$ and target y (desired output).

$$l(x, y) = \frac{1}{N} \sum_{n=1}^N (\hat{y}_n - y_n)^2$$

- The most commonly used loss
 - Closed form solutions
- Very sensitive to outliers
 - The error for outlier points is squared
 - Overfit the outliers

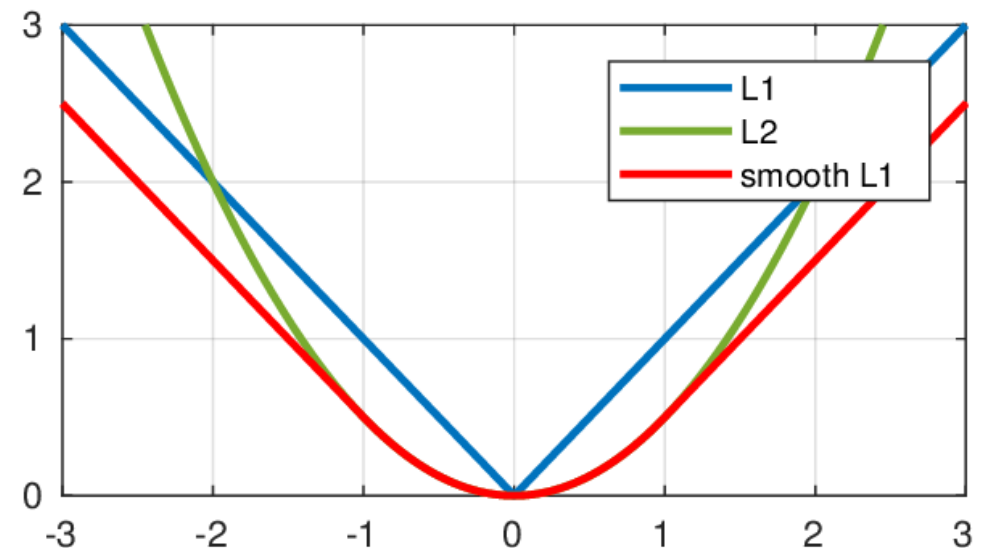


Loss Functions - Regression

L1 Loss (Mean Absolute Error) measures the mean absolute error between predictions $\hat{y}$ and target y (desired output).

$$l(x, y) = \frac{1}{N} \sum_{n=1}^N |\hat{y}_n - y_n|$$

- Robust against outliers and noise
- Not differentiable at 0
 - This complicates learning



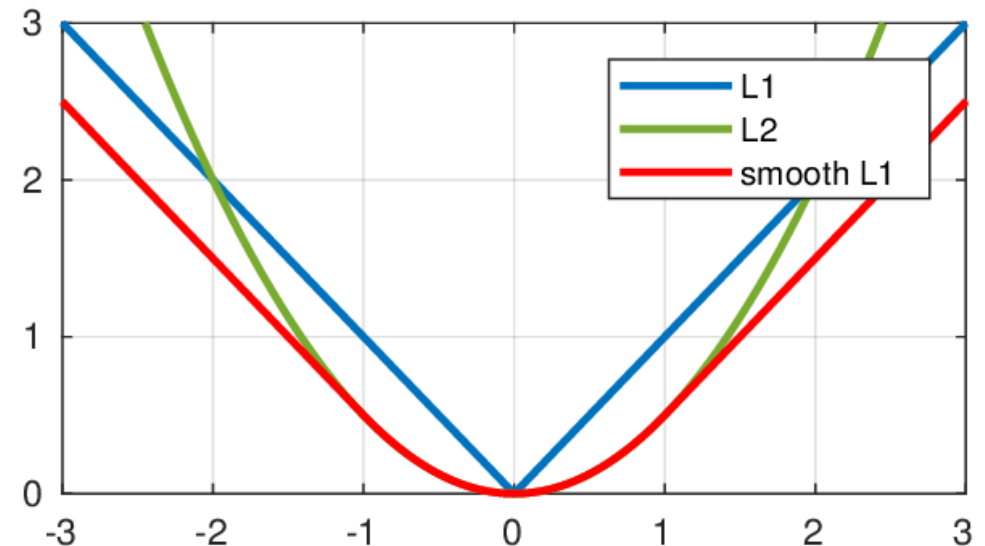
Loss Functions - Regression

Smooth L1 Loss uses L2 loss when the absolute element wise error falls below 1 and L1 loss otherwise.

$$l(x, y) = \frac{1}{N} \sum_{n=1}^N z_n$$

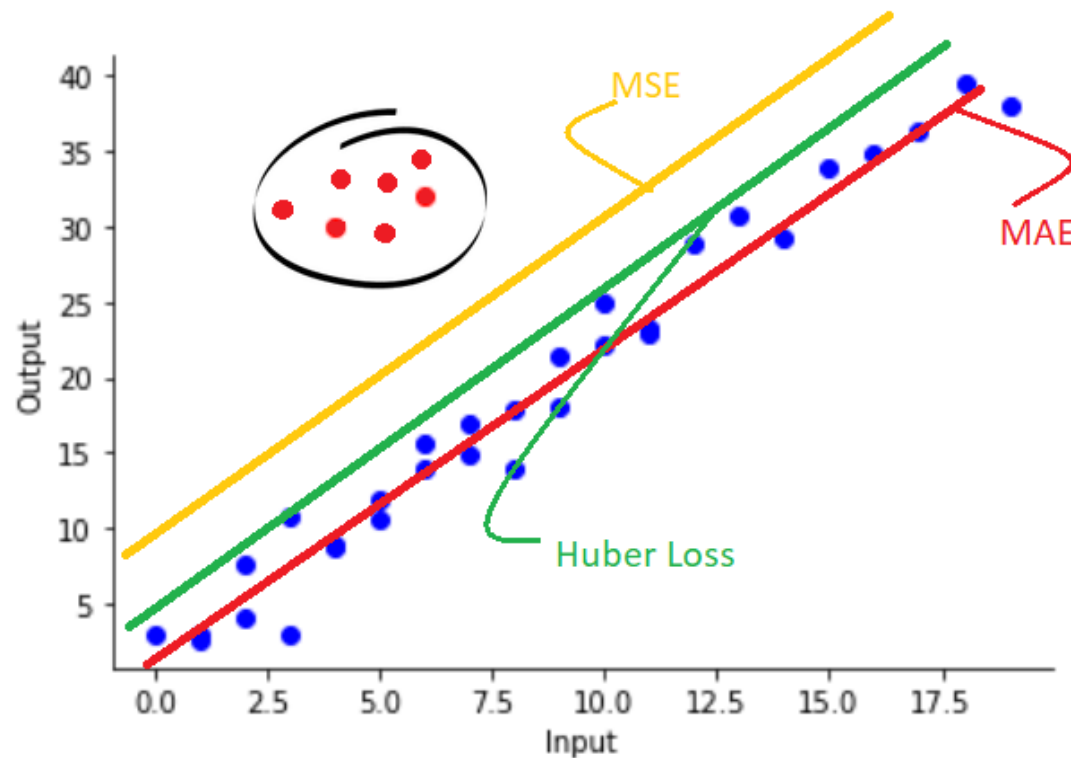
$$z_n = \begin{cases} 0.5(x_n - y_n)^2, & \text{if } |x_n - y_n| < 1 \\ |x_n - y_n| - 0.5, & \text{otherwise} \end{cases}$$

- Combines the advantages of both L1 and L2



Loss Functions - Regression

Smooth L1 Loss versus L1 Loss versus L2 Loss in the presence of outliers.



Loss Functions - Classification

Cross-Entropy Loss is used to learn the probability distribution of the data with respect to the classes they belong.

- Penalizes greatly for being confident and wrong

$$l(x_n, y_n) = - \sum_c w_c \cdot \log \left(\frac{\exp(\hat{y}_n^{(c)})}{\sum_j \exp(\hat{y}_n^{(j)})} \right) \cdot y_n^{(c)}$$

Loss Functions - Classification

Cross-Entropy Loss is used to learn the probability distribution of the data with respect to the classes they belong.

- Penalizes greatly for being confident and wrong

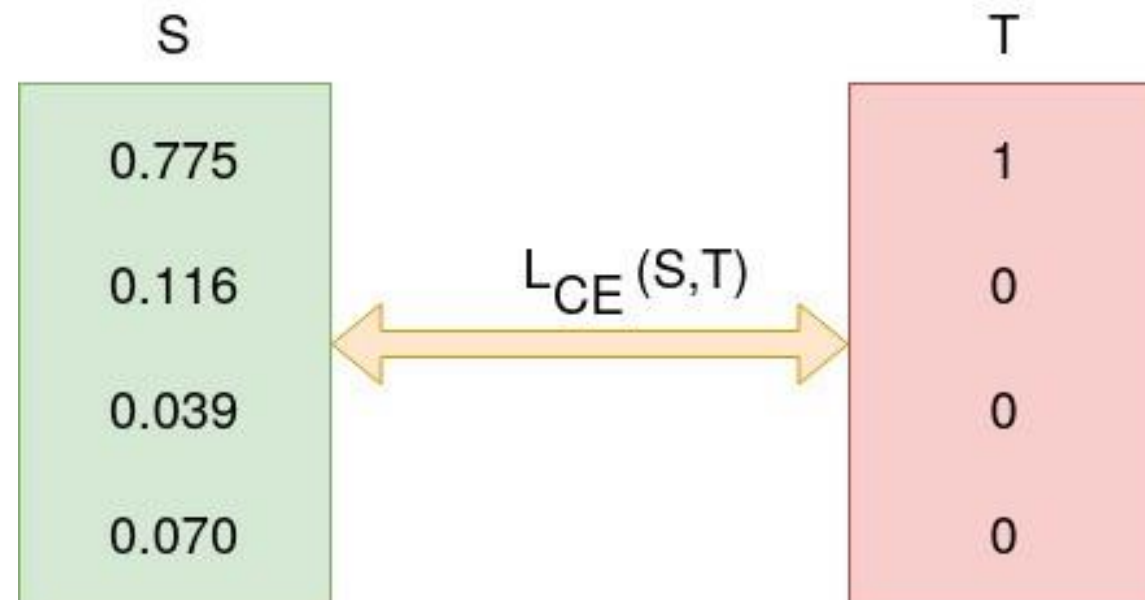
$$l(x_n, y_n) = - \sum_c \underbrace{w_c}_{\downarrow} \cdot \log \left(\frac{\exp(\hat{y}_n^{(c)})}{\sum_j \exp(\hat{y}_n^{(j)})} \right) \cdot y_n^{(c)}$$

The weight vector is useful if we deal with imbalanced classification problems

Loss Functions - Classification

Cross-Entropy Loss is used to learn the probability distribution of the data with respect to the classes they belong.

- Penalizes greatly for being confident and wrong



Loss Functions - Classification

Binary Cross-Entropy Loss is similar to cross-entropy loss when the data belong to **two** different classes.

$$l(\hat{y}_n, y_n) = -(y_n \log \hat{y}_n + (1 - y_n) \log(1 - \hat{y}_n))$$

Loss Functions - Classification

Binary Cross-Entropy Loss is similar to cross-entropy loss when the data belong to **two** different classes.

$$l(\hat{y}_n, y_n) = -\underbrace{(y_n \log \hat{y}_n)}_{\text{Cancels out when the desired output is 0}} + \underbrace{(1 - y_n) \log(1 - \hat{y}_n)}_{\text{Cancels out when the desired output is 1}}$$

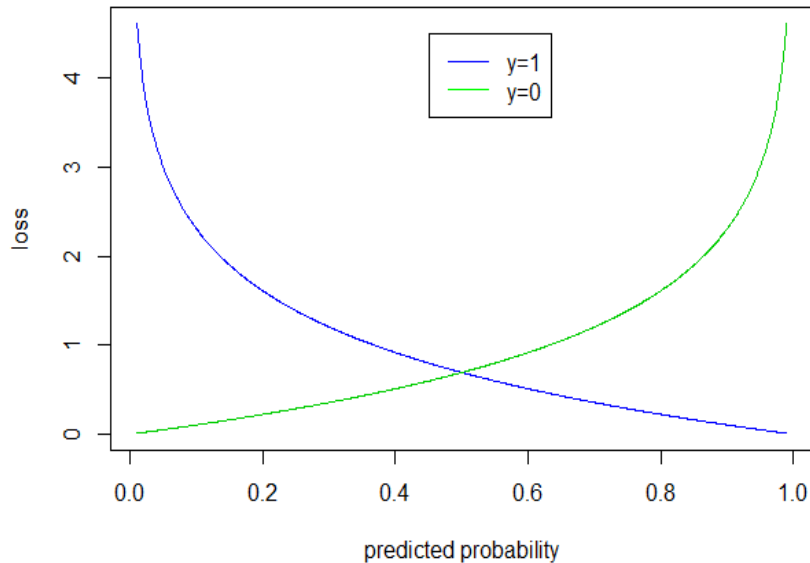
Cancels out when
the desired output
is 0

Cancels out when
the desired output
is 1

Loss Functions - Classification

Binary Cross-Entropy Loss is similar to cross-entropy loss when the data belong to **two** different classes.

$$l(\hat{y}_n, y_n) = -\underbrace{(y_n \log \hat{y}_n)}_{\text{Cancels out when the desired output is 0}} + \underbrace{(1 - y_n) \log(1 - \hat{y}_n)}_{\text{Cancels out when the desired output is 1}}$$



Cancels out when
the desired output
is 0

Cancels out when
the desired output
is 1

This formula assumes that $\hat{y}_n$ and y_n are probabilities, so they are strictly between 0 and 1.

Loss Functions - Classification

Kullback-Leibler (KL) Divergence measures how two probability distributions are different from each other.

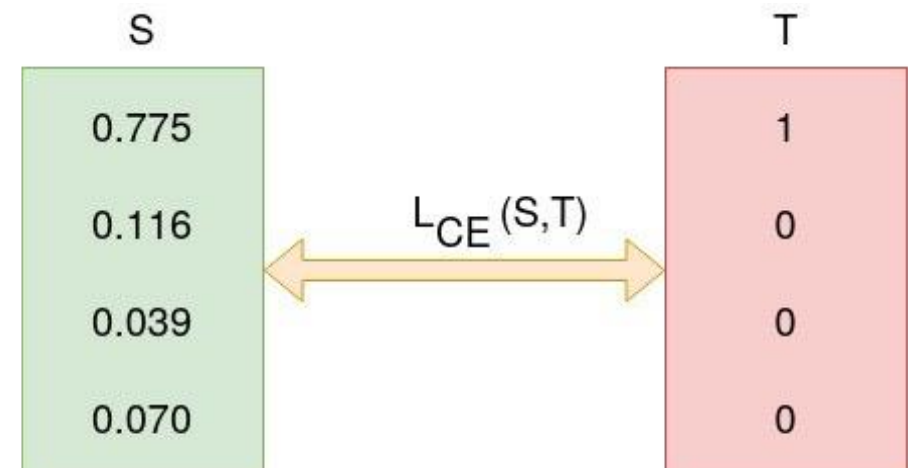
$$l(\hat{y}_n, y_n) = y_n \log \frac{y_n}{\hat{y}_n} = y_n (\log y_n - \log \hat{y}_n)$$

Loss Functions - Classification

Kullback-Leibler (KL) Divergence measures how two probability distributions are different from each other.

$$l(\hat{y}_n, y_n) = y_n \log \frac{y_n}{\hat{y}_n} = y_n (\log y_n - \log \hat{y}_n)$$

Expectation of the
logarithmic difference
between the two
distributions



Loss Functions - Classification

Kullback-Leibler (KL) Divergence measures how two probability distributions are different from each other.

$$l(\hat{y}_n, y_n) = y_n \log \frac{y_n}{\hat{y}_n} = y_n (\log y_n - \log \hat{y}_n)$$

- Non-negative.
- The same can be achieved with cross entropy with lesser computation.

Loss Functions - Ranking

Margin Ranking Loss measures the relative distance between pairs of inputs belonging to a dataset.

$$l(\hat{y}_{n,1}, \hat{y}_{n,2}, y_n) = \max\left(0, \Delta - y_n \cdot (\hat{y}_{n,1} - \hat{y}_{n,2})\right)$$

- $y_n \in \{-1, 1\}$: 1 if $\hat{y}_{n,1} > \hat{y}_{n,2}$ and -1 otherwise
- If $y_n \cdot (\hat{y}_{n,1} - \hat{y}_{n,2})$ is larger than a margin Δ the loss is zero.
- If $y_n \cdot (\hat{y}_{n,1} - \hat{y}_{n,2})$ is smaller than a margin Δ the loss increases linearly.

It is widely used for **preference learning** and **ranking** tasks and tasks that include **subjectively defined** variables.

Loss Functions – Representation Learning

Triplet Margin Loss measures the relative similarity between samples.

$$l_n(a_n, p_n, n_n) = \max(0, \Delta + d(a_n, p_n) - d(a_n, n_n))$$

- a_n is anchor sample, p_n is a sample that is similar to the anchor and n_n is a sample that is dissimilar to the anchor.
- Sends the first distance toward 0 and the second distance larger than some margin.
- The only thing that matters is that the distance between the good pair is smaller than the distance between the bad pair.

Loss Functions – Representation Learning

Cosine Embedding Loss measures whether two inputs are similar or dissimilar, using the cosine distance.

$$l_n(x_{n,1}, x_{n,2}) = \begin{cases} 1 - \cos(x_{n,1}, x_{n,2}) & \text{if } x_{n,1} \text{ and } x_{n,2} \text{ are similar} \\ \max(0, \cos(x_{n,1}, x_{n,2}) - \Delta) & \text{otherwise} \end{cases}$$

- Normalised Euclidean distance.
- Positive cases result to aligned vector representations.
- Negative cases result to orthogonal vector representations.
- Building block for **contrastive** learning.

Loss Functions – Representation Learning

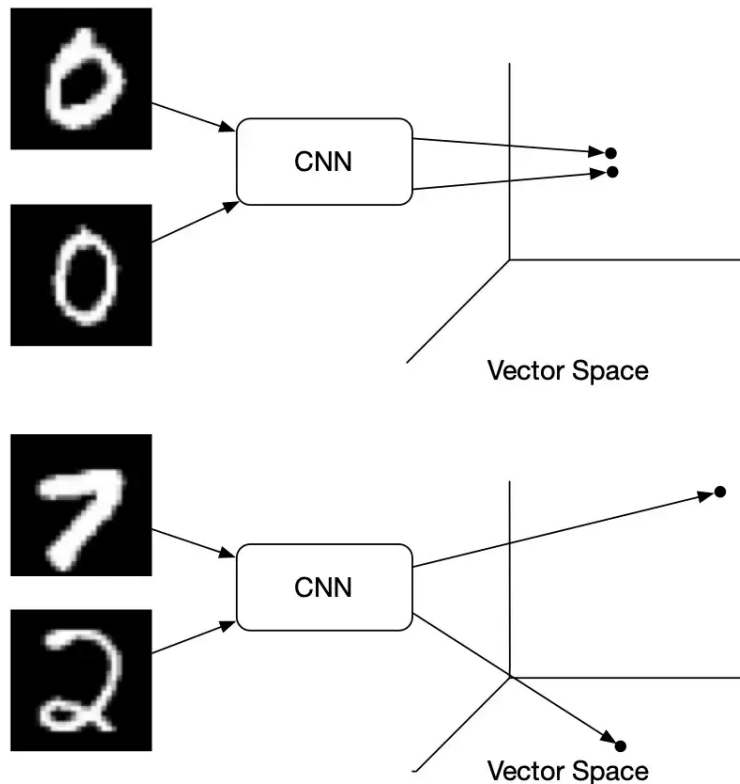
Contrastive Loss is used for (un)supervised representation learning.

$$l_n(x_n, p, \{n_k\}_{k=1}^K) = -\log \frac{\exp\left(\frac{\text{sim}(x_n, p)}{\tau}\right)}{\sum_{k=1}^K \exp\left(\frac{\text{sim}(x_n, n_k)}{\tau}\right)}$$

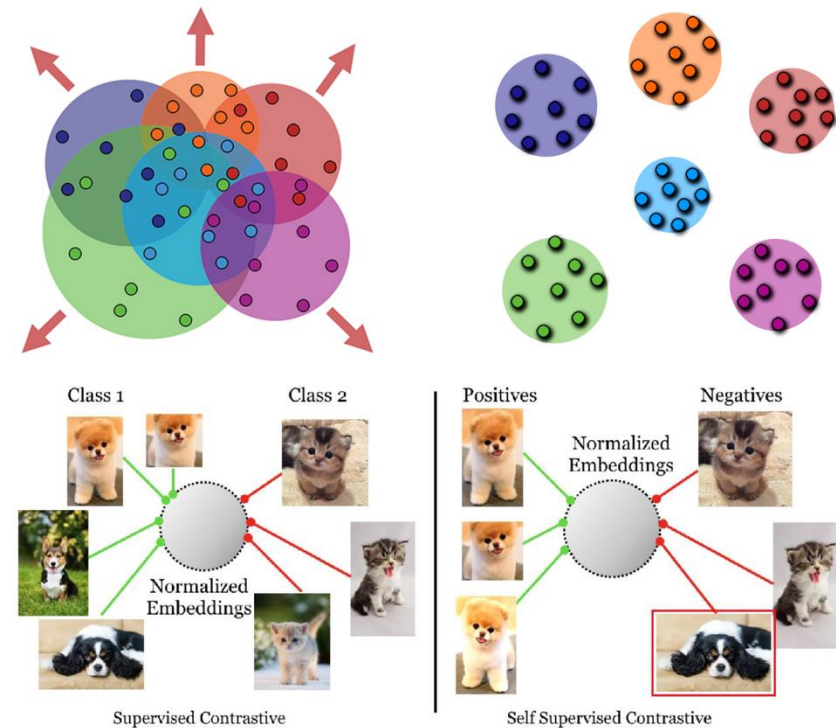
- Contrasts an anchor point against a positive (similar) example and again K negative examples.
- **Unsupervised**: the positive example corresponds to a transformation of x_n and negative examples are all the examples in the dataset except x_n .
- **Supervised**: positive are examples that belong to the same class as x_n and negative are those that belong to different classes.

Loss Functions – Representation Learning

Contrastive Loss is used for (un)supervised representation learning.



<https://towardsdatascience.com/contrastive-loss-explained-159f2d4a87ec>

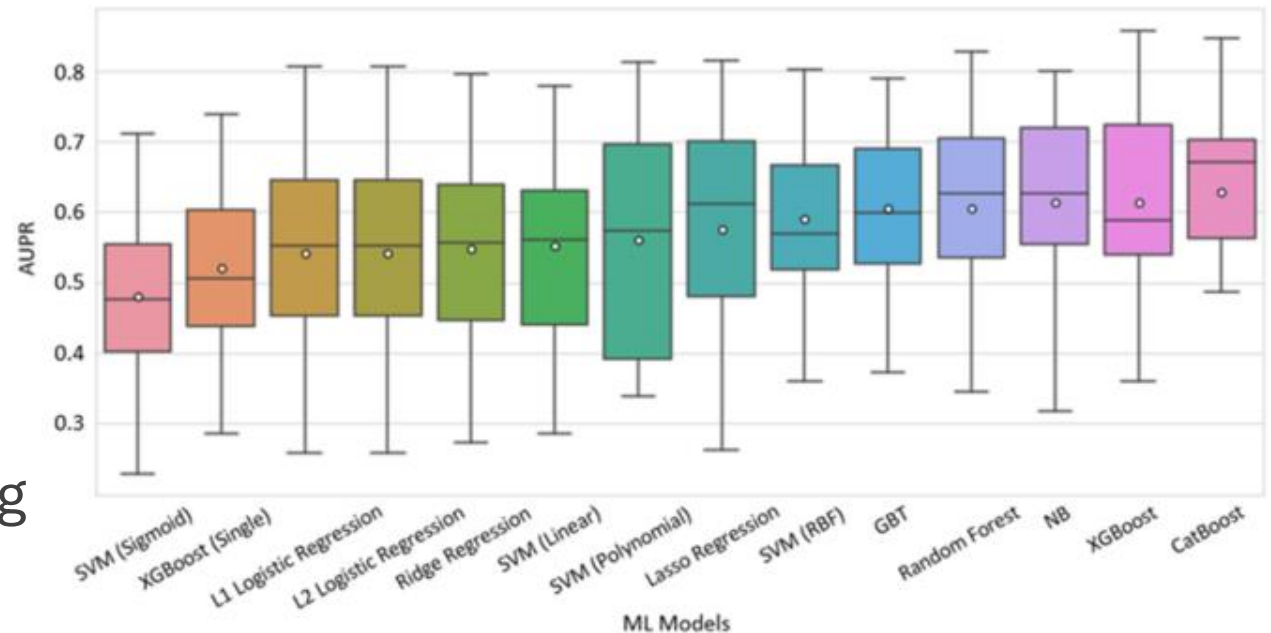


<https://gowrishankar.info/blog/introduction-to-contrastive-loss-similarity-metric-as-an-objective-function/>

Performance Metrics

Performance metrics tell you if your models are making progress and put a number on it.

- They are usually different from loss functions
- Loss functions are used for **training**, while performance metrics for **testing**
- Define our **goals**
- **Problem depended**
 - Regression metrics
 - Classification metrics
 - Ranking metrics
 - Representation learning metrics



Performance Metrics - Regression

Mean Squared Error (MSE) and **Mean Absolute Error (MAE)** are perhaps the most popular metric used for regression problems.

- **MSE** finds the **average of the squared difference** between the target value and the value predicted by the regression model.

$$MSE = \frac{1}{N} \sum_{n=1}^N (x_n - y_n)^2$$

- **MAE** finds the **average of the absolute difference** between the target value and the value predicted by the regression model.

$$MAE = \frac{1}{N} \sum_{n=1}^N |x_n - y_n|$$

Performance Metrics - Regression

Mean Squared Error (MSE) and **Mean Absolute Error (MAE)** are perhaps the most popular metric used for regression problems.

- **MSE** finds the **average of the squared difference** between the target value and the value predicted by the regression model.

$$MSE = \frac{1}{N} \sum_{n=1}^N (x_n - y_n)^2$$

→ MSE units correspond to the squared units of x_n

- **MAE** finds the **average of the absolute difference** between the target value and the value predicted by the regression model.

$$MAE = \frac{1}{N} \sum_{n=1}^N |x_n - y_n|$$

→ MAE units are the same as the units of x_n

Performance Metrics - Regression

R-squared (R^2) provides an indication of the **goodness of fit** of a set of predicted values and actual output values.

$$R^2 = 1 - \frac{\sum_n (y_n - x_n)^2}{\sum_n (y_n - \bar{y})^2}$$

- Compares the fit of the trained model with that of a **horizontal line**, which serves as **baseline**.
- If the model fit is **worse (better)** than that of the horizontal line, then R^2 is **negative (positive)**.
- R^2 is bounded between -1 and 1.

Performance Metrics - Classification

Confusion matrix presents a summary of the classification performance in terms of true/false positives and true/false negatives.

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Performance Metrics - Classification

Accuracy is the fraction of predictions our model got right out of all the predictions.

$$\text{accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

- It ranges between zero and one.
- Great for balanced datasets.
- Problematic for imbalanced datasets.
- Extreme case -> models that are not predictive

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Performance Metrics - Classification

Precision indicates the proportion of positive predictions that are actually correct.

$$\text{precision} = \frac{TP}{TP+FP}$$

- To have the whole picture, it is used along with recall.
- It overcomes the problem of accuracy when the data are imbalanced.

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Performance Metrics - Classification

Recall indicates the ratio of actual positives predicted as such against the total number of positive examples.

$$\text{recall} = \frac{TP}{TP+FN}$$

- To have the whole picture, it is used along with precision.
- It overcomes the problem of accuracy when the data are imbalanced.

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Performance Metrics - Classification

F1-score balances precision and recall. It corresponds to the harmonic mean of both precision and recall.

$$\text{F1 score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

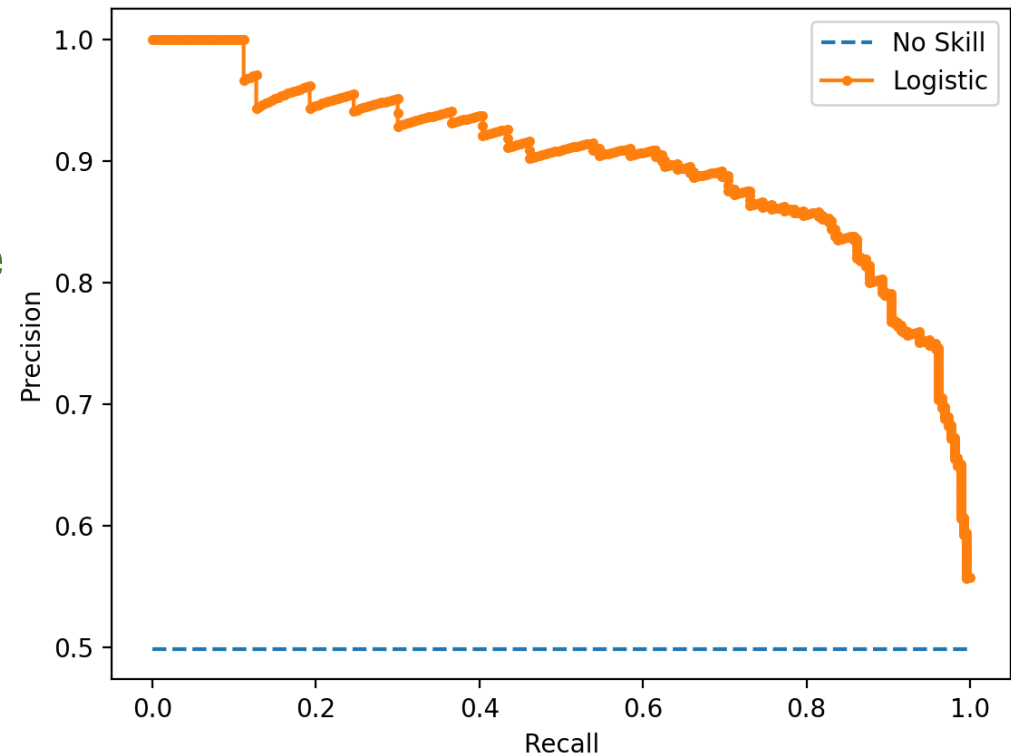
- It is used when both precision and recall are equally important.
- Overcomes the limitations of accuracy when imbalanced datasets are used.

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Performance Metrics - Classification

Precision/recall curve plots performance over a y-axis showing precision and an x-axis which is recall.

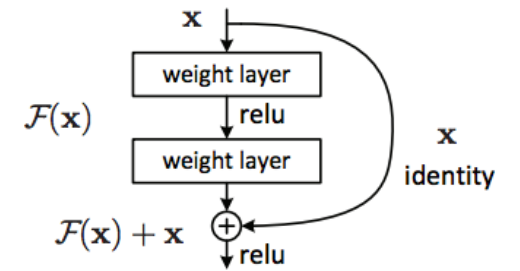
- Each point is evaluated at different threshold values for $P(\text{positive} \mid x)$.
- Great when the different classes are not equally important.



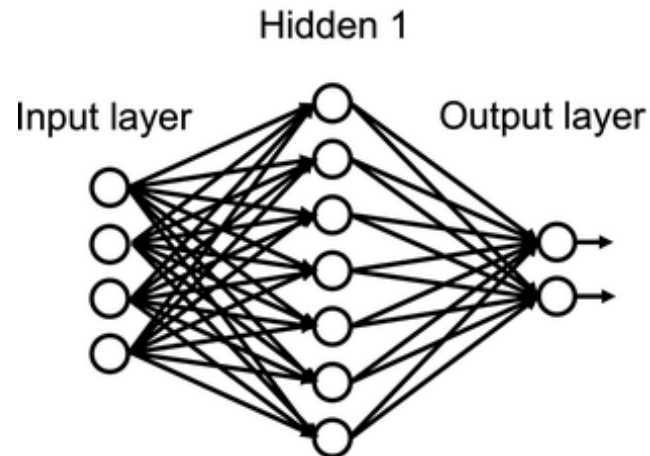
Popular Architectures

Deep Dense Neural Networks

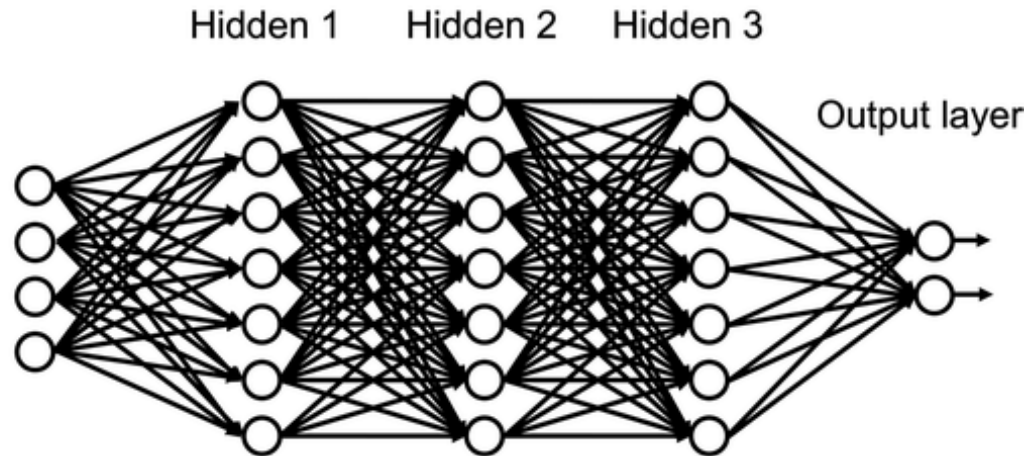
- Classification problems
- Regression problems
- Ranking problems
- Residual blocks to avoid the vanishing gradient problem



Feedforward Neural Network



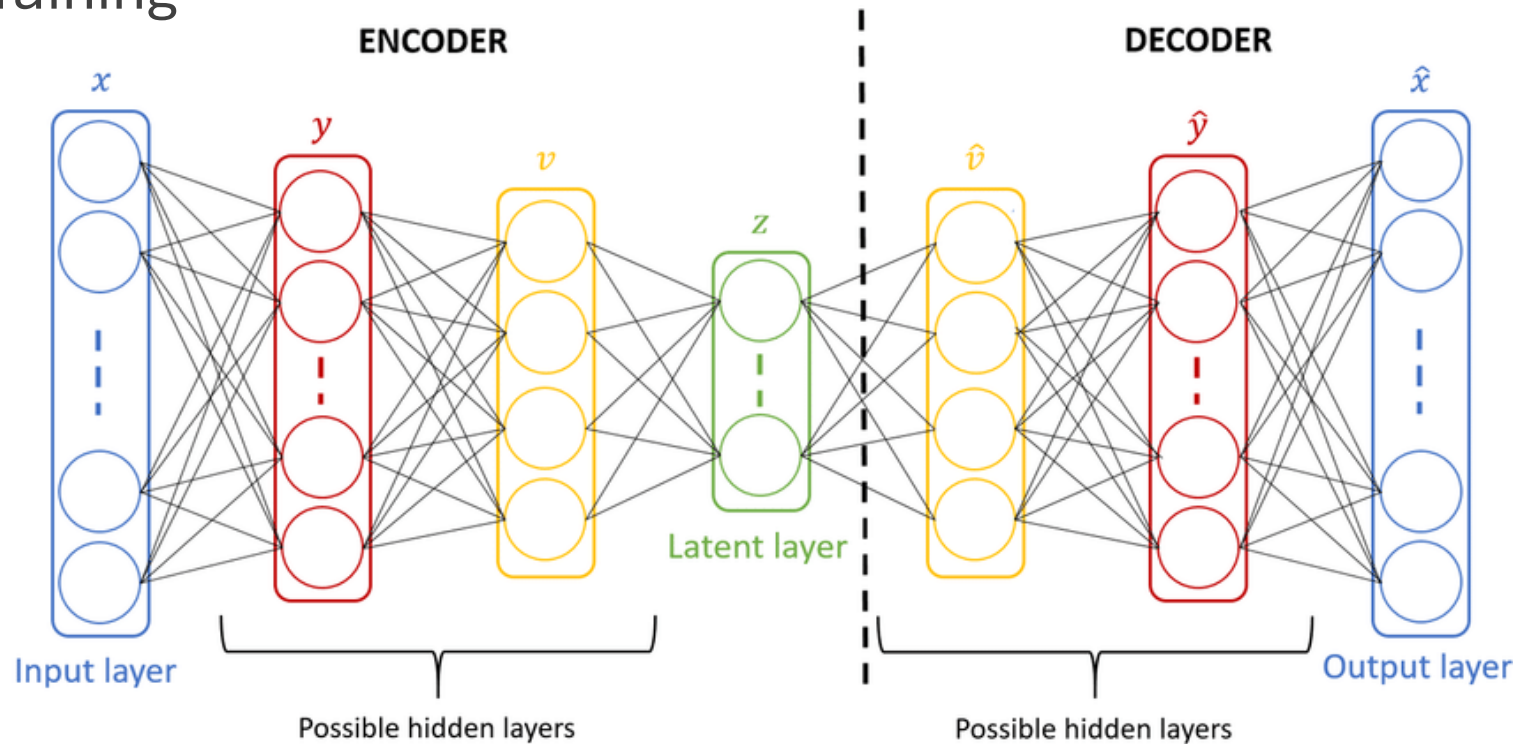
Deep Neural Network



Popular Architectures

Autoencoders

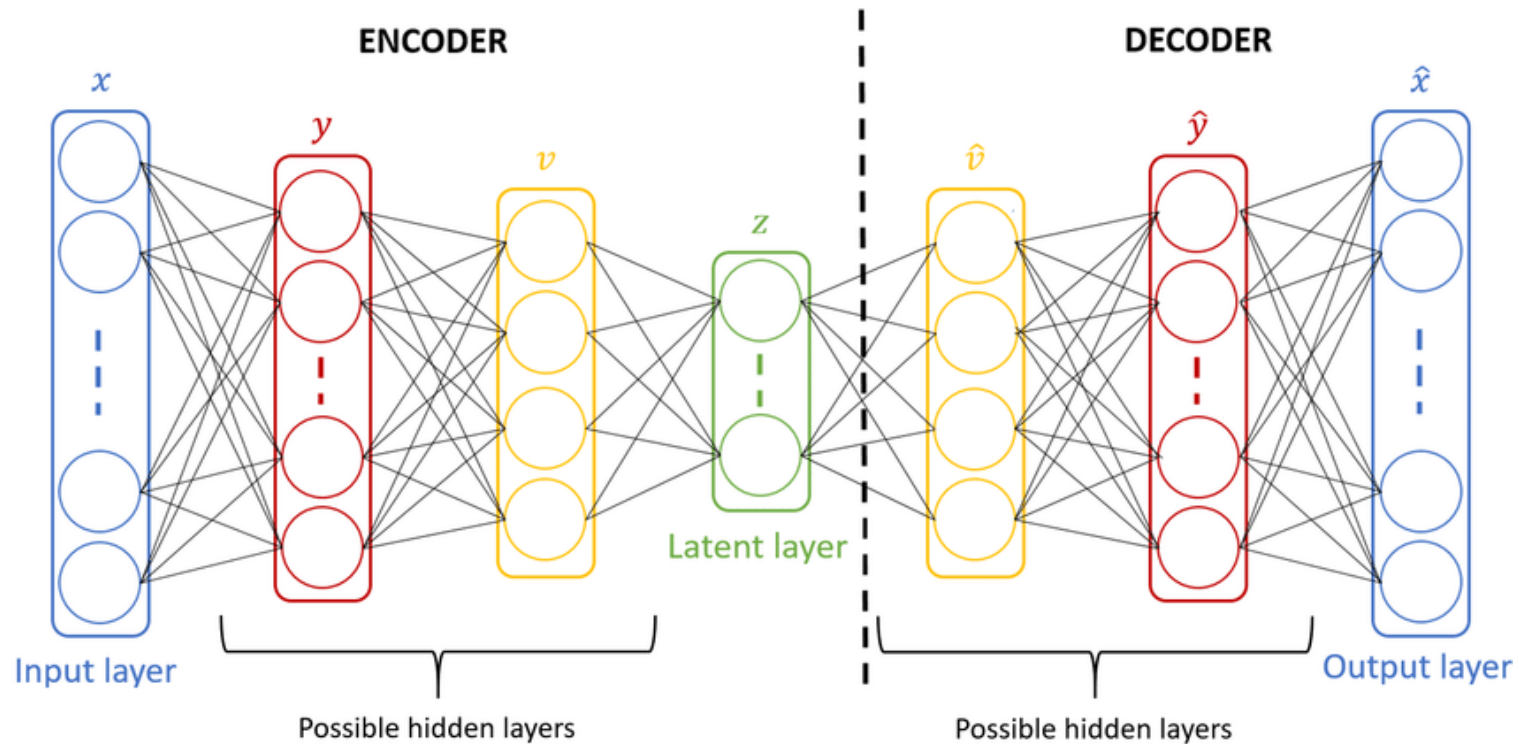
- Automatically learn high-level representations of the input
- Pretraining



Popular Architectures

Autoencoders: variations

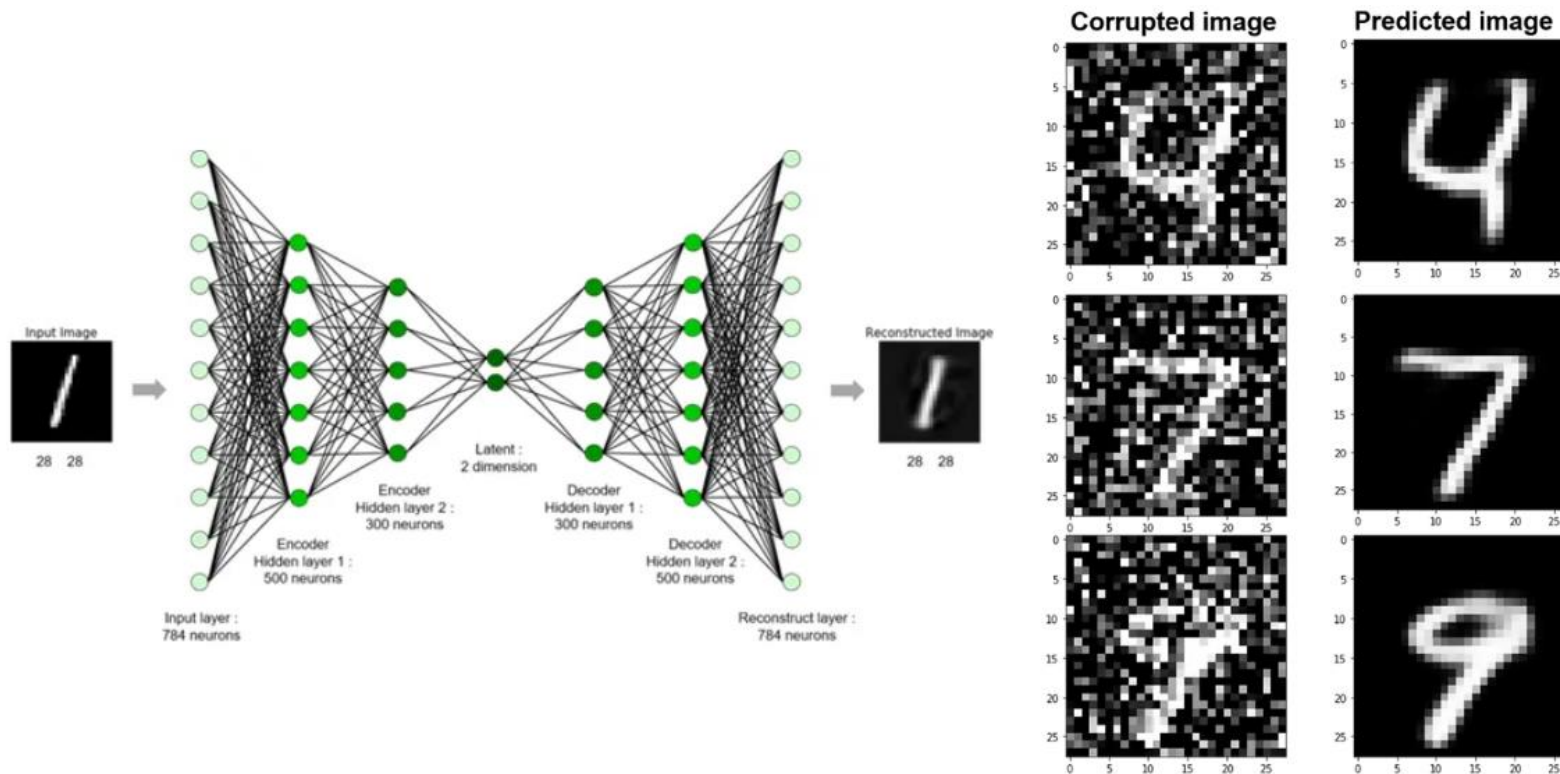
- Denoising autoencoder (overcomplete representations)



Popular Architectures

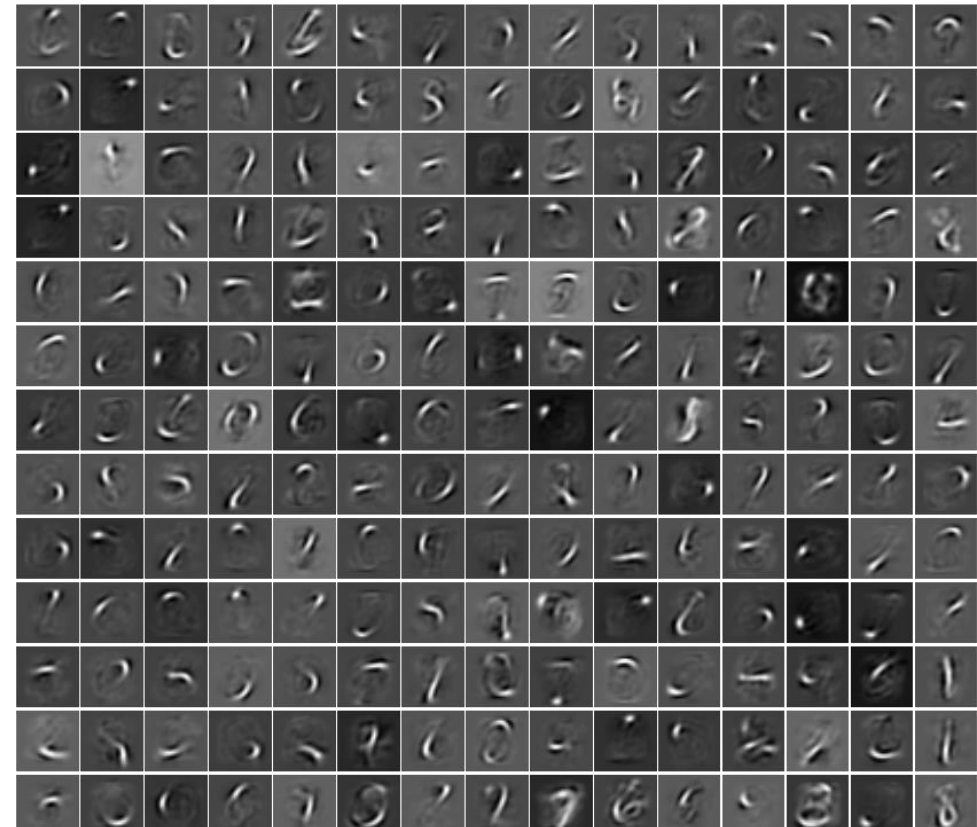
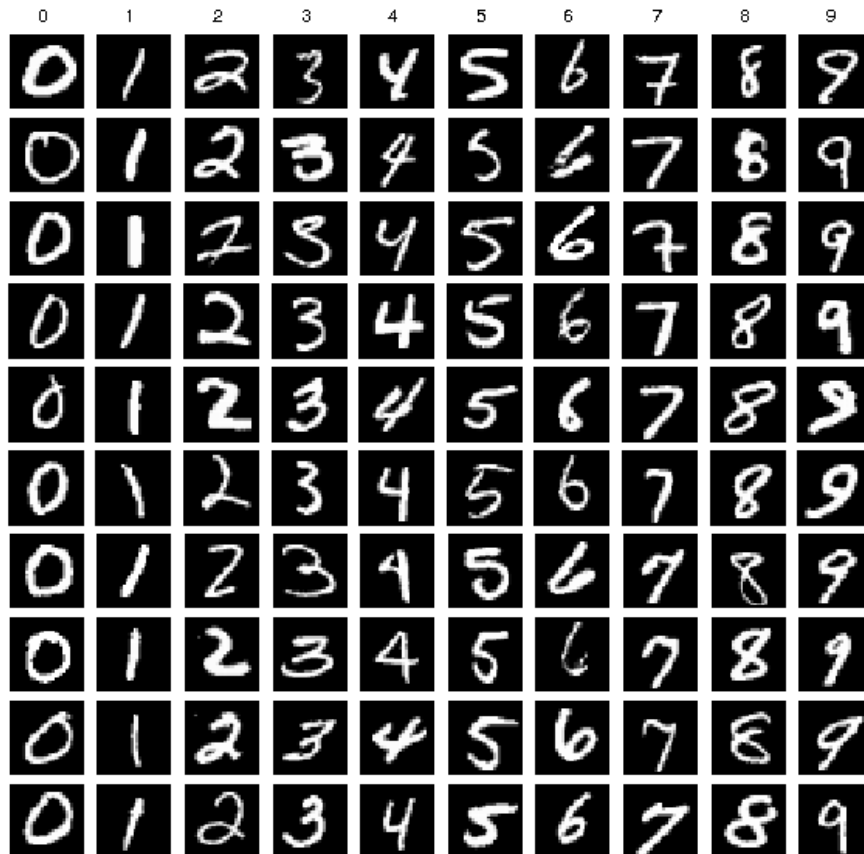
Autoencoders: variations

- Denoising autoencoder (overcomplete representations)



Popular Architectures

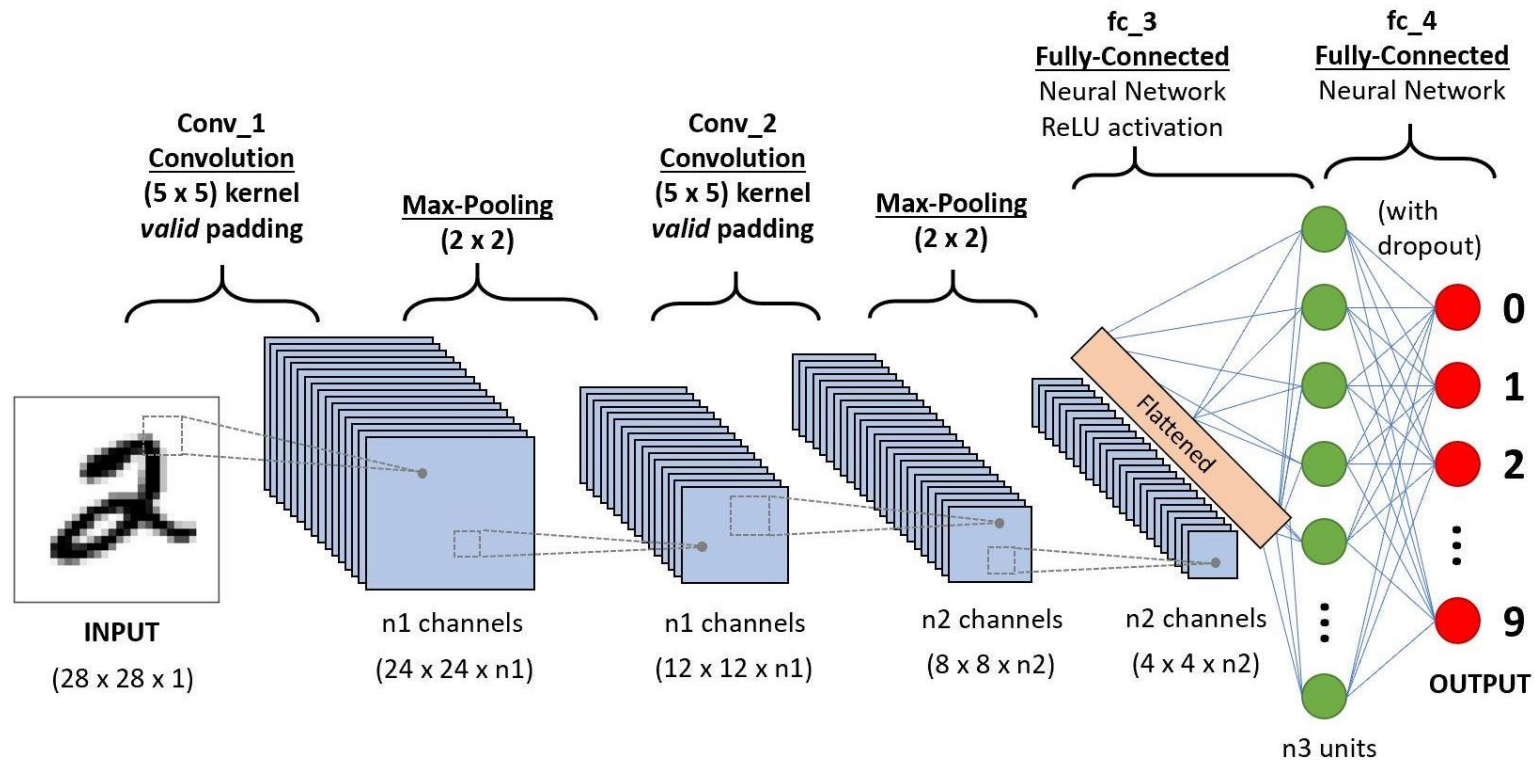
Autoencoders: features visualisation



Popular Architectures

Convolutional Neural Networks

- Feed forward neural network
- Analyse visual images by processing data with grid like topology



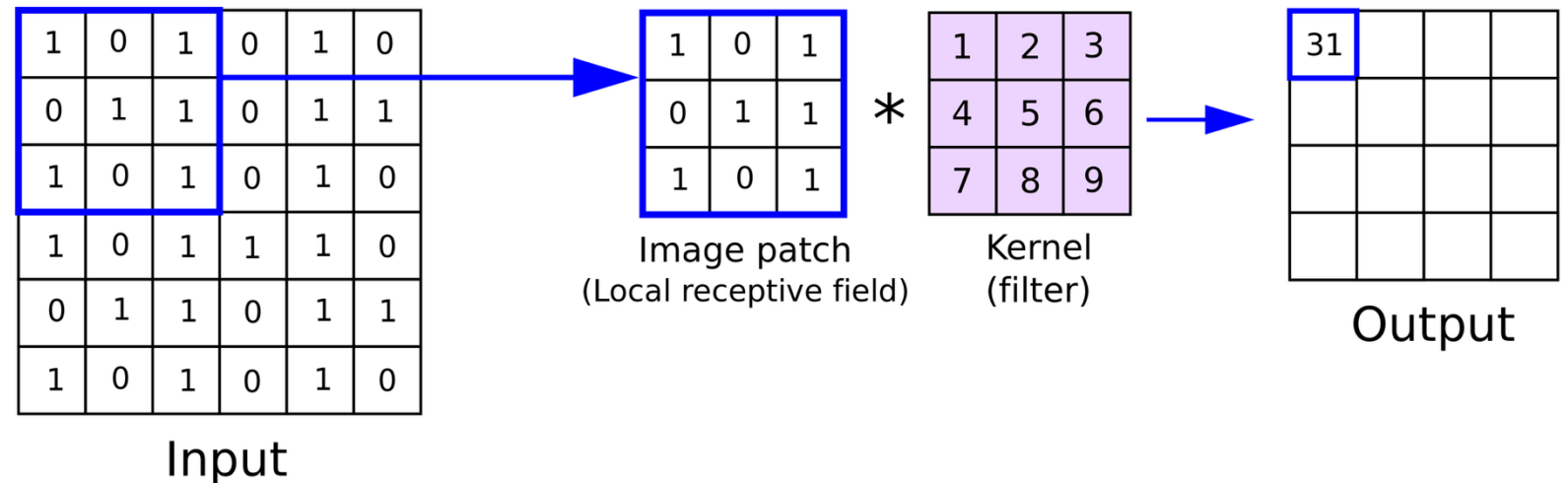
Popular Architectures

Convolutional Neural Networks: convolutional layers

- They apply multiple learnable filters to an input to extract local features.
- They transform the input into feature maps that highlight patterns like edges or textures useful for tasks such as image recognition

Parameters:

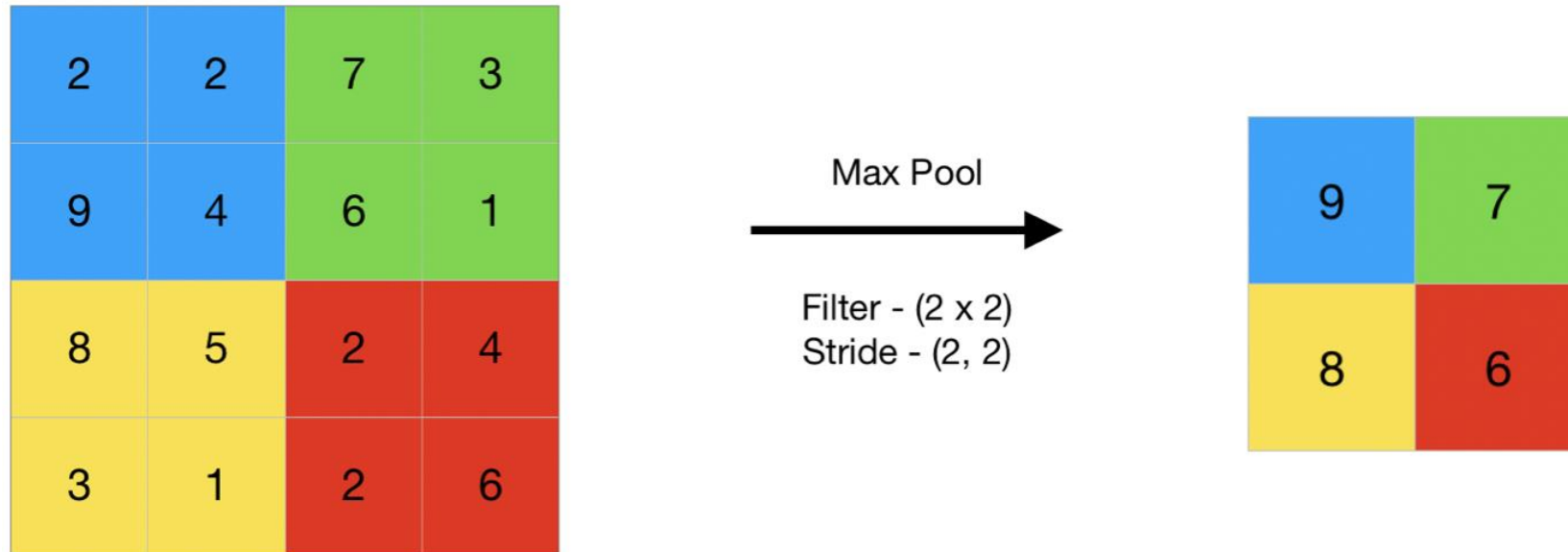
- Number of filters
- Size of filters
- Stride
- Padding



Popular Architectures

Convolutional Neural Networks: pooling layers

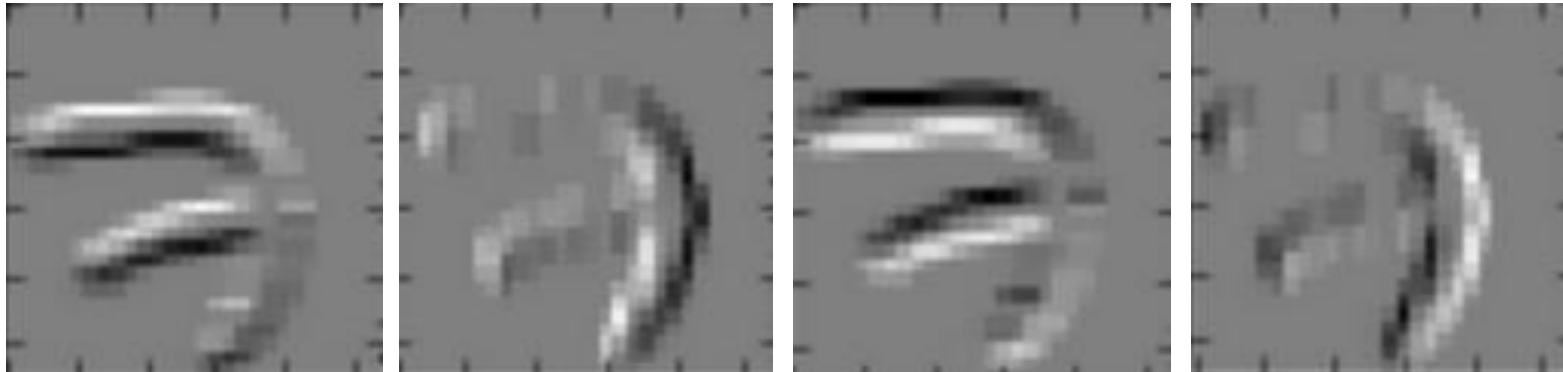
- They reduce the spatial dimensions of feature maps.
- They decrease computational complexity and increase scale invariance
- No trainable parameters



Popular Architectures

Convolutional Neural Networks: features visualisation

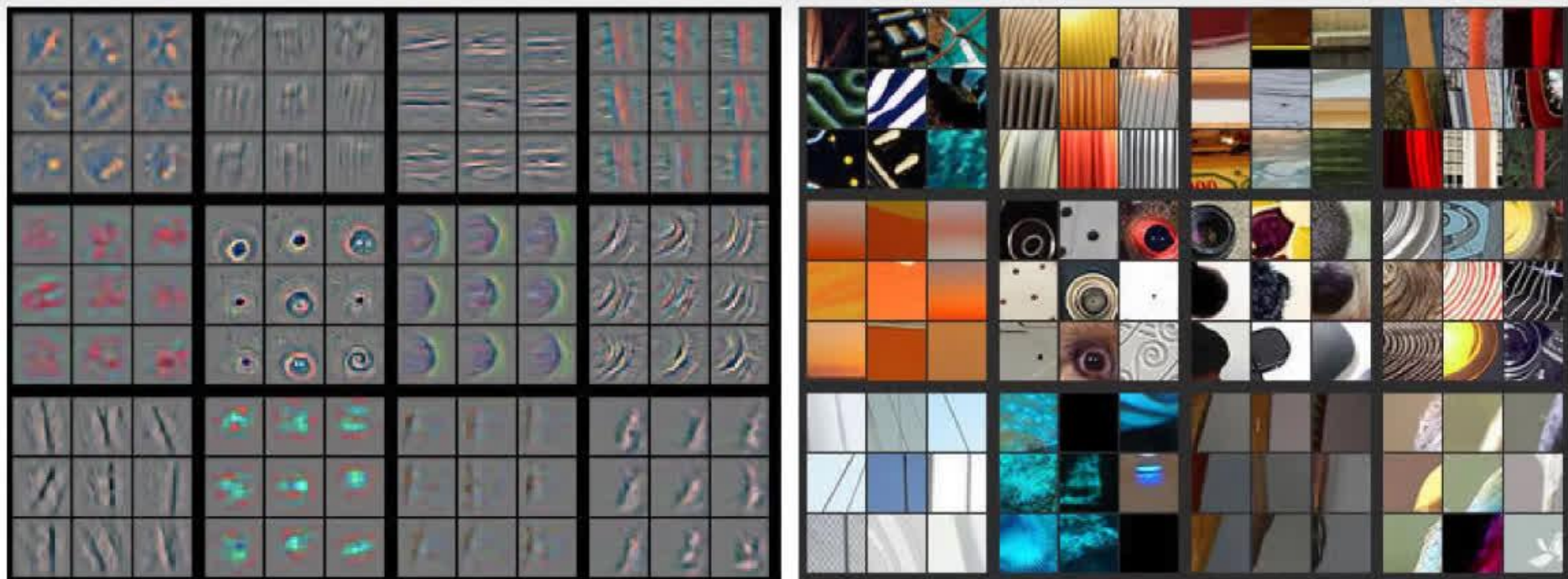
- First layers learn filters that extract basic image properties



Popular Architectures

Convolutional Neural Networks: features visualisation

- Intermediate layers learn filters that extract intermediate-level image properties



Popular Architectures

Convolutional Neural Networks: features visualisation

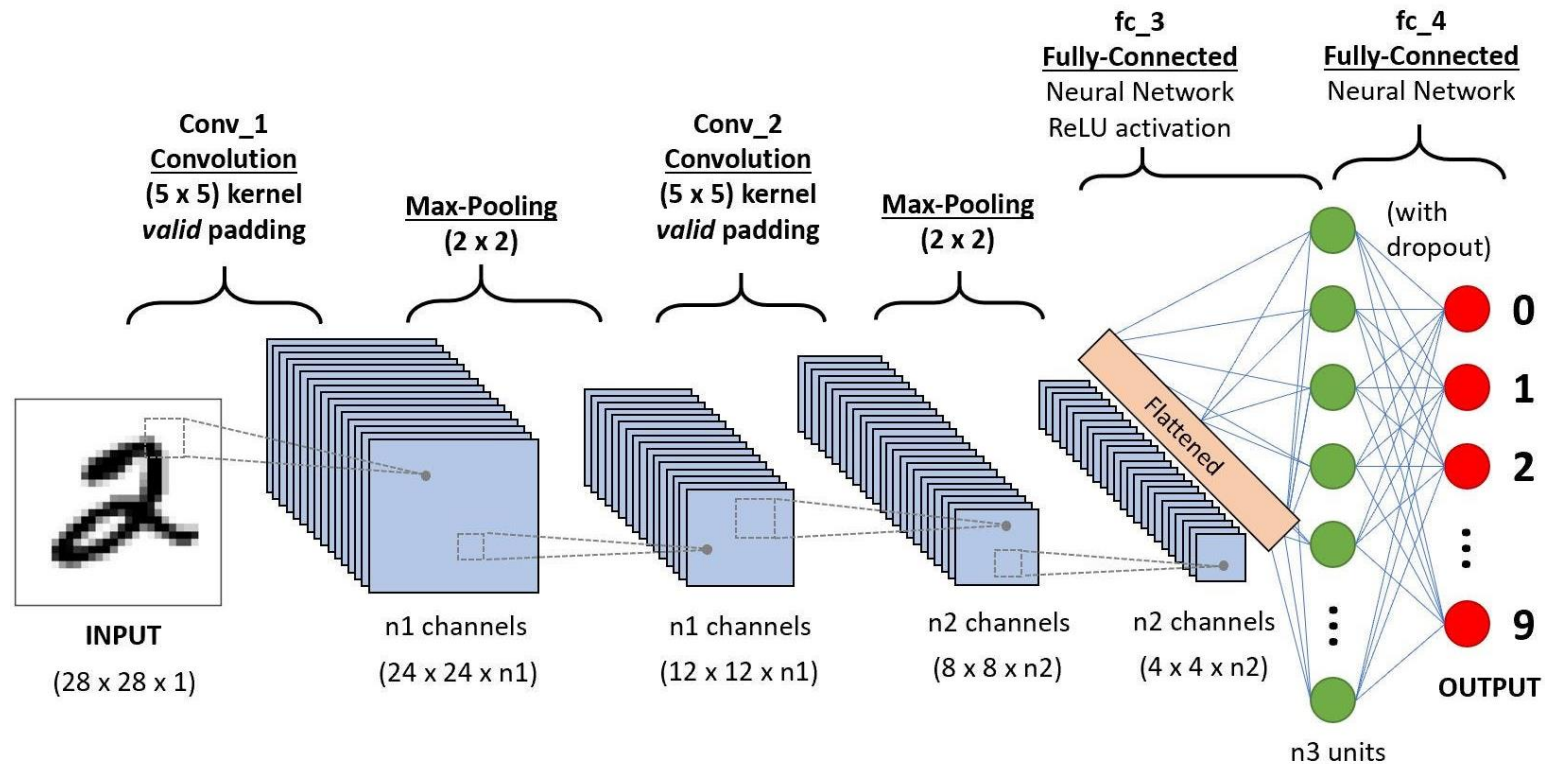
- Final layers learn kernels that extract high-level image properties



Popular Architectures

Convolutional Neural Networks: output

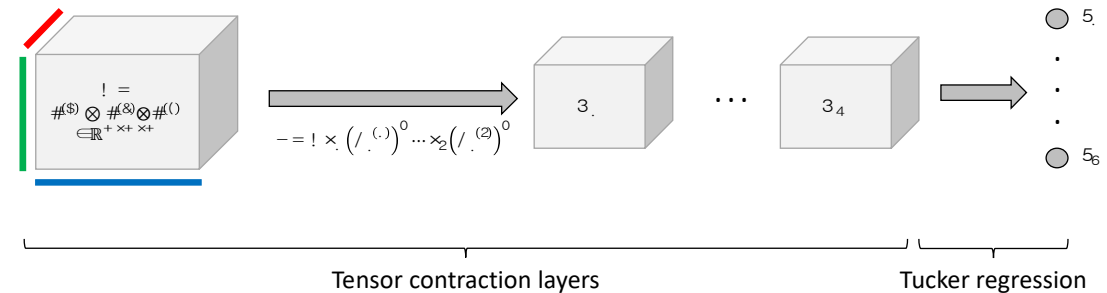
- The output of the last convolutional layer is transformed to a vector and used as input to a dense neural network



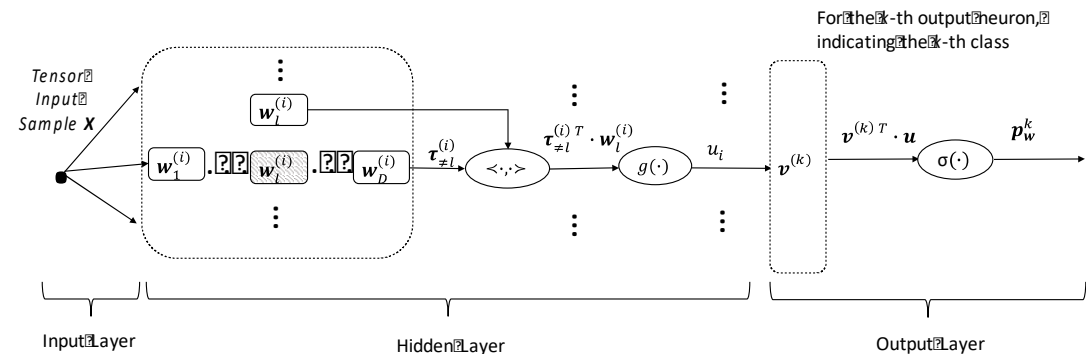
Popular Architectures

Tensor Neural Networks

- Tensor inputs
- Exploit the structure of the data
- Tensor algebra
- Small number of trainable parameters
- Few training data
- Exploit the structural information in the input
- No high-order convolutions



Tensor projections: Kossaifi, Jean, et al. "Tensor regression networks." *Journal of Machine Learning Research* 21.123 (2020): 1-21.



CP decomposition: Makantasis, Konstantinos, et al. "Tensor-based classification models for hyperspectral data analysis." *IEEE Transactions on Geoscience and Remote Sensing* 56.12 (2018): 6884-6898.

Popular Architectures

Long Short-Term Memory Networks (LSTMs)

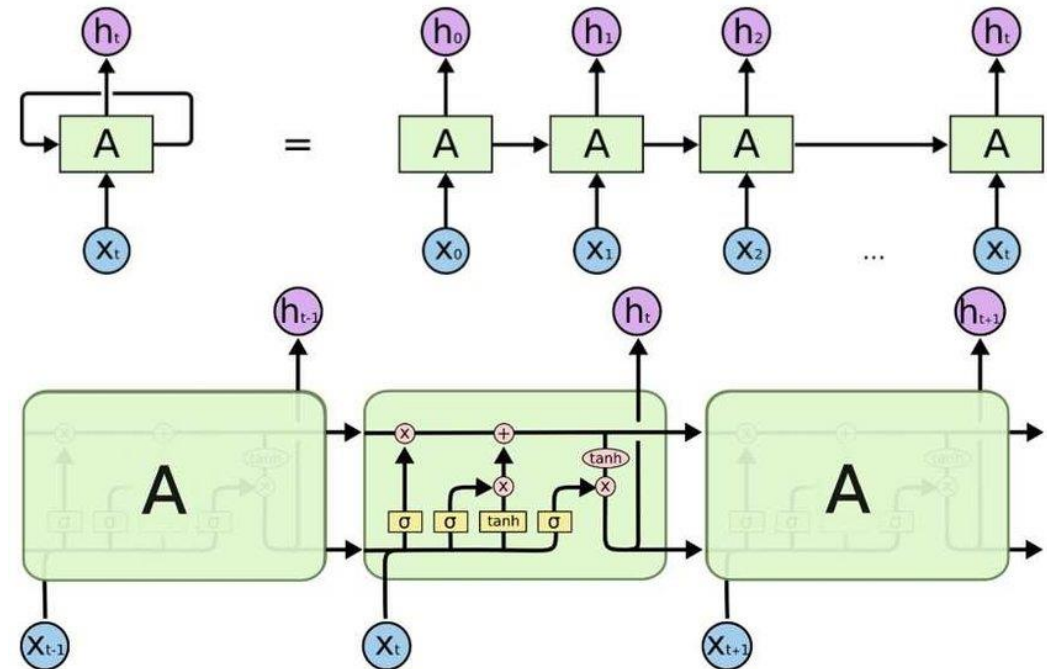
- Designed to learn from **sequential data** by maintaining a **memory** of previous inputs.

Key features:

Memory cells: Allow the network to remember information over long sequences.

Gates: Control what information is kept or forgotten.

Applications: Text, speech, music, time series, and more.



Popular Architectures

Long Short-Term Memory Networks (LSTMs)

- **Forget Gate** - decides what to throw away from memory

$$f_t = \sigma(W_f \cdot x_t + U_f \cdot h_{t-1} + b_f)$$

- **Input Gate** - decides what new information to store

$$i_t = \sigma(W_i \cdot x_t + U_i \cdot h_{t-1} + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot x_t + U_C \cdot h_{t-1} + b_C)$$

- **Update Cell** - combines old and new information

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- **Output Gate** - decides what to output

$$o_t = \sigma(W_o \cdot x_t + U_o \cdot h_{t-1} + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Popular Architectures

Long Short-Term Memory Networks (LSTMs): How a gate works

Consider, for example, the **forget** gate.

- It computes a vector of values between 0 and 1, one for each component of the cell state C_{t-1} .
- These values act like soft switches that decide how much of the past memory to keep.
 - If $f_{t,i} \approx 1$ keep the i-th memory element (don't forget).
 - If $f_{t,i} \approx 0$ erase the i-th memory element (forget).
 - Values in between $\rightarrow$ partial forgetting.

The forget gate learns over time which parts of the memory are useful and which are not, based on the current input x_t and past state h_{t-1} .

Popular Architectures

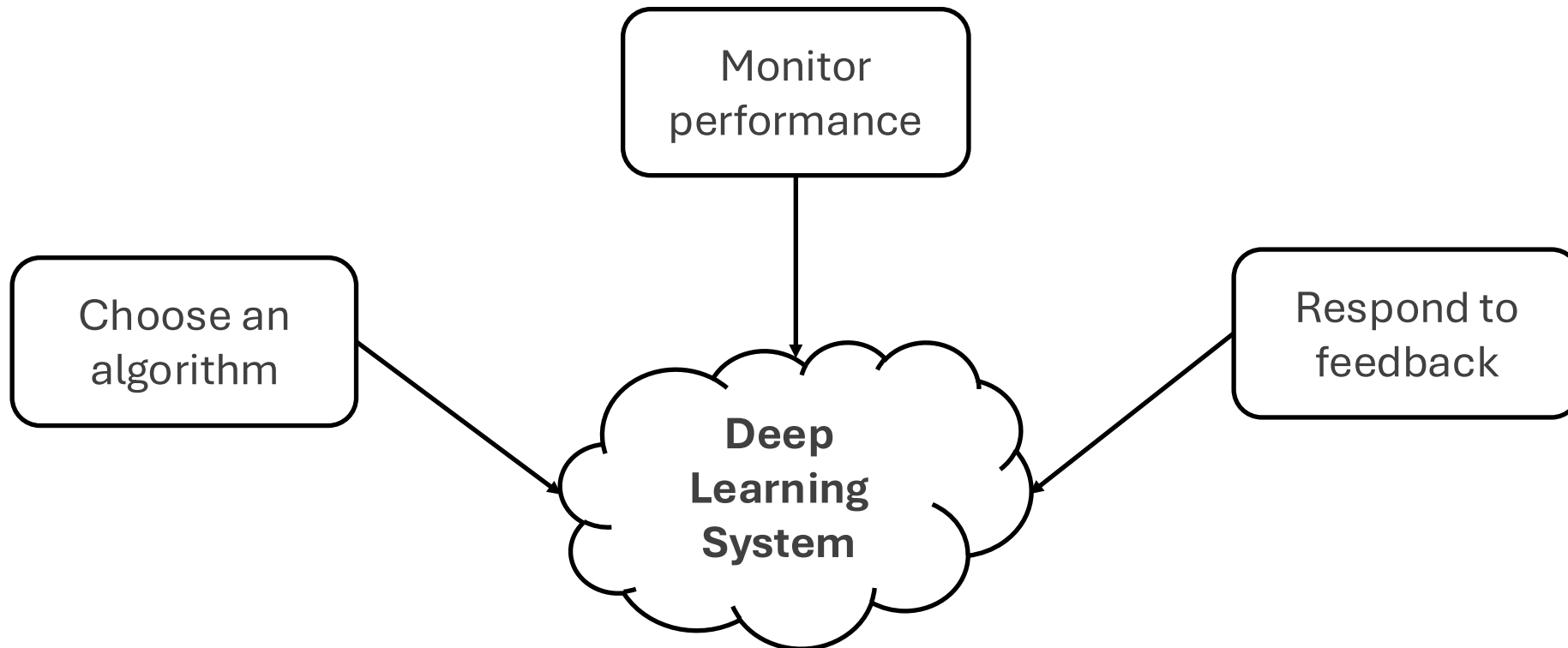
Long Short-Term Memory Networks (LSTMs): Outputs

The output of an LSTM can be:

- **A vector**
 - The network outputs the last hidden state h_T .
 - Many-to-one tasks (e.g., sentiment classification, where the whole input sequence maps to a single label).
- **A sequence**
 - The network outputs the full sequence of hidden states $[h_1, h_2, \dots, h_T]$
 - Many-to-many or sequence-to-sequence tasks.
 - Stack two or more LSTM layers in a network.

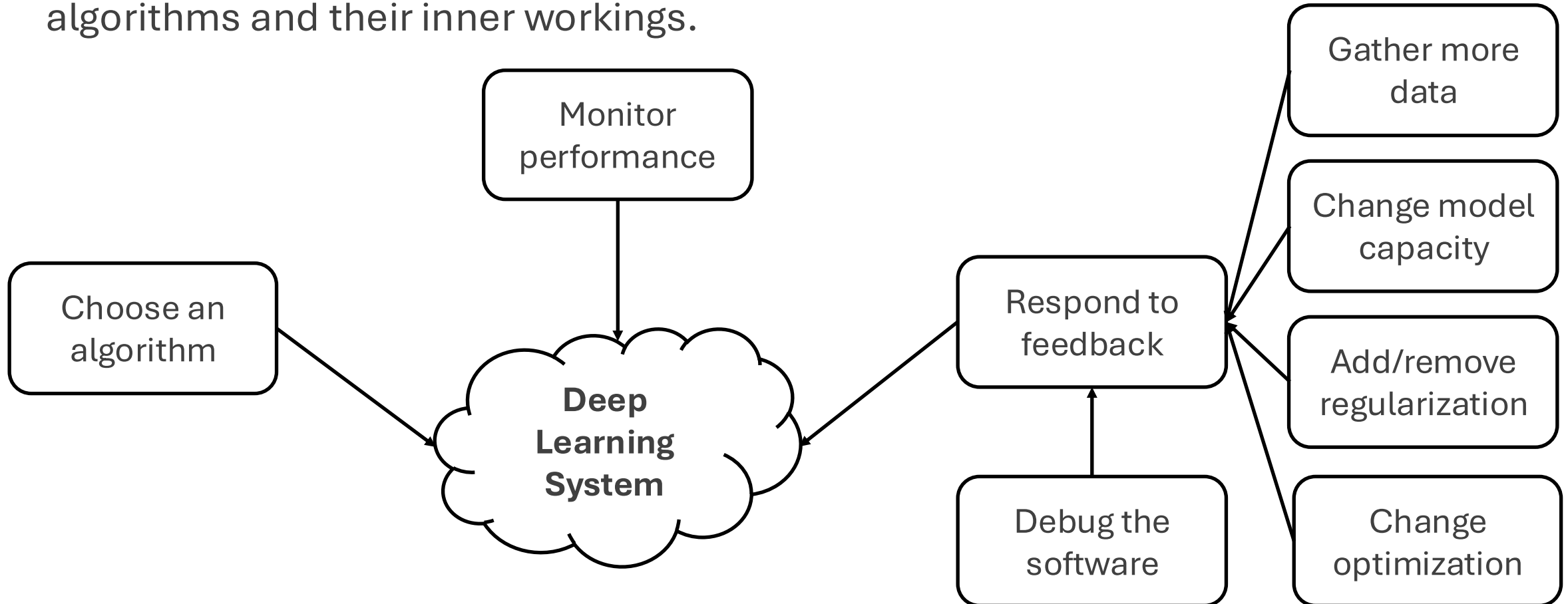
Practical Methodology

Successfully applying deep learning requires more than knowledge of the algorithms and their inner workings.



Practical Methodology

Successfully applying deep learning requires more than knowledge of the algorithms and their inner workings.

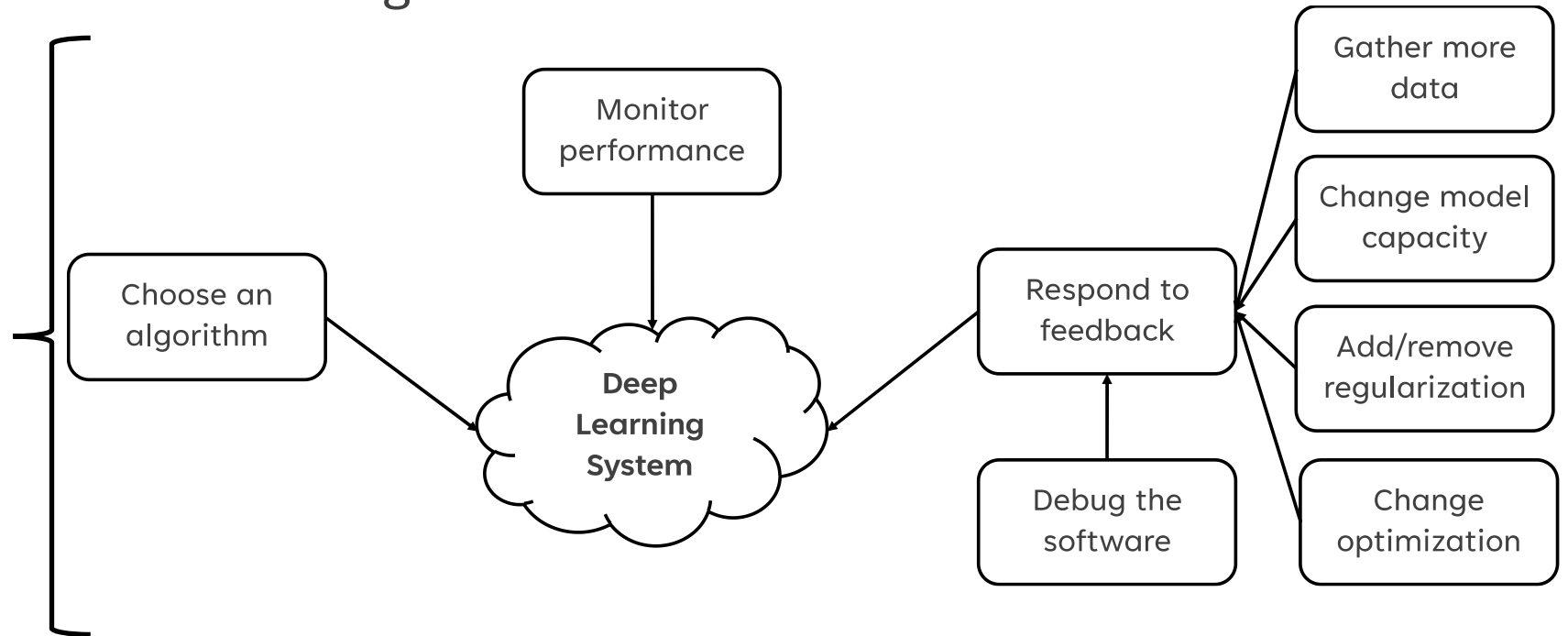


Practical Methodology

Successfully applying deep learning requires more than knowledge of the algorithms and their inner workings.

All these operations are

- Time consuming
- Costly



It is important to determine in advance the right course of action rather than blindly guessing

Practical Methodology

Practical Design Process

1. Determine your goals
 - What error metric to use
 - What the target value for that metric is
 - Metrics and values are driven by the application at hand
2. Establish a working baseline as soon as possible
 - End-to-end pipeline

Practical Methodology

Practical Design Process

3. Instrument the system well to determine bottlenecks
 - Which components are performing worse than expected?
 - Why? Overfitting, underfitting, defect in the data, software bug?
4. Make incremental changes based on findings from your instrumentation
 - Add more data
 - Adjust hyperparameters
 - Change gradient based optimization algorithm

Practical Methodology - Goals

Determining the goals is the **first necessary step**:

- The error metric will guide all future actions.
- An idea of the desired level of performance.

Zero error rate might be impossible

- Bayes error: minimum possible error
- Incomplete information about the target variable
- Intrinsically stochastic problems

Practical Methodology - Goals

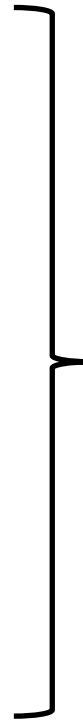
The error metric **is application dependent**

- Regression
 - MSE
 - MAE
 - R-squared
- Classification
 - Accuracy
 - Precision/Recall
 - Precision-Recall curve

Practical Methodology - Goals

The error metric **is application dependent**

- Regression
 - MSE
 - MAE
 - R-squared
- Classification
 - Accuracy
 - Precision/Recall
 - Precision-Recall curve



Determine the performance metric
ahead of time, then concentrate on
improving this metric.

Practical Methodology - Goals

Determining the **desired level** of performance

- **Academia**
 - Previous published studies
 - State-of-the-art benchmark results
- **Industry**
 - Safe
 - Cost-effective
 - Appealing to consumers

Practical Methodology - Baselines

Establish a **reasonable end-to-end** system:

- Choose a **general category** of a model based on the structure of the data
 - Vector input -> feedforward network
 - Topological structured input -> convolutional networks
 - Sequential input/output -> LSTMs, Transformers

Practical Methodology - Baselines

Establish a **reasonable end-to-end** system:

- Choose a **general category** of a model based on the structure of the data
 - Vector input -> feedforward network
 - Topological structured input -> convolutional networks
 - Sequential input/output -> LSTMs, Transformers
- Begin with **piecewise linear** activation functions
 - ReLUs and their generalizations

Practical Methodology - Baselines

Establish a **reasonable end-to-end** system:

- Choose a **general category** of a model based on the structure of the data
 - Vector input -> feedforward network
 - Topological structured input -> convolutional networks
 - Sequential input/output -> LSTMs, Transformers
- Begin with **piecewise linear** activation functions
 - ReLUs and their generalizations
- Choose an **optimization** algorithm
 - SGD with momentum and decaying learning rate
 - Adam

Practical Methodology - Baselines

Establish a **reasonable end-to-end** system:

- If the dataset is **not** large enough, then
 - Add mild forms of regularization (dropout)
 - Early stopping should be used almost universally

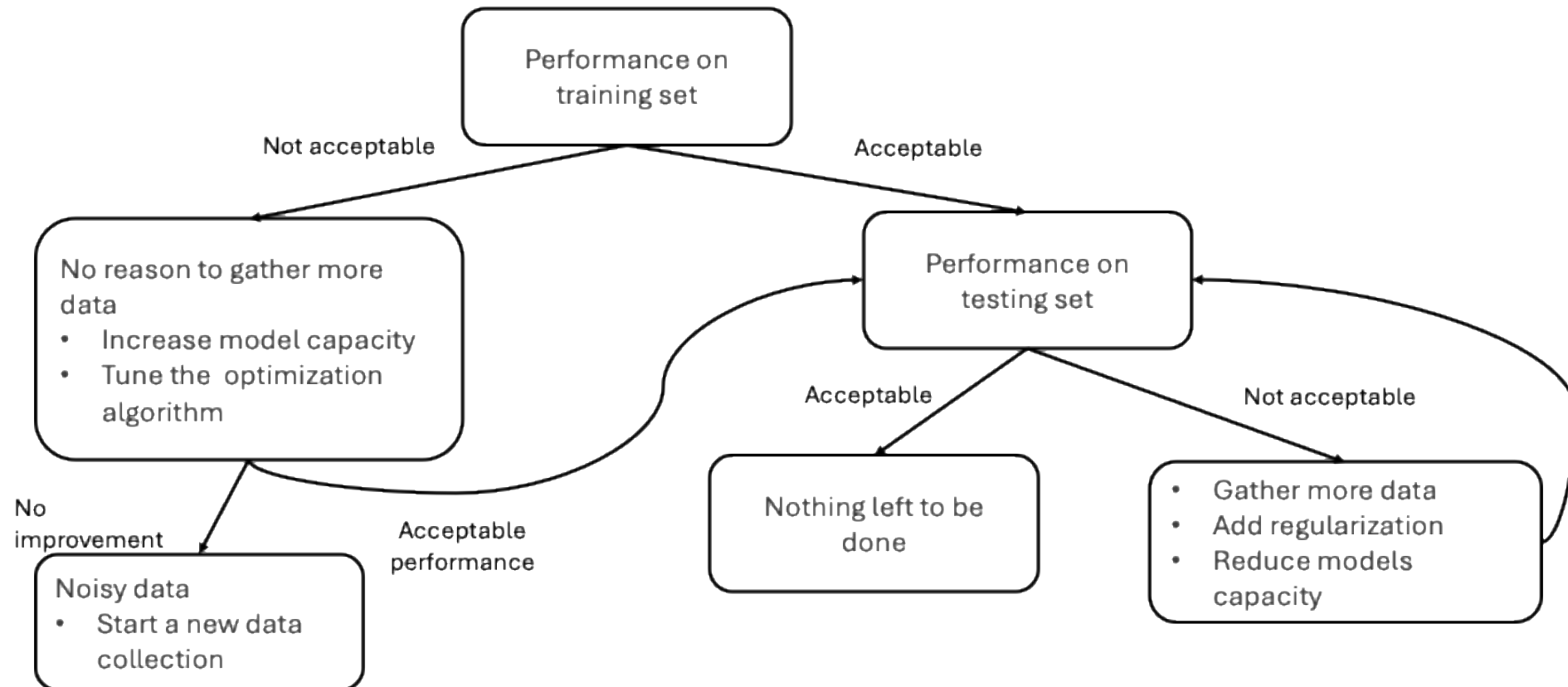
Practical Methodology - Baselines

Establish a **reasonable end-to-end** system:

- If the dataset is **not** large enough, then
 - Add mild forms of regularization (dropout)
 - Early stopping should be used almost universally
- If the task is similar to another task that has been **studied extensively** then
 - Start by copying the model and algorithm that is known to perform best
 - State-of-the-art methodology

Practical Methodology - Data

Determining whether to **gather** more data



Practical Methodology - Hyperparameters

Deep learning algorithms come with many hyperparameters that **control their behaviour**. Hyperparameters also affect:

- Performance
- Memory cost of the algorithm
- Time for running the algorithm

Practical Methodology - Hyperparameters

Deep learning algorithms come with many hyperparameters that **control their behaviour**. Hyperparameters also affect:

- Performance
- Memory cost of the algorithm
- Time for running the algorithm

} Hyperparameters should be set
based on **computation budget**

Practical Methodology - Hyperparameters

Deep learning algorithms come with many hyperparameters that **control their behaviour**. Hyperparameters also affect:

- Performance
 - Memory cost of the algorithm
 - Time for running the algorithm
- } Hyperparameters should be set based on **computation budget**

Two different approaches for setting the hyperparameters

- **Manual** hyperparameter setting
 - Understand what each hyperparameter does
- **Automatic** hyperparameter setting
- Much more computationally costly

Practical Methodology - Hyperparameters

Manual hyperparameter setting requires understanding the interrelationship between:

- Hyperparameters
- Training error
- Generalization error
- Computational resources

Goal: find the lowest generalization error subject to some runtime and memory budget

Practical Methodology - Hyperparameters

Automatic hyperparameter setting does not require understanding what each hyperparameter does, but it is **computationally intensive**.

Grid search:

1. For each hyperparameter select a small finite set to explore
2. Train a model for each combination of hyperparameter values
3. The best hyperparameters correspond to the model that achieves the best validation performance

Practical Methodology - Hyperparameters

Grid search: How to select a small finite set to explore

- Select the values conservatively
 - Based on prior experience
 - Similar experiments
- Typically, use a logarithmic scale
 - Learning rate {0.1, 0.01, 0.001, 0.0001}
 - Number of hidden units {50, 100, 200, 400, 800, 1600}

The computational cost **grows exponentially** with the number of parameters.

Practical Methodology - Debugging

Deep learning systems are **difficult to debug**

- We don't know a priori what the intended behaviour is.
- Multiple components that are adaptive

What can we do?

- Test parts of the deep learning system in isolation
- Develop simple tests
- Check the gradients
- Very simple input/output pairs

Practical Methodology - Debugging

Debugging strategies:

1. Visualize the model in action
 - Manually evaluate model's predictions for specific inputs
 - Determine whether the quantitative performance numbers are reasonable
2. Visualise the worst mistakes
 - Incorrect predictions with high confidence
 - Hard examples can reveal problems with the data
3. Reasoning about software using train and test error
 - Errors in models saving or no learning at all
4. Fit a tiny dataset